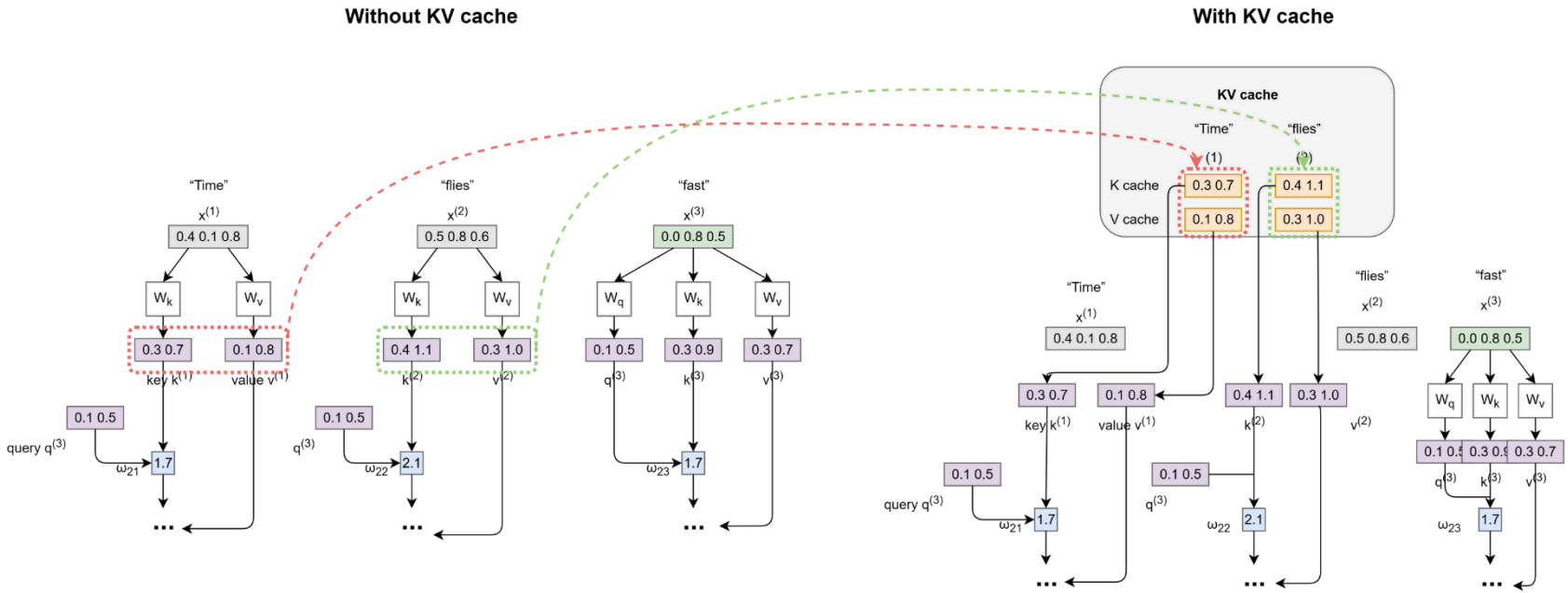
LLM 모델의 이해와 확장 — 2부

LLM 효율화

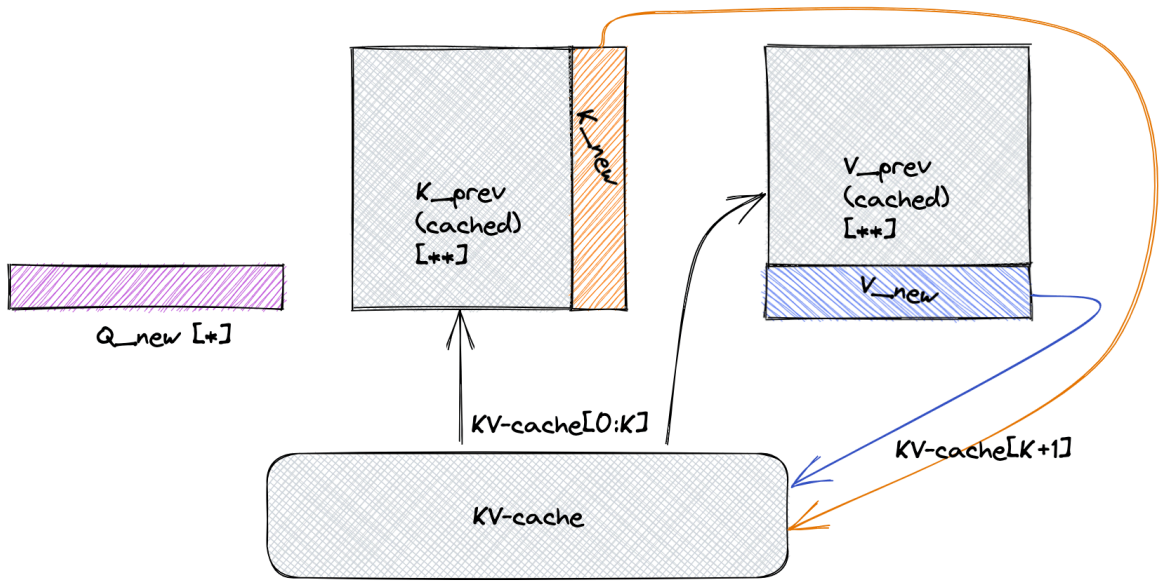
KV Caching

계산 효율화, 메모리 증가



KV Caching

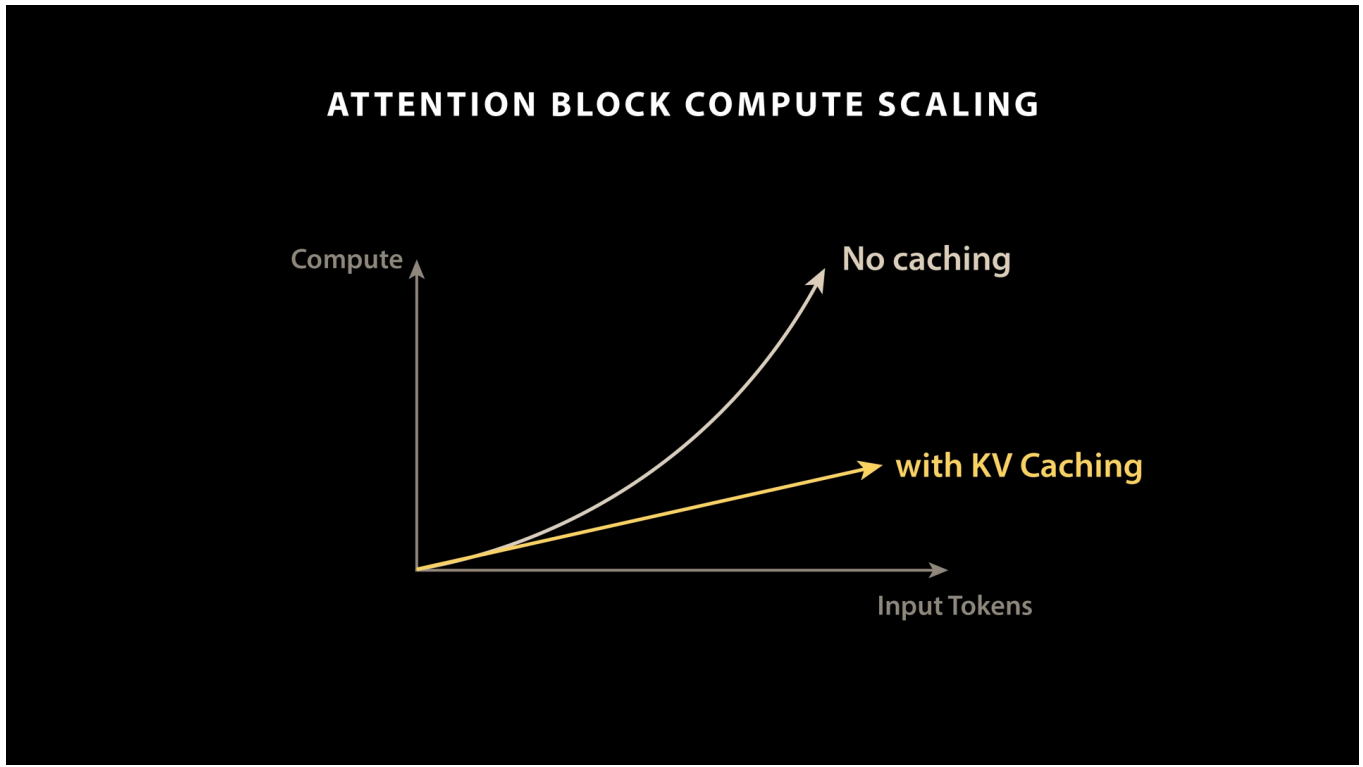
> *회색은 캐싱되어 있으니, 색깔이 있는 부분만 새로 계산을 하면 됨*



Notes:
---
* When processing token[K], we only need the K'th row of Q
** When processing token[K], we require the full K & V tensors, but we can mostly reuse the cached values
(This enables skipping the computation of K & V

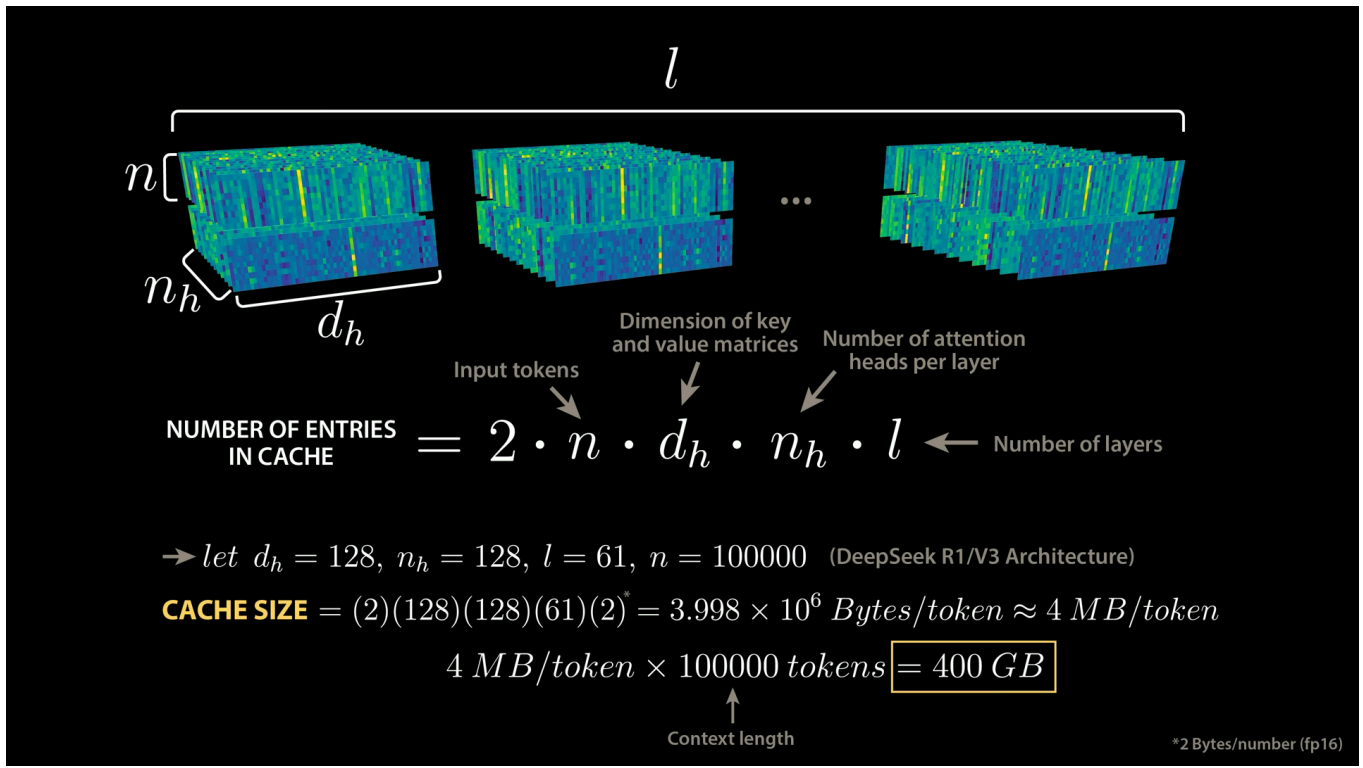
KV Caching

> *Attention 연산량이 확연하게 줄어듦*

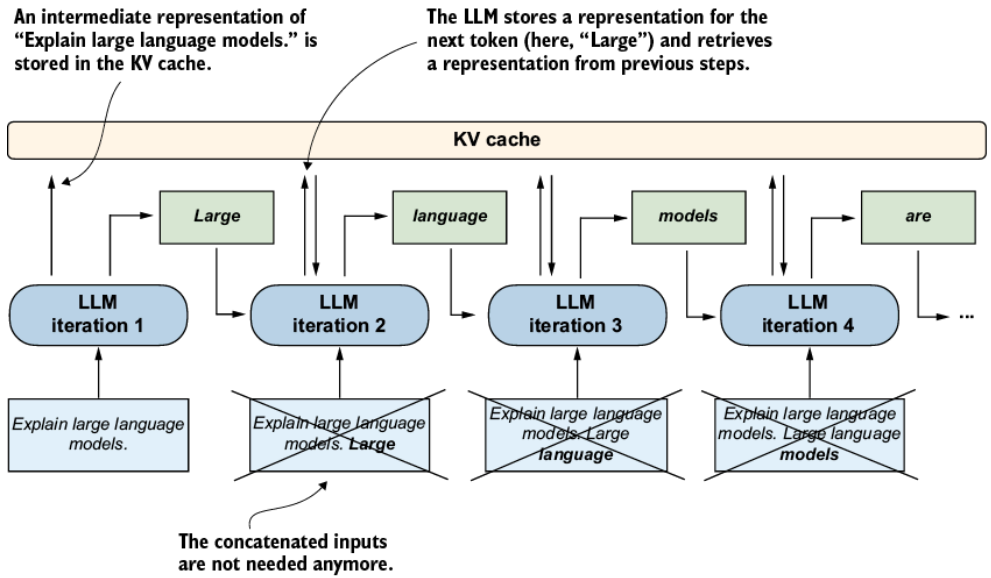


KV Caching

> 메모리가 증가함



KV cache: 과거 계산을 저장해 재사용



- 왼쪽 첫 실행: 전체 입력의 key·value 저장
- 이후 실행: 새 token 하나 + 저장한 과거 상태 사용

출처: 『Build a Reasoning Model (From Scratch)』 그림 2-16

KV cache 적용: 전체 입력은 처음 한 번만

```
from reasoning_from_scratch.qwen3 import KVCache
@torch.inference_mode()
def generate_text_basic_stream_cache(model, token_ids, max_new_tokens,
                                     eos_token_id=None):
    model.eval()
    cache = KVCache(n_layers=model.cfg["n_layers"])
    model.reset_kv_cache()
    out = model(token_ids, cache=cache)[:, -1]
    for _ in range(max_new_tokens):
        next_token = torch.argmax(out, dim=-1, keepdim=True)
        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break
```

1. 새 cache를 만들고 전체 입력을 한 번 처리한다
2. 가장 높은 점수의 ID를 고른다
3. EOS를 확인한 뒤 ID를 내보낸다
4. 다음 실행은 새 token만 전달한다

최적화 효과는 장치와 구현에 따라 달라진다

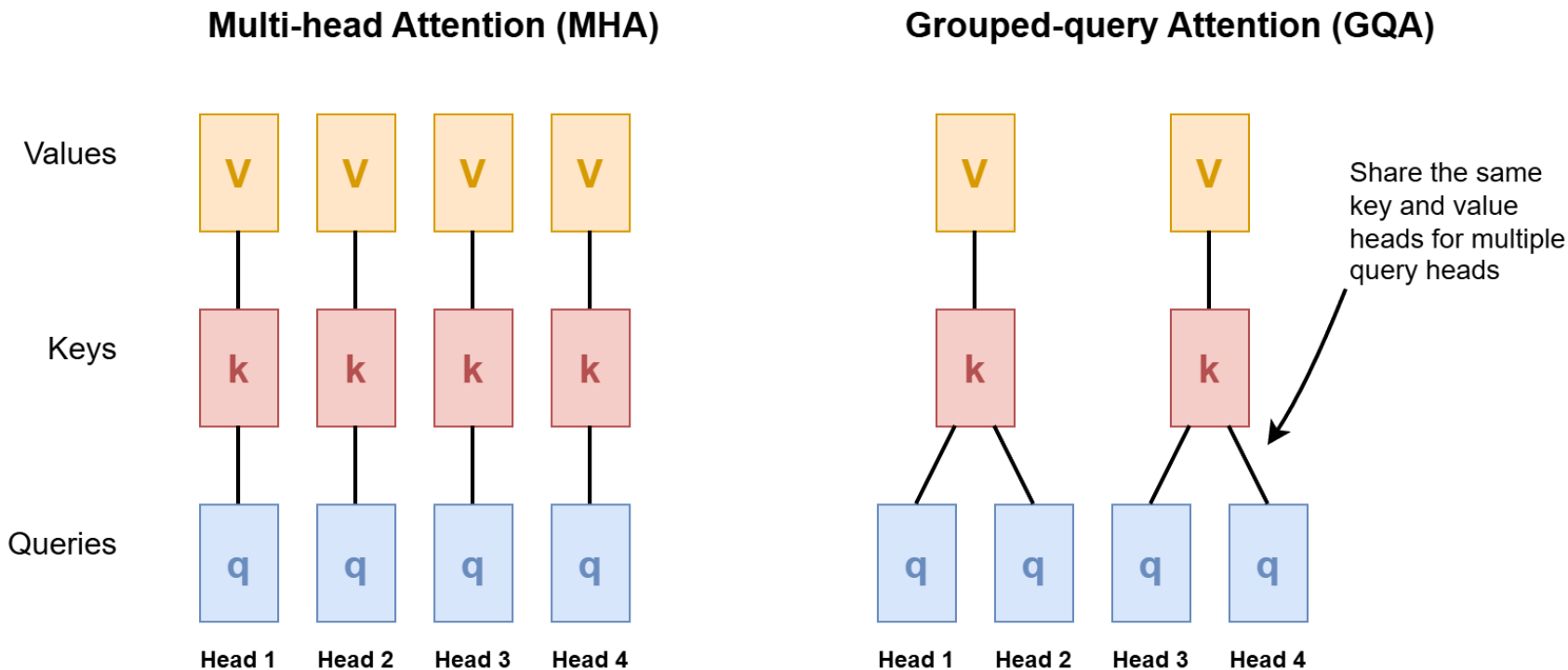
방식	M4 CPU	M4 GPU	H100 GPU	H100 메모리
기본	5	27	51	1.55 GB
기본 + 컴파일	6	43	164	1.81 GB
KV cache	28	41	48	1.52 GB
KV cache + 컴파일	68	71	141	1.81 GB

단위: token/초 · 동일한 단일 입력 · 원문 표 2.1

- H100에서도 이 구현의 KV cache가 항상 더 빠르지는 않다
- 연산이 뛰어난 경우는 KV cache보다 직접 계산하는 것이 빠를 수 있다.
- 동적으로 커지는 cache를 처리하기 위해서는 메모리와 계산 속도를 고려해야 한다.

GQA

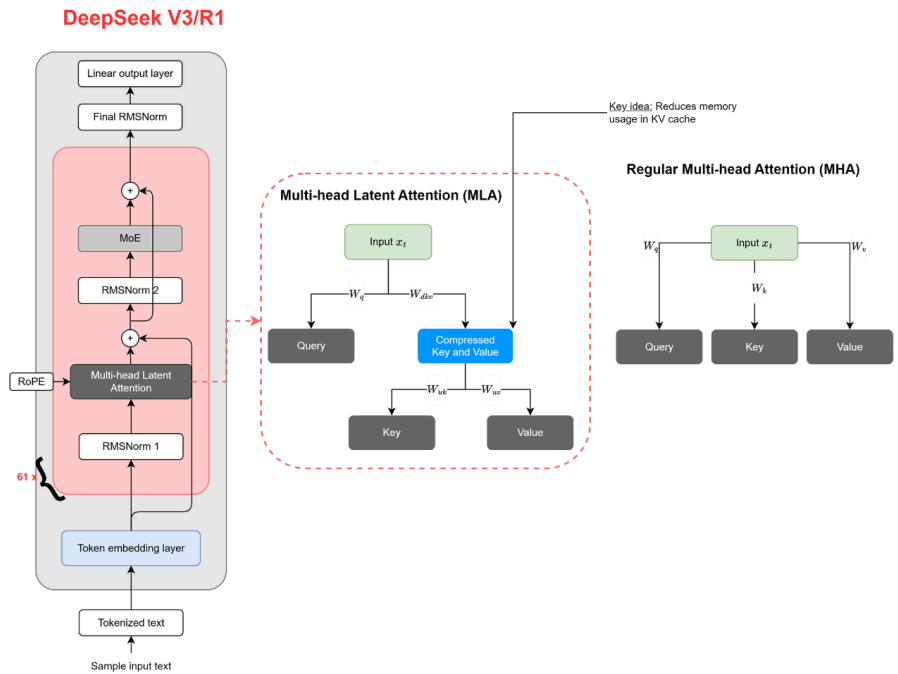
K,V를 그룹화


Multi-head Attention (MHA)
Grouped-query Attention (GQA)
Values
Keys
Queries
v
k
q
Head 1
Head 2
Head 3
Head 4
Share the same key and value heads for multiple query heads

MLA

KV에 대한 Latent Vector를 생성

- 더 큰 배치 사이즈, 긴 문맥 유지
- Latent 계산 비용 < 메모리 비용
- Latent Vector를 모든 헤드가 공유



MLA: Latent KV Cache

> *K,V를 그대로 캐시하지 않고, 압축된 Latent를 캐시*

일반 Attention

$$K = XW_K, \quad V = XW_V$$

$$\text{Cache} = (K_1, V_1), \ldots, (K_{n_h}, V_{n_h})$$

- Head마다 K,V를 저장
- Head 수가 많을수록 KV Cache 증가
- 매 토큰마다 Head별 K,V 전체를 HBM에서 읽음

MLA

$$L_{KV} = XW_{DKV}$$ ① 입력을 저차원 Latent로 압축

$$K = L_{KV}W_{UK}, \quad V = L_{KV}W_{UV}$$ ② Latent에서 K, V를 복원

$$\text{Cache} = L_{KV}$$ ③ 캐시에는 Latent만 저장

- 공유 Latent만 저장
- Head별 Projection으로 표현력 유지
- 매 토큰마다 작은 공유 Latent만 HBM에서 읽음

MLA: 추가 Compute를 없애는 재배열

> *연산량은 그대로거나 오히려 줄고, HBM에서 읽는 데이터는 공유 Latent뿐이게 만드는 것이 핵심*

Key 쪽 계산

$$QK^T = Q(L_{KV}W_{UK})^T$$

$$= QW_{UK}^T L_{KV}^T = Q' L_{KV}^T$$

$$(Q' = QW_{UK}^T)$$

- K를 복원하지 않음
- Query를 Latent-Key 공간으로 보냄

Value 쪽 계산

$$O = AV = A(L_{KV}W_{UV})$$

$$= (AL_{KV})W_{UV}$$

- V를 복원하지 않음
- Latent에 Attention을 먼저 적용
- W_{UV}는 Output Projection 쪽에 결합

MLA: KV Cache 크기 비교

> *추론 시점에 저장하는 것은 Head별 K,V가 아니라 공유 Latent*

Scaling

$$\text{MHA Cache} \propto 2 \times n \times d_h \times n_h \times L$$

$$\text{MLA Cache} \propto n \times d_{latent} \times L$$

- n_h: Attention Head 수
- d_{latent}: 공유 Latent 차원
- 핵심: Head 수에 비례해 커지는 부분을 제거

DeepSeek R1/V3 기준

방식	토큰당 KV Cache	성능 특성
MHA	4 MB	높음
GQA	500 KB	중간
MLA	70 KB	더 높음

- MLA는 MHA 대비 약 57배 감소
- 전통적 Transformer 대비 생성 속도 6배 이상
- 더 긴 Context와 더 큰 Batch에 유리

MLA

> *성능은 높고, 메모리는 적게 차지함*

Unlike what previous papers found, this paper finds that MHA performs much BETTER than GQA

Benchmark (Metric)	# Shots	Dense 7B w/ MQA	Dense 7B w/ GQA (8 Groups)	Dense 7B w/ MHA
# Params	-	7.1B	6.9B	6.9B
BBH (EM)	3-shot	33.2	35.6	**37.0**
MMLU (Acc.)	5-shot	37.9	41.2	**45.2**
C-Eval (Acc.)	5-shot	30.0	37.7	**42.9**
CMMLU (Acc.)	5-shot	34.6	38.4	**43.5**

Table 8 | Comparison among 7B dense models with MHA, GQA, and MQA, respectively. MHA demonstrates significant advantages over GQA and MQA on hard benchmarks.

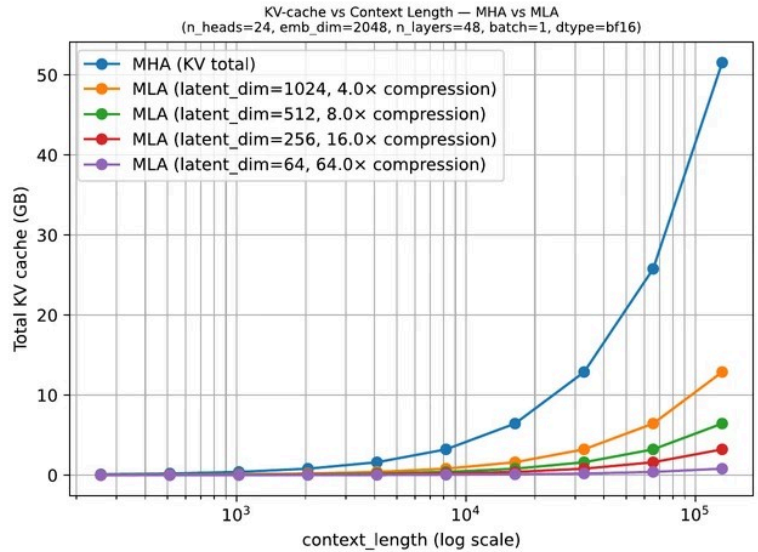
Benchmark (Metric)	# Shots	Small MoE w/ MHA	Small MoE w/ MLA	Large MoE w/ MHA	Large MoE w/ MLA
# Activated Params	-	2.5B	2.4B	25.0B	21.5B
# Total Params	-	15.8B	15.7B	250.8B	247.4B
KV Cache per Token (# Element)	-	110.6K	15.6K	860.2K	34.6K
BBH (EM)	3-shot	37.9	**39.0**	46.6	**50.7**
MMLU (Acc.)	5-shot	48.7	**50.0**	57.5	**59.0**
C-Eval (Acc.)	5-shot	**51.6**	50.9	57.9	**59.2**
CMMLU (Acc.)	5-shot	52.3	**53.4**	60.7	**62.5**

Table 9 | Comparison between MLA and MHA on hard benchmarks. DeepSeek-V2 shows better performance than MHA, but requires a significantly smaller amount of KV cache.

The memory requirements for MLA are much lower than for MHA

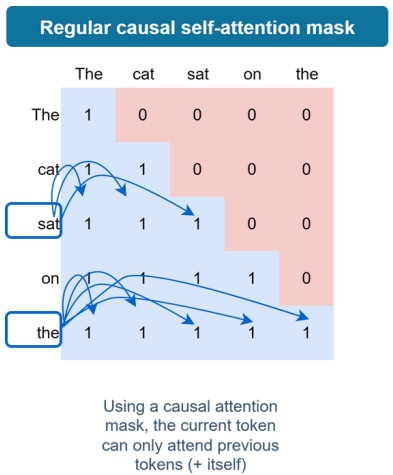
MLA performs better than MHA (here tested on Mixture-of-Experts architectures)

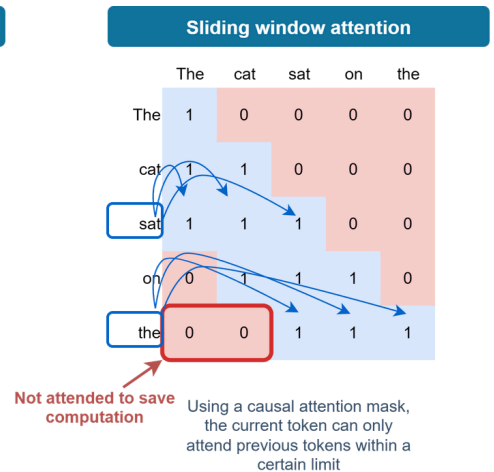
Annotated tables from the DeepSeek-V2 paper, https://arxiv.org/abs/2405.04434

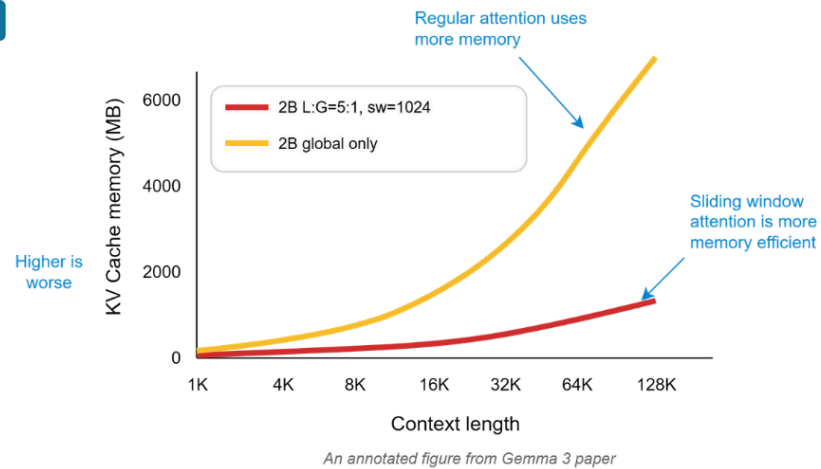


Sliding window attention

> *일정 길이 만큼의 이전 입력만 바라봄*



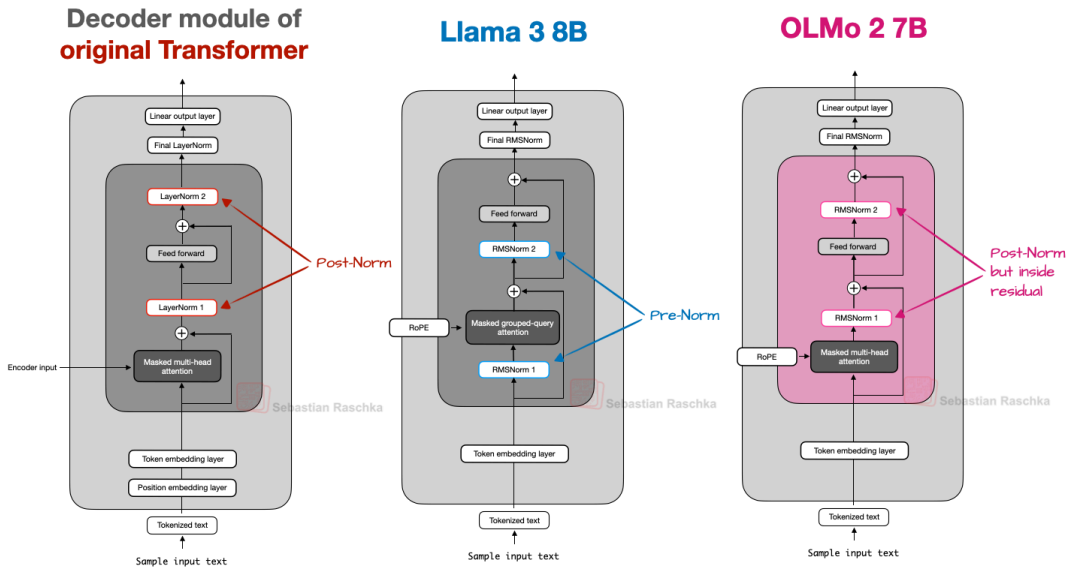




An annotated figure from Gemma 3 paper

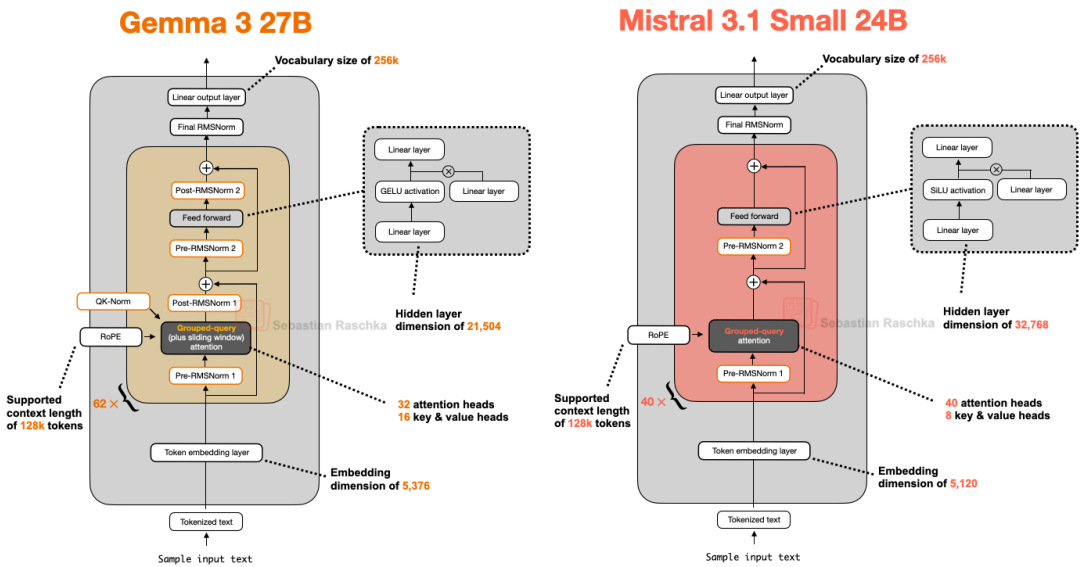
정규화 레이어 배치

> *Pre-Norm을 많이 하는 추세, But 아직은 잘 모름*



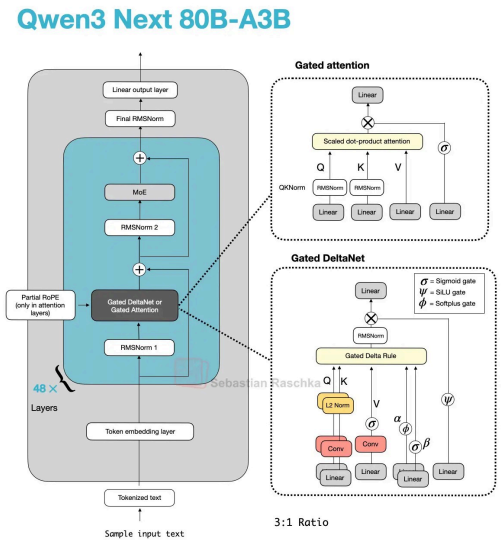
넓이를 키울 것인가? 깊이를 키울 것인가?

> *넓으면 학습이 좀 더 안정적이고 빠름, 깊으면 성능이 더 유연적인 사고를 함*



Gated DeltaNet

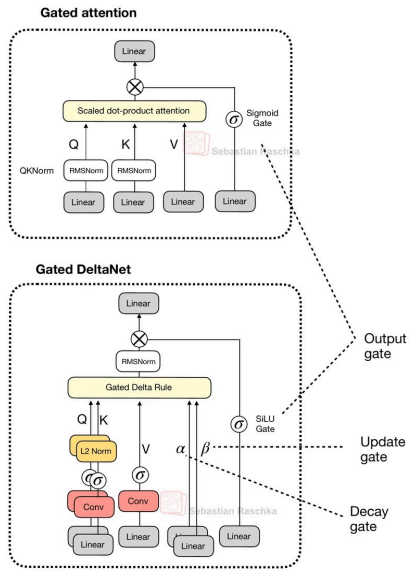
> *Qwen3-Next와 Kimi Linear에서 사용하는 Attention 메카니즘*



- 어떤 정보가 중요하며, 얼마나 통과시킬지"를 동적으로 결정하는 필터
- 선형으로 계산

Gated DeltaNet

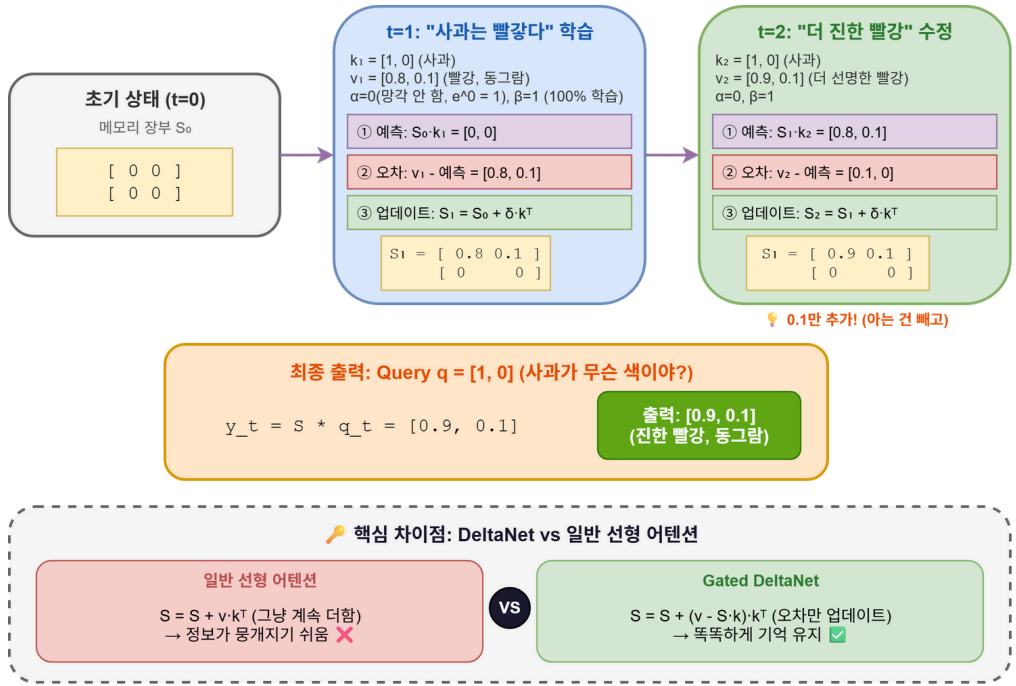
> *Update Gate, Decay Gate, Output Gate를 통해 어떤 정보를 전달할지 결정*



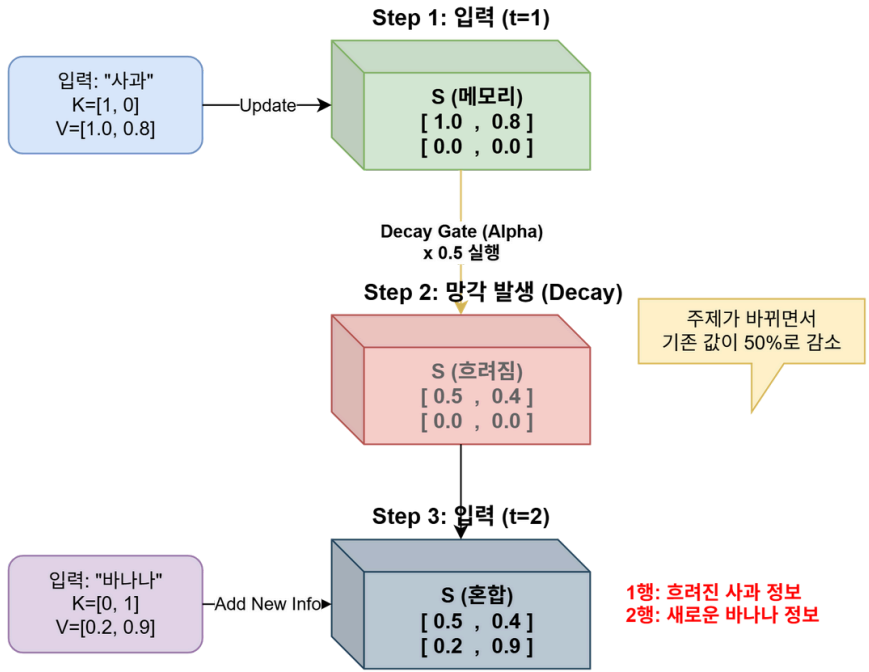
- Q와 K를 L2 정규화하여 수치적 안정성을 확보
- 현재 토큰을 기준으로 앞뒤 1~2개 토큰의 정보를 살짝 섞어주는 역할

- Output Gate:어떤 정보가 중요하며, 얼마나 통과시킬지"를 동적으로 결정하는 필터
- Update Gate: 현재 들어온 새로운 정보를 메모리에 얼마나 강하게 반영할지 결정
- Decay Gate: 과거의 정보를 얼마나 유지하고 얼마나 잊어 버릴지를 결정

Gated DeltaNet: 원리

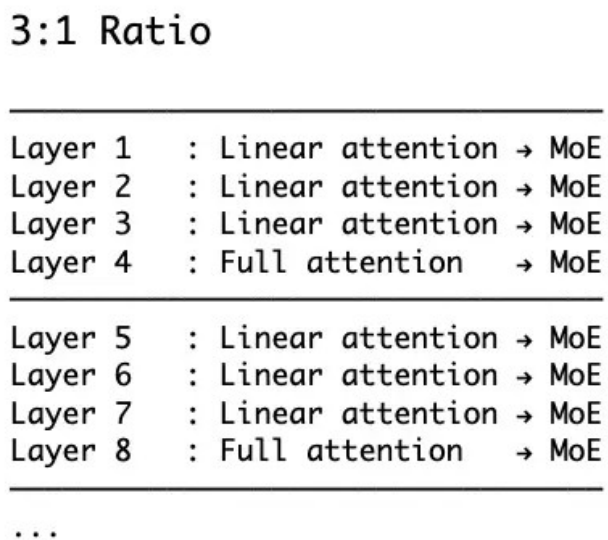


Gated DeltaNet: Decay Gate



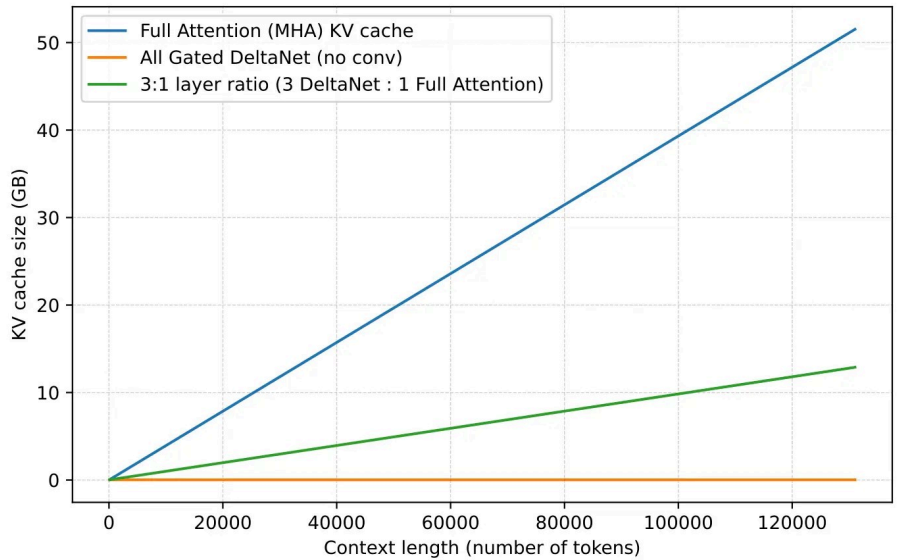
Gated DeltaNet

> *Gated DeltaNet(Linear attention)과 Gated Attention(Full attention)의 hybrid 구조*



Gated DeltaNet: 메모리 절약

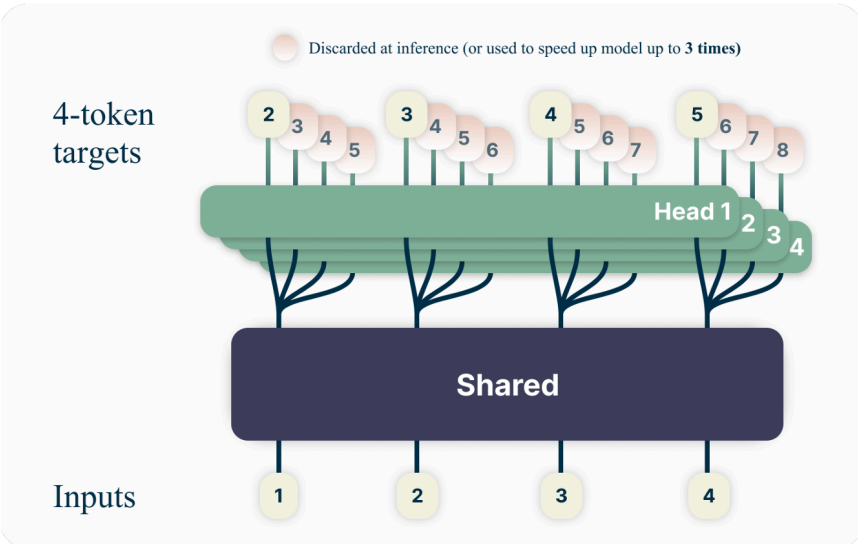
> *메모리 절약*

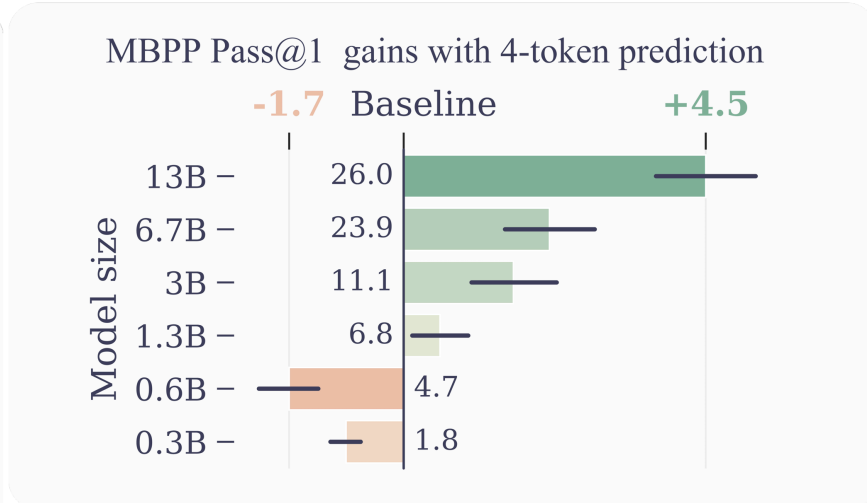


Multi-token Prediction

> *모델 학습 속도와 성능 향상*

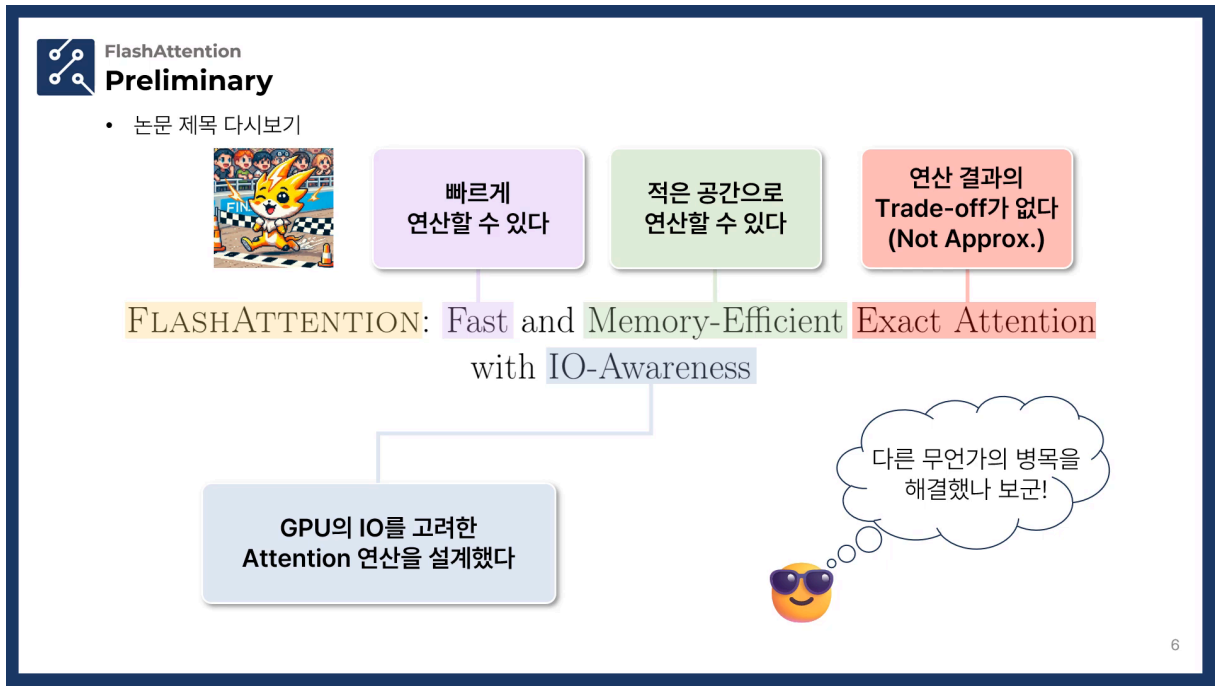
- Head1은 t+1, Head2는 t+2, Head3는 t+3, Head4는 t+4 예측
- 추론할 때는 Head2, Head3, Head4 제거
- 문맥을 잘 이해할 수 요도록 만들고 모델이 클수록 성능 향상





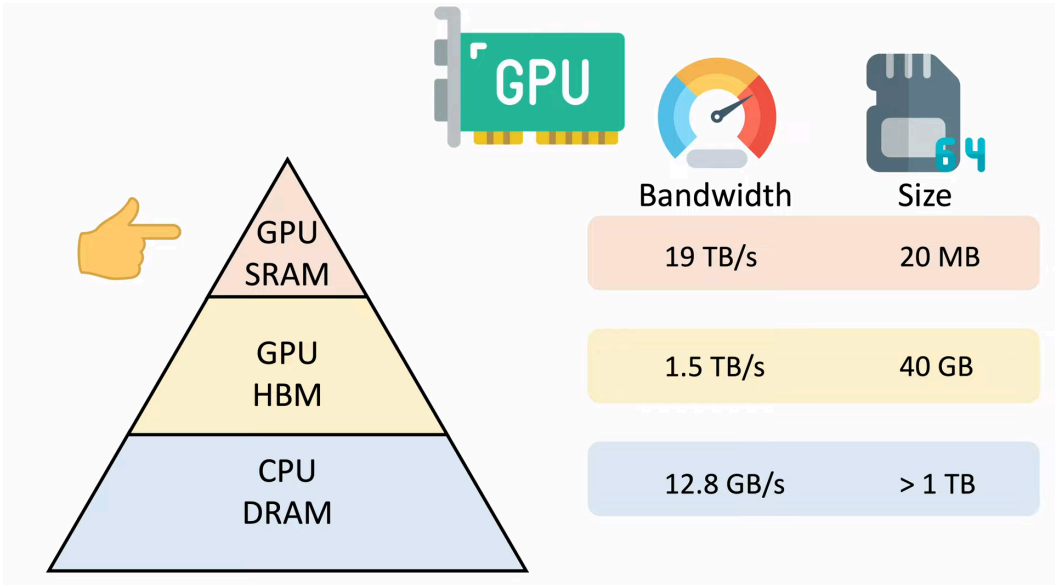
Flash Attention

> *I/O를 최적화하여 학습 속도를 2-3배 향상*



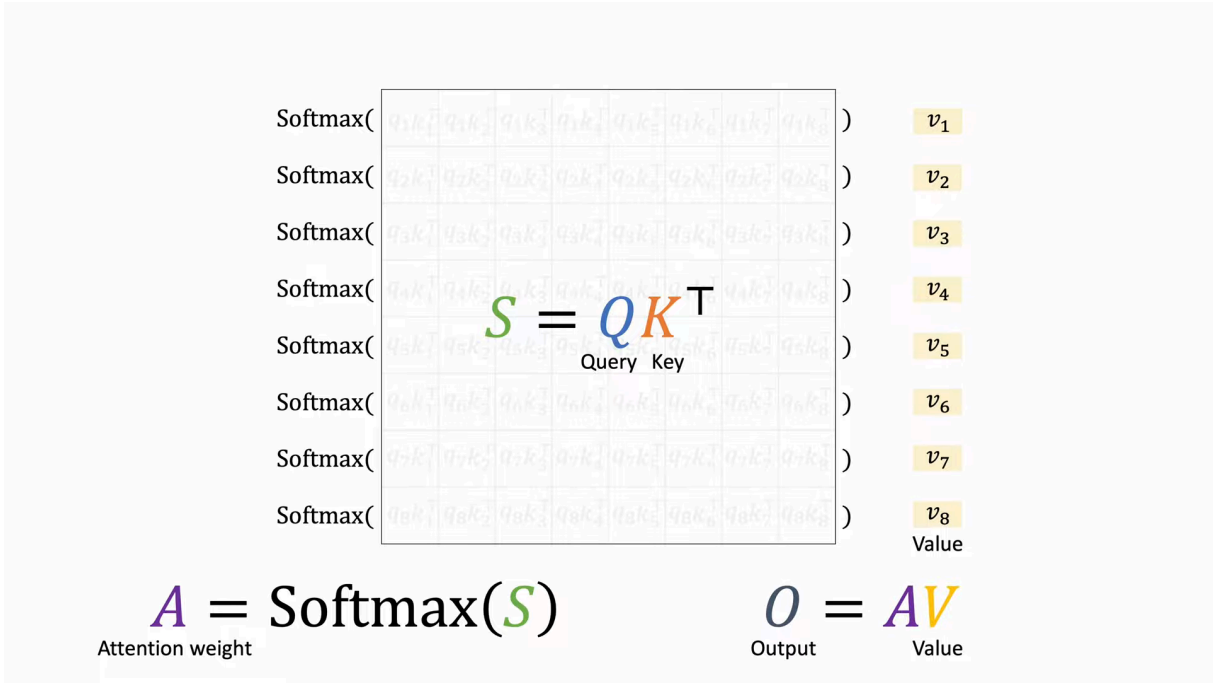
Flash Attention

> *실제 연산은 GPU에 있는 사이즈가 작은 SRAM에서 발생*



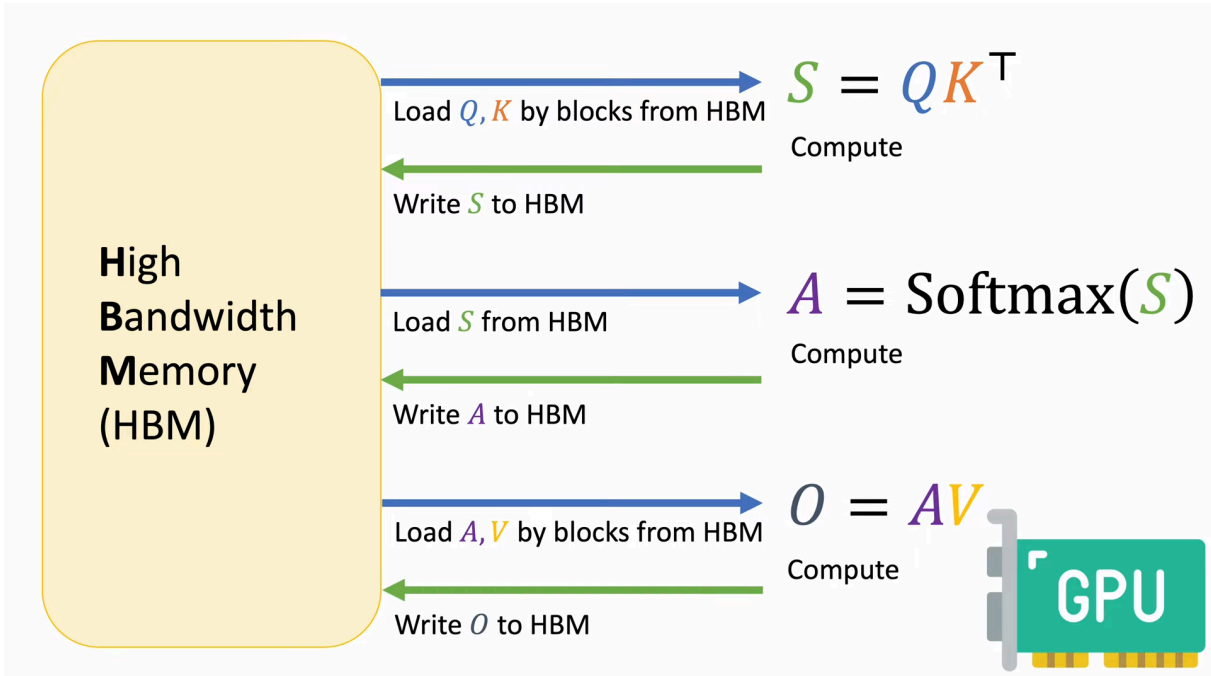
Flash Attention

> 계산 과정에서 많은 메모리가 요구:N2가 빈번하게 발생



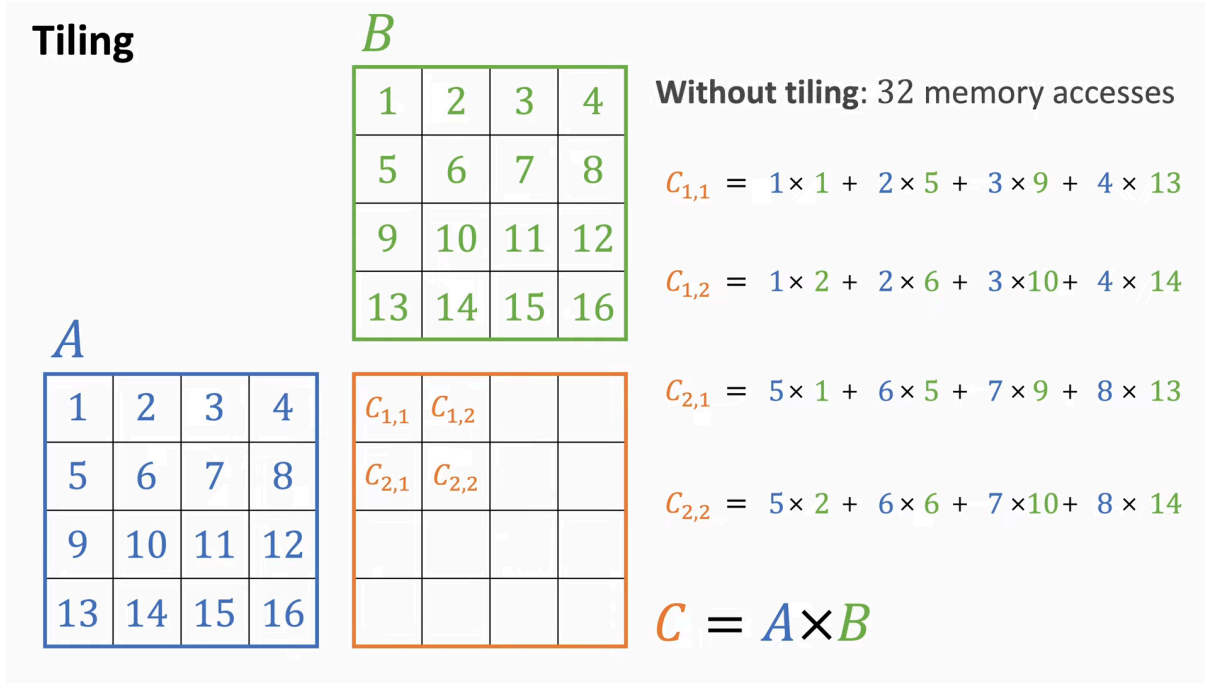
Flash Attention

> 메모리 I/O가 빈번하게 발생



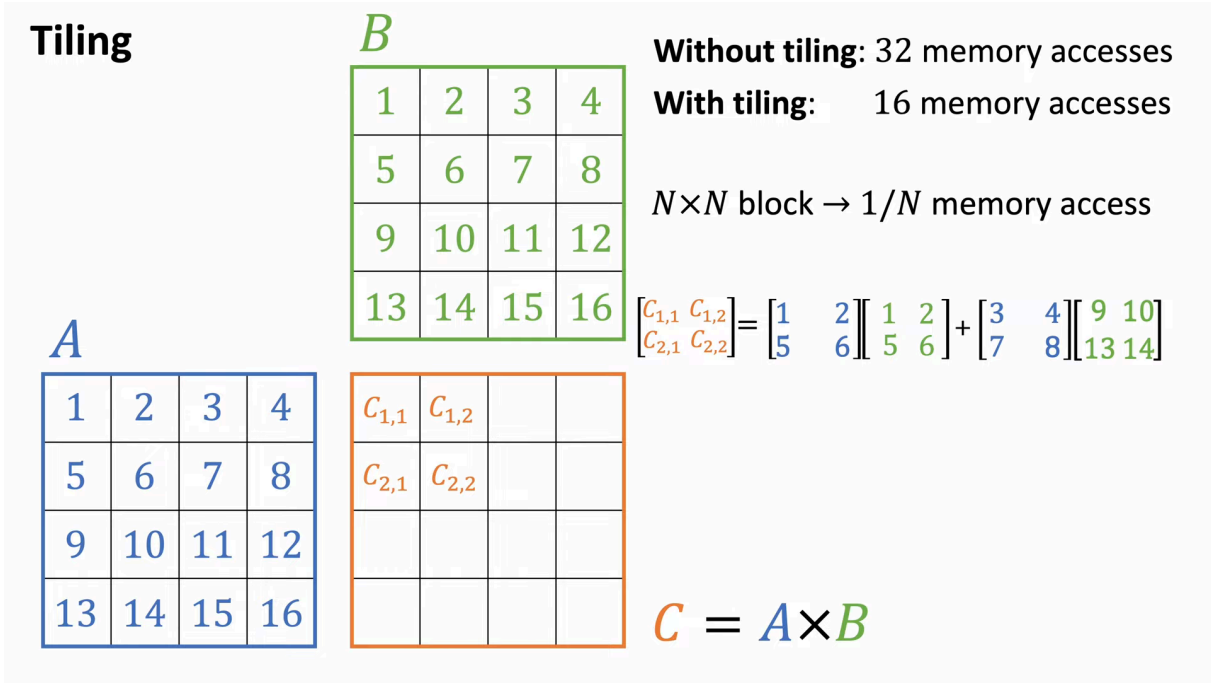
Flash Attention

> *Tiling이 없으면 A 메모리 엑세스 + B 메모리 엑세스= 32번 발생*



Flash Attention

> *Tiling을 통해 메모리 I/O를 줄일 수 있음*



Flash Attention

> *Softmax는 mN과 dN을 구하는 for loop가 존재*

$$A = \text{Softmax}(S) \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$m_N = \max(\{x_1, x_2, \cdots, x_N\})$$

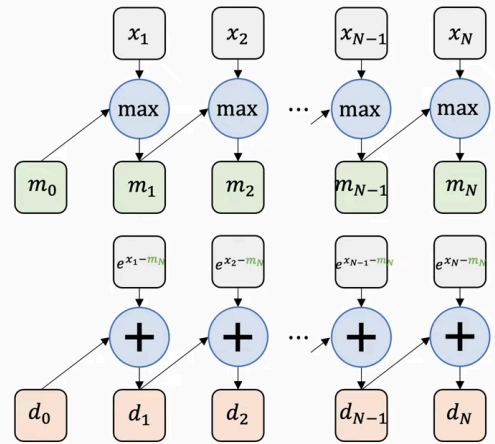
$$m_0 = -\infty$$
$$\textbf{for } 1 \le i \le N \textbf{ do}$$
$$m_i = \max(m_{i-1}, x_i)$$

$$d_N = \sum_{i=1}^{N} e^{x_i - m_N}$$

$$d_0 = 0$$
$$\textbf{for } 1 \le i \le N \textbf{ do}$$
$$d_i = d_{i-1} + e^{x_i - m_N}$$

$$A = \{a_1, a_2, \cdots, a_N\}$$
$$a_i = e^{x_i - m_N} / d_N$$

$$\textbf{for } 1 \le i \le N \textbf{ do}$$
$$a_i = e^{x_i - m_N} / d_N$$



Flash Attention

> *mN, dN 통합*

$$A = \text{Softmax}(S) \quad S = \{x_1, x_2, \cdots, x_N\}$$

$m_0 = -\infty$

for $1 \le i \le N$ **do**

$m_i = \max(m_{i-1}, x_i)$

$d_0 = 0$

for $1 \le i \le N$ **do**

$d_i = d_{i-1} + e^{x_i - m_N}$

for $1 \le i \le N$ **do**

$a_i = \dfrac{e^{x_i - m_N}}{d_N}$

$d_i = \sum_{j=1}^{i} e^{x_j - m_N}$ $\quad d'_i = \sum_{j=1}^{i} e^{x_j - m_i}$ $\quad d'_N = d_N$

$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_i}\right) + e^{x_i - m_i}$

더미값:결과는 1

$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_i}\right) \cdot e^{m_{i-1}} \cdot e^{-m_i - 1} + e^{x_i - m_i}$

$d'_i = \left(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}}\right) \cdot e^{m_{i-1} - m_i} + e^{x_i - m_i}$

d'_{i-1}

Flash Attention

Oi통합

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$m_0 = -\infty$$

$$d_0 = 0$$

$$o_i = \sum_{j=1}^{i} \frac{e^{x_j - m_N}}{d'_N} v_j \qquad o_N = o'_N$$

for $1 \le i \le N$ **do**

$$x_i = qk_i^\top$$

$$m_i = \max(m_{i-1}, x_i)$$

$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = \sum_{j=1}^{i} \frac{e^{x_j - m_i}}{d'_i} v_j$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j - m_i}}{d'_i} v_j + \frac{e^{x_i - m_i}}{d'_i} v_i$$

$$o'_i = \sum_{j=1}^{i-1} \frac{e^{x_j - m_i}}{d'_i} \frac{d'_{i-1}}{d'_{i-1}} \frac{e^{-m_{i-1}}}{e^{-m_{i-1}}} v_j + \frac{e^{x_i - m_i}}{d'_i} v_i$$

$$o'_i = \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} v_j\right) \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

o'_{i-1}

$o_0 = 0$

for $1 \le i \le N$ **do**

$$a_i = \frac{e^{x_i - m_N}}{d'_N}$$

$$o_i = o_{i-1} + a_i v_i$$

return o_N

Flash Attention

> *하나의 Loop로 통합: Flash Attention*

$$S = QK^\top \qquad A = \mathrm{Softmax}(S) \qquad O = AV \qquad S = \{x_1, x_2, \cdots, x_N\}$$

$$x_i = qk_i^\top$$

for $1 \le i \le N$ **do**

$$x_i = qk_i^\top$$

$$m_i = \max(m_{i-1}, x_i)$$

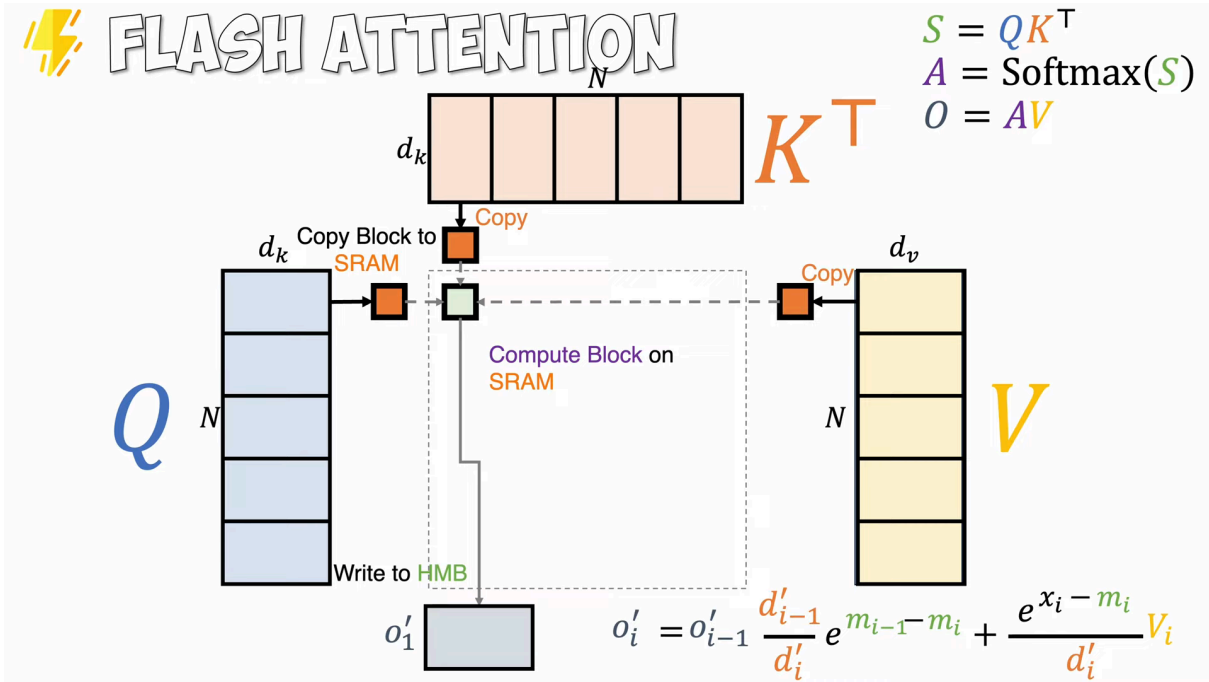
$$d'_i = d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

$$o'_i = o'_{i-1} \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} v_i$$

return o'_N

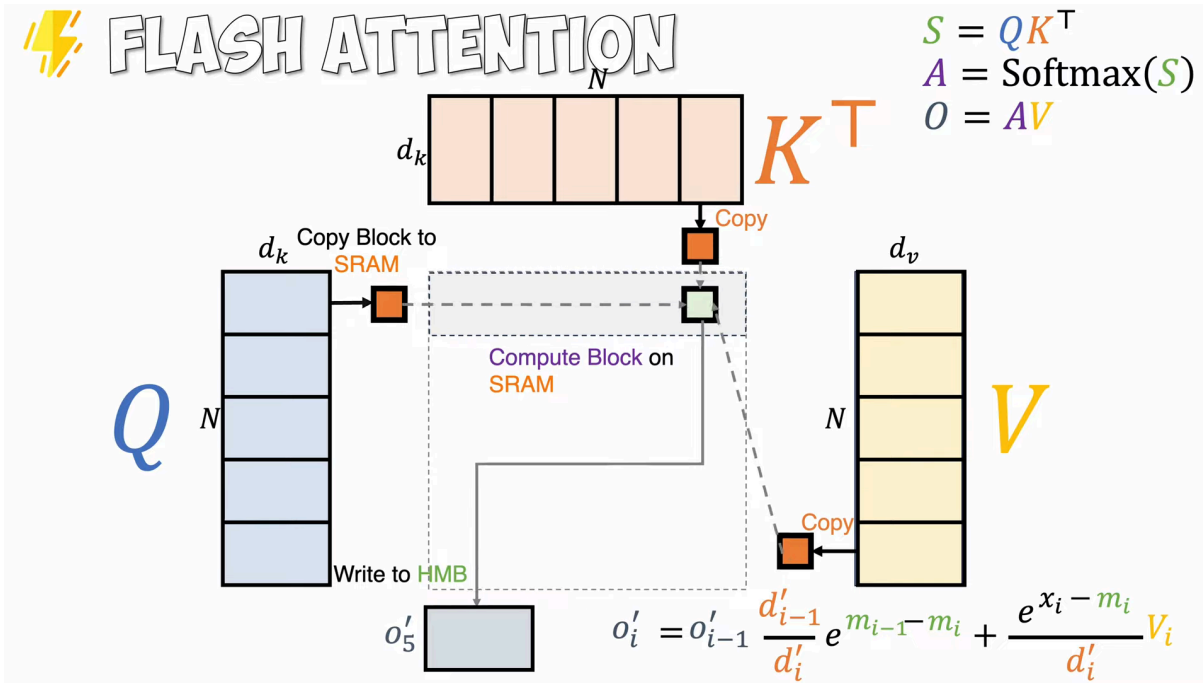
Flash Attention

> 여러번 대용량 메모리를 주고 받는 일이 없음



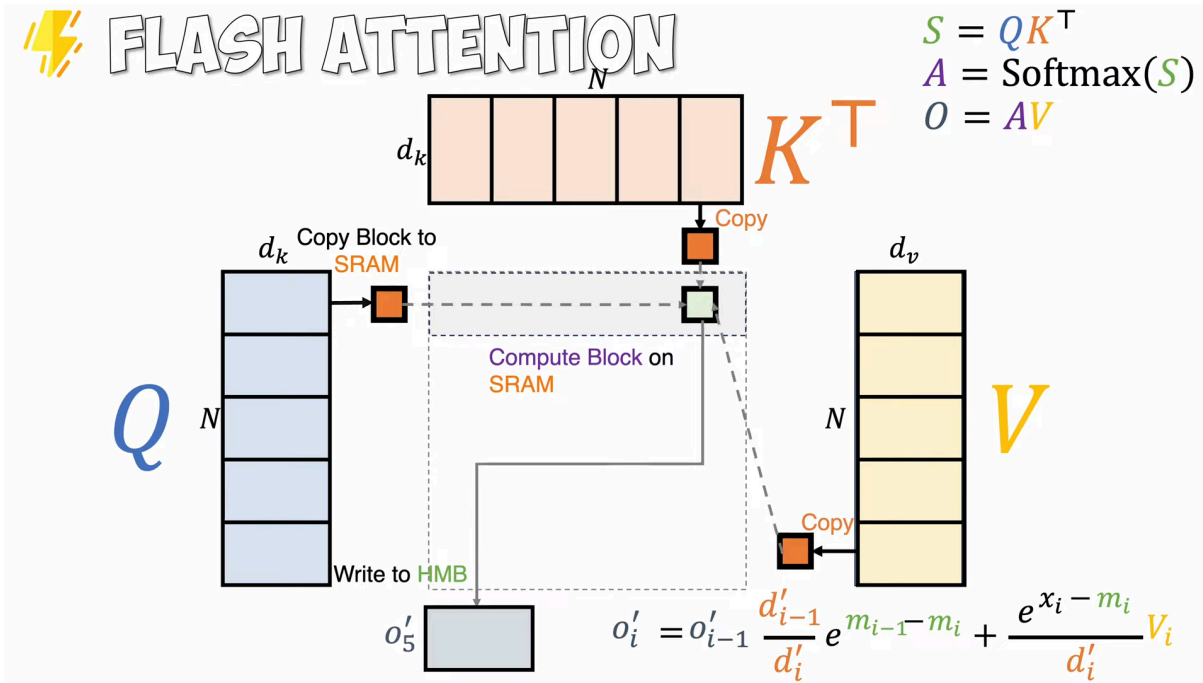
Flash Attention

> 계산에 필요한 메모리만 SRAM에 Copy하고, 계산 결과만 HBM에 씀



Flash Attention

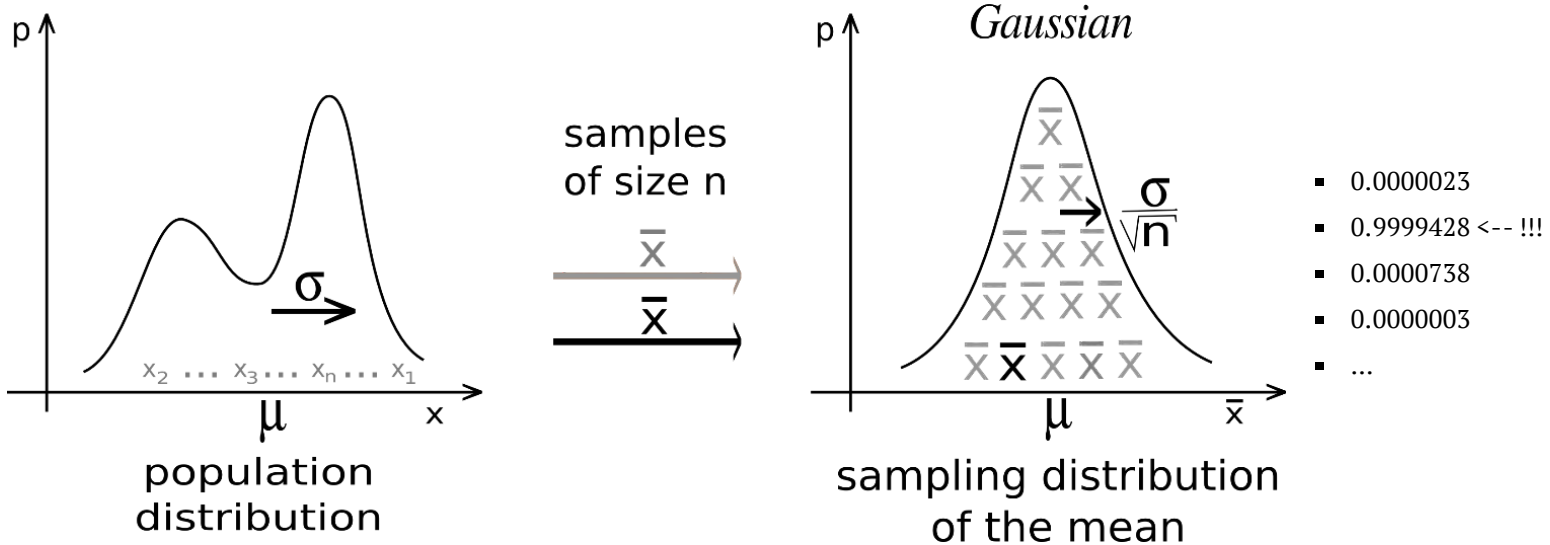
> 계산에 필요한 메모리만 SRAM에 Copy하고, 계산 결과만 HBM에 씀



TurboQuant

> *KV를 양자화해서 메모리 사용량을 줄이고, 추론 속도를 빠르게 함*

- 정규화 분포를 만들어서 양자화



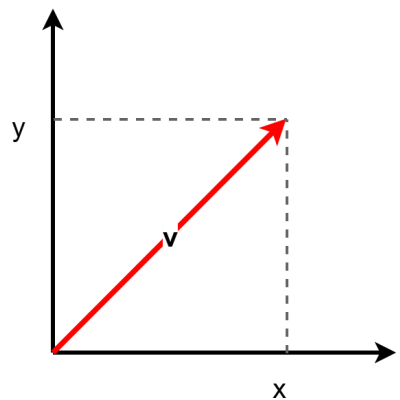
- 0.0000023
- 0.9999428 <-- !!!
- 0.0000738
- 0.0000003
- ...

TurboQuant

벡터 표현

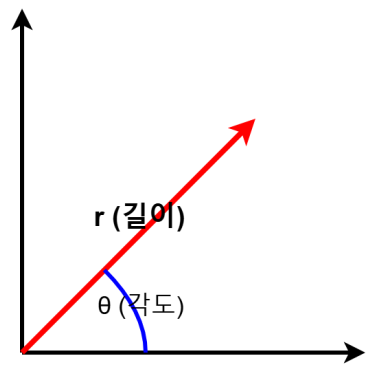
직교좌표계 표현
(Cartesian Coordinates)

x 좌표, y 좌표



극좌표계 표현
(Polar Coordinates)

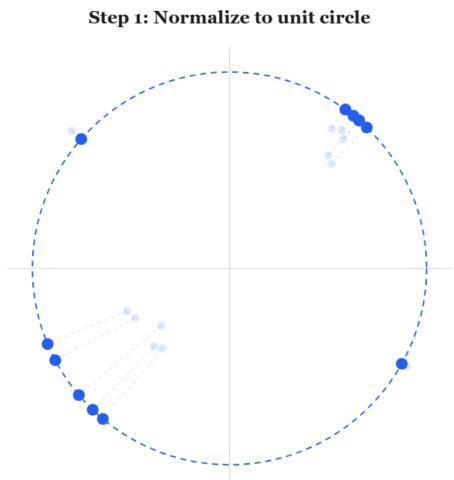
길이(r), 각도(θ)

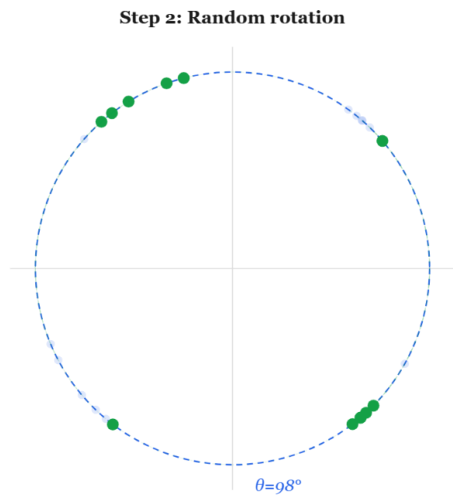


TurboQuant

> *무작위 회전을 통해 정규 분포로 만듦*

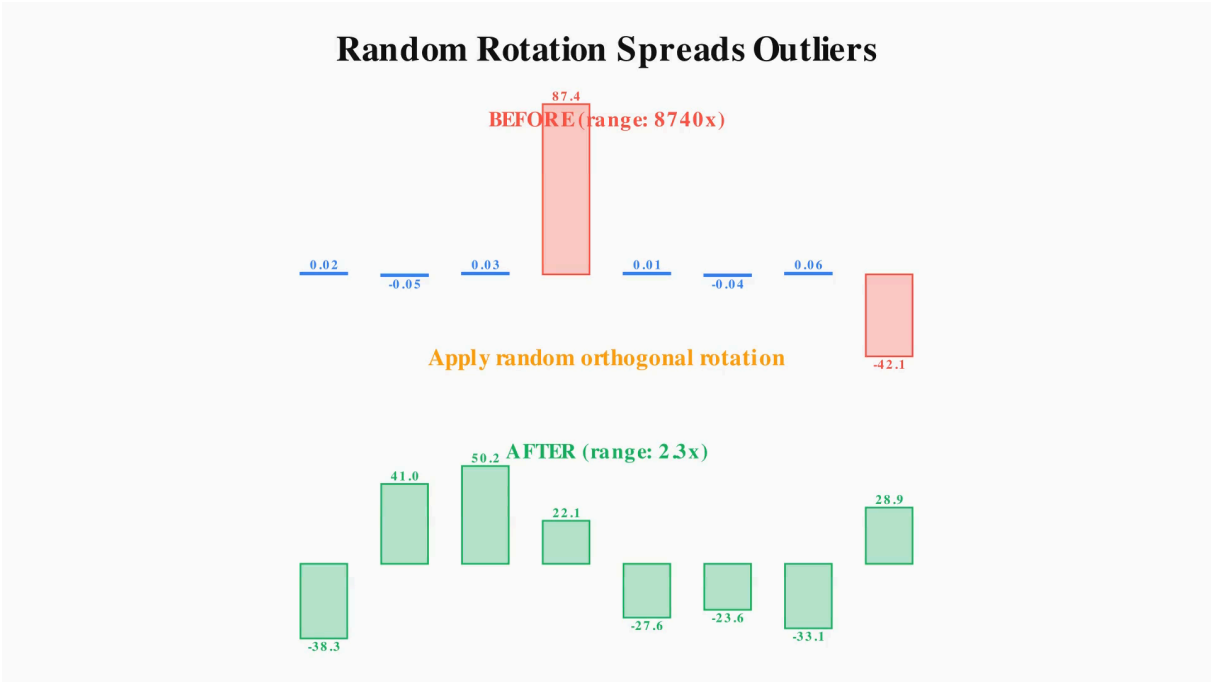
$$x_i' = \sum_{j=1}^{n} R_{ij}\, x_j = R_{i1}x_1 + R_{i2}x_2 + \cdots + R_{in}x_n$$





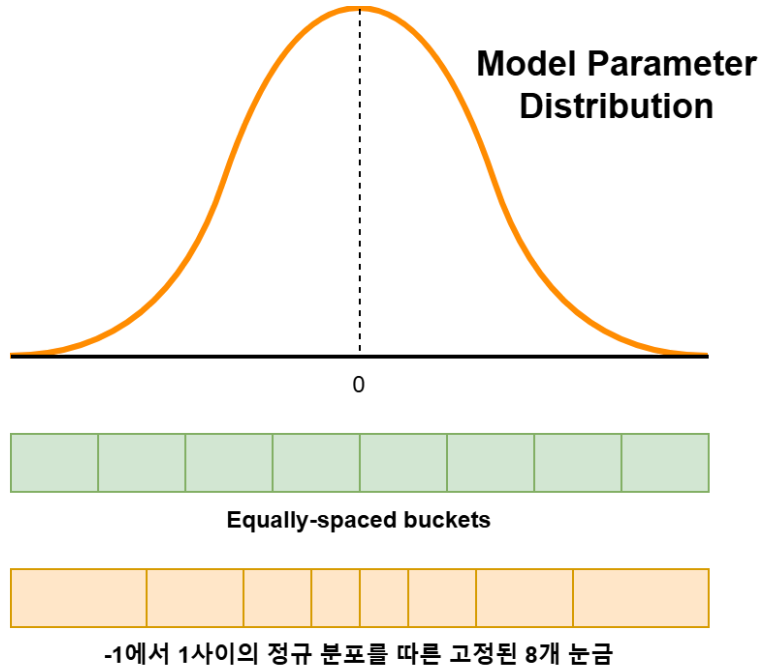
TurboQuant

> *무작위 회전을 통한 이상치 분산*



TurboQuant

> *회전한 단위 백터를 8구간으로 양자화, scale 대신 원본 백터의 길이 저장*



TurboQuant

> *QJL: 1비트 부호 정보를 저장하여 Attention Score 잔차 보정(양자화아님)*

STAGE 1

PolarQuant

```
# Stage 1: PolarQuant
R = random_orthogonal_matrix(d)    # d×d 직교 행렬 생성
x_rot = R @ x                      # 회전하여 에너지 균등 분산
centroids = lloyd_max(beta_dist)   # 목표 비트에 맞는 양자화
q_idx = nearest(x_rot, centroids)  # 각 좌표를 스칼라 양자화
norm = ||x||                       # 원본 norm 별도 저장
```

STAGE 2

QJL

```
# Stage 2: QJL 잔차 보정
residual = x_rot - dequant(q_idx, centroids)  # 양자화 잔
S = random_sign_matrix(m, d)        # +1,-1부호만 랜덤으로 
sketch = sign(S @ residual)         # 1-bit sketch (부호만
# 복원: dequant(q_idx) + QJL_decode(sketch, S)
```

참고: 도대체 종 모양에서 원주율 π가 왜? | 확률과 통계

TurboQuant

> *KV 캐시 메모리를 줄여 더 많은 Context가 가능하게 함*

- 양자화를 통해 메모리 IO 부하를 줄여서 추론을 빠르게함
- 메모리가 줄어들어서 메모리가 많이 필요하지 않다는 잘못된 해석
- Softmax에 의해 QJL의 노이즈가 뻥튀기 되어 QJL이 성능을 떨어뜨리다는 결과가 있음

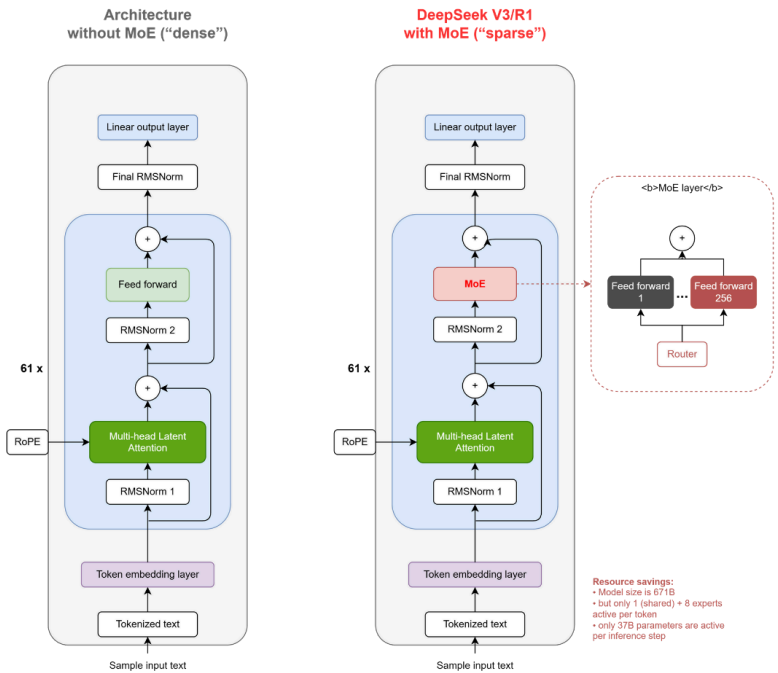
참고: tonbistudio/turboquant-pytorch

MOE (Mixture of Experts)

MOE

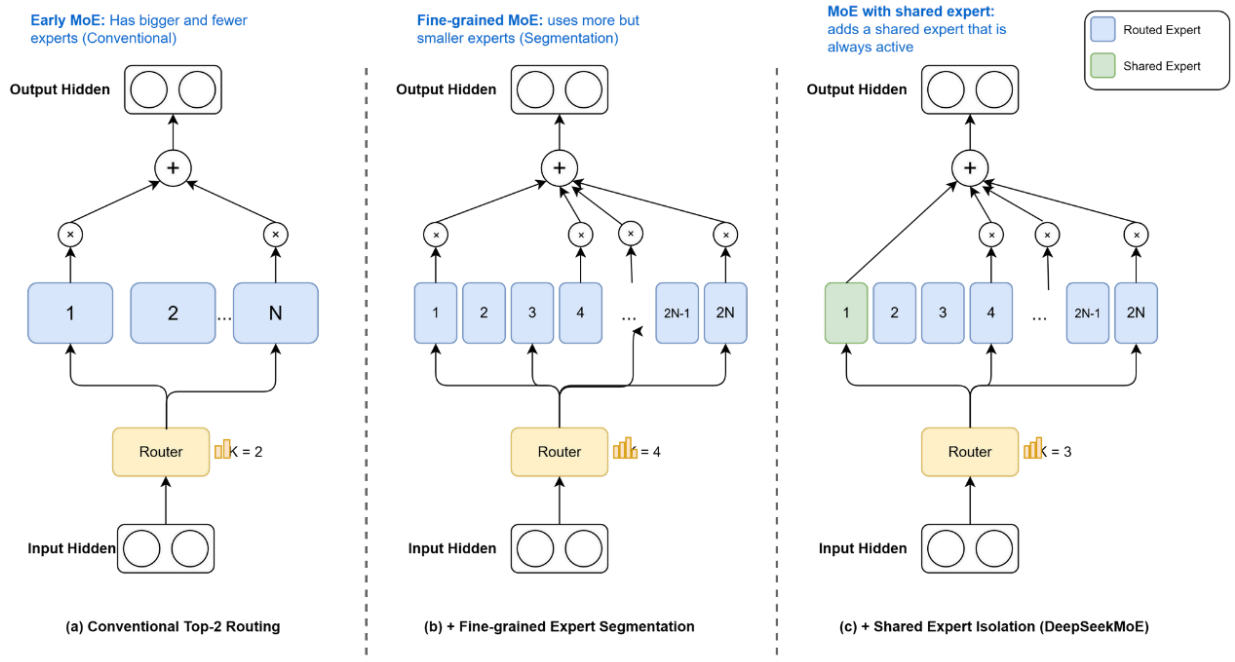
Inference 속도 향상

- 학습이 어려움(Router)
- 공통 전문가는 동일한 지식을 공유



MOE

> *Expert 세분화, 공동 Expert 추가*



MOE: 코드로 보는 구조 (MiniMind)

> *FeedForward(SwiGLU FFN)를 여러 개 두고, Router가 토큰별로 expert를 선택*

```python
class MOEFeedForward(nn.Module):
    def __init__(self, config):
        self.gate = nn.Linear(hidden_size, num_experts, bias=False)  # Router
        self.experts = nn.ModuleList([
            FeedForward(config, intermediate_size=moe_intermediate_size)
            for _ in range(num_experts)
        ])
```

```python
class FeedForward(nn.Module):           # 각 Expert의 내부 구조 (SwiGLU)
    def forward(self, x):
        return down_proj(act_fn(gate_proj(x)) * up_proj(x))
```

- `gate` : 각 토큰이 어떤 expert를 쓸지 결정하는 라우터
- `experts` : `num_experts` 개의 독립 FFN (각각 `moe_intermediate_size` 로 더 작은 크기)

MoE 순전파: 토큰 벡터 펼치기

> *라우터가 토큰별 전문가를 선택하고, 전문가 출력을 가중합합니다.*

예제는 전문가 4개 중 토큰마다 2개를 선택합니다.

```
batch_size, seq_len, hidden_dim = x.shape
x_flat = x.view(-1, hidden_dim)
```

입력	형태	예시
`x`	`[배치 크기, 문장 길이, 벡터 차원]`	`[2, 3, 768]`
`x_flat`	`[전체 토큰 수, 벡터 차원]`	`[6, 768]`

입력은 토큰 ID가 아니라 토큰을 표현하는 벡터입니다.

라우터 점수와 Top-K 선택

```
scores = F.softmax(self.gate(x_flat), dim=-1)
topk_weight, topk_idx = torch.topk(
    scores, k=self.config.num_experts_per_tok,
    dim=-1, sorted=False
)
```

전문가	한 토큰의 확률	Top-2 선택
0	0.1	
1	0.5	선택
2	0.3	선택
3	0.1	

`topk_idx` 는 전문가 번호, `topk_weight` 는 대응하는 가중치입니다.

전문가 수가 늘면 전체 파라미터는 늘지만, `K` 가 고정되면 토큰당 활성 전문가 수는 유지됩니다.

선택한 가중치 정규화

> *`norm_topk_prob=True` 이고 `K > 1` 일 때 적용합니다.*

```
topk_weight = topk_weight / (
    topk_weight.sum(dim=-1, keepdim=True) + 1e-20
)
```

선택 전문가	기존 가중치	정규화 후
1	0.5	`0.5 / 0.8 = 0.625`
2	0.3	`0.3 / 0.8 = 0.375`

각 토큰에서 선택한 전문가 가중치의 합을 1로 만듭니다.

Top-1: 출력 가중치와 기울기 처리

> *`norm_topk_prob=True` 이고 `K = 1` 이면 별도 경로를 사용합니다.*

```
top1 = torch.topk(
    F.softmax(self.gate(x_flat.detach()), dim=-1),
    k=1, dim=-1, sorted=False
)[0]
topk_weight = top1 - top1.detach() + 1.0
```

단계	동작
순전파	`top1 - top1 + 1 = 1` 로 전문가 출력을 그대로 사용
역전파	`top1` 경로를 통해 라우터 파라미터에 기울기 전달

`detach()` 는 값을 유지하면서 해당 경로의 기울기를 끊습니다.

전문가별 토큰 계산과 결과 누적

```
y = torch.zeros_like(x_flat)
for i, expert in enumerate(self.experts):
    mask = (topk_idx == i)
    if mask.any():
        token_idx = mask.any(dim=-1).nonzero().flatten()
        weight = topk_weight[mask].view(-1, 1)
        y.index_add_(
            0, token_idx,
            (expert(x_flat[token_idx]) * weight).to(y.dtype)
        )
```

라우터가 배정한 토큰만 전문가가 처리하고, 가중치를 곱해 원래 위치에 더합니다.

```
한 토큰의 출력 = 0.625 × 전문가 1의 출력 + 0.375 × 전문가 2의 출력
```

token_idx: 현재 전문가를 선택한 토큰 위치

> *토큰 4개의 선택 결과에서 현재 전문가가 `i = 2` 라고 가정합니다.*

```
topk_idx = [[0, 2], [1, 3], [0, 1], [2, 3]]
```

토큰 행 번호	선택 전문가	전문가 2 선택 여부
0	`[0, 2]`	True
1	`[1, 3]`	False
2	`[0, 1]`	False
3	`[2, 3]`	True

```
mask = (topk_idx == i)                          # [N, K]
token_idx = mask.any(dim=-1).nonzero().flatten() # [0, 3]
```

`x_flat[token_idx]` 는 토큰 0과 3의 벡터를 가져옵니다.

미선택 전문가: 0 기울기로 계산 그래프 연결

> *담당 토큰이 없는 전문가는 그래프에서 빠져 기울기가 `None` 일 수 있습니다. DDP는 GPU 간 기울기를 동기화하므로, 기본 설정에서는 오류가 발생할 수 있습니다.*

```
elif self.training:  # 현재 전문가를 선택한 토큰이 없을 때
    y[0, 0] += 0 * sum(p.sum() for p in expert.parameters())
```

항목	이 코드의 효과
순전파	유한한 파라미터 합에 0을 곱해 더하므로 출력 유지
계산 그래프	미선택 전문가의 모든 파라미터를 출력에 연결
역전파	이 연결의 기울기는 0 → DDP의 미사용 파라미터 처리 지원

`y[0, 0]`은 연결을 위한 위치이며, 첫 토큰을 이 전문가에게 배정하지 않습니다.

계산 그래프 연결은 이 코드가, 전문가 선택의 쏠림 완화는 `aux_loss`가 담당합니다.

보조 손실: 선택 빈도와 평균 확률

```
if self.training and self.config.router_aux_loss_coef > 0:
    load = F.one_hot(topk_idx, self.config.num_experts).float().mean(0)
    self.aux_loss = (
        (load * scores.mean(0)).sum()
        * self.config.num_experts
        * self.config.router_aux_loss_coef
    )
```

값	의미
`load`	선택 슬롯별 각 전문가의 실제 선택 비율
`scores.mean(0)`	전체 토큰에서 각 전문가의 평균 라우터 확률

선택 빈도와 평균 확률을 결합해 특정 전문가에 대한 쏠림을 완화합니다.

선택 빈도 계산: one_hot과 평균

> *전문가 4개 중 토큰마다 1개를 선택하는 예제입니다. (`K=1`, `E=4`)*

```
topk_idx = torch.tensor([[0], [0], [1], [2]])
load = F.one_hot(topk_idx, num_classes=4).float().mean(0)
```

토큰	선택 전문가	one-hot 표현
0	0	`[1, 0, 0, 0]`
1	0	`[1, 0, 0, 0]`
2	1	`[0, 1, 0, 0]`
3	2	`[0, 0, 1, 0]`

```
토큰 차원으로 평균: load = [[0.50, 0.25, 0.25, 0.00]]
```

전문가 0은 토큰의 50%, 전문가 1과 2는 각각 25%가 선택했습니다.

`load: [K, E]` × 평균 확률: `[E]` → 모든 슬롯·전문가 합산 → 스칼라 (`K` : 선택 수, `E` : 전문가 수)

보조 손실 계산 예제

> *`load`는 실제 선택 빈도, `scores.mean(0)`은 선택 전 라우터 확률의 평균입니다.*

전문가	선택 빈도	평균 확률	곱
0	0.50	0.40	0.200
1	0.25	0.30	0.075
2	0.25	0.20	0.050
3	0.00	0.10	0.000

```
self.aux_loss = (
    (load * scores.mean(0)).sum()
    * self.config.num_experts
    * self.config.router_aux_loss_coef
)
```

이 예제에서는 `aux_loss = 0.325 × 4 × 보조 손실 계수` 입니다.

보조 손실의 학습 경로와 출력 전달

> *`load`의 정수 선택 인덱스에는 기울기가 흐르지 않고, `scores`를 통해 라우터를 학습합니다.*

```
if self.training and self.config.router_aux_loss_coef > 0:
    # 선택 빈도와 평균 확률로 계산한 스칼라 보조 손실 저장
    self.aux_loss = ...
else:
    self.aux_loss = scores.new_zeros(1).squeeze()
```

결과	처리 및 전달
전문가 출력의 가중합	`[6, 768]` → `[2, 3, 768]`로 복원해 반환
보조 손실	학습 중 양의 계수일 때 계산, 그 외에는 같은 장치·자료형의 스칼라 `0`

각 MoE 층의 `self.aux_loss`를 합산해 학습 코드에 전달합니다.

```
loss = res.loss + res.aux_loss  # SFT: 주 손실 + 보조 손실
```

Recent Developments in LLM Architectures

왜 Long-Context 비용인가

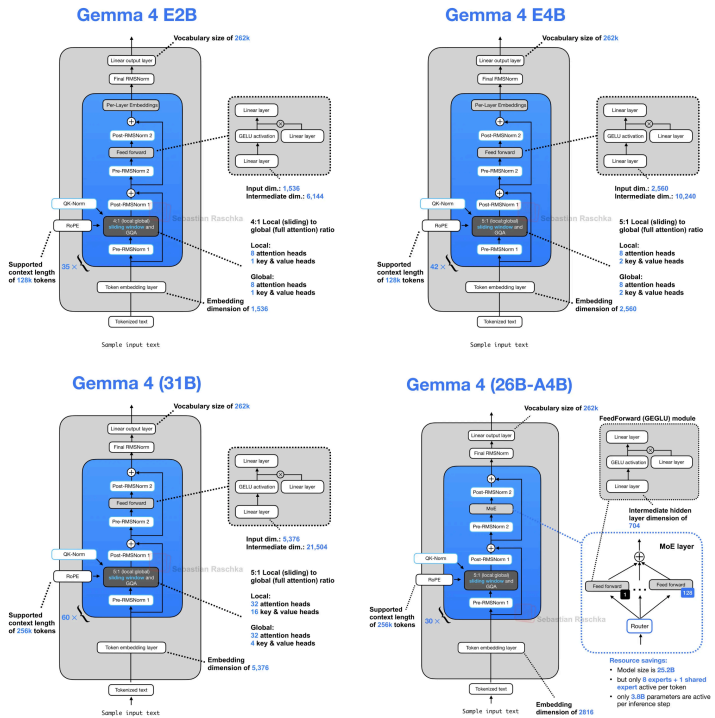
늘어나는 것	병목
Reasoning trace	더 긴 KV cache
Agent memory	더 많은 memory traffic
Tool-use transcript	Attention 계산 비용 증가
Multi-document context	Prefill + decode 비용 증가

> 최근 설계의 핵심은 "모델을 작게 만들자"가 아니라, **긴 문맥을 들고 있어도 싸게 추론하자**이다.

다루는 아키텍처

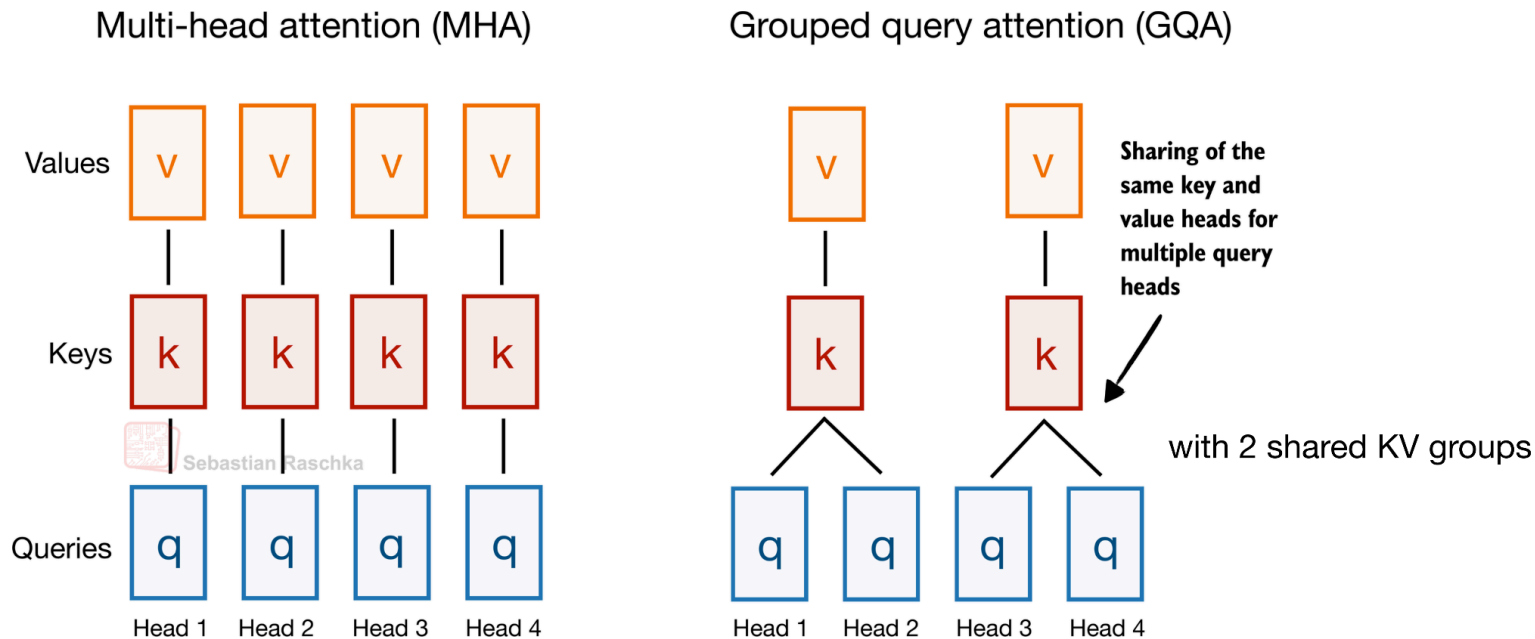
- **KV Sharing**: 레이어 간 K/V 재사용으로 cache 절감
- **Per-Layer Embeddings**: main stack 대신 embedding으로 capacity 확보
- **Attention Budgeting**: layer마다 query head 수를 다르게 배정
- **CCA**: compressed latent space에서 attention 직접 수행
- **mHC**: 다중 residual stream + 안정적 mixing
- **CSA/HCA**: sequence dimension 자체를 압축

1. Gemma 4: Cross-Layer KV Sharing



Gemma 4 계열: **E2B**, **E4B**, **26B MoE**, **31B dense**. 핵심은 뒤쪽 layer가 앞쪽 layer의 K/V를 **재사용**하는 cross-layer attention.

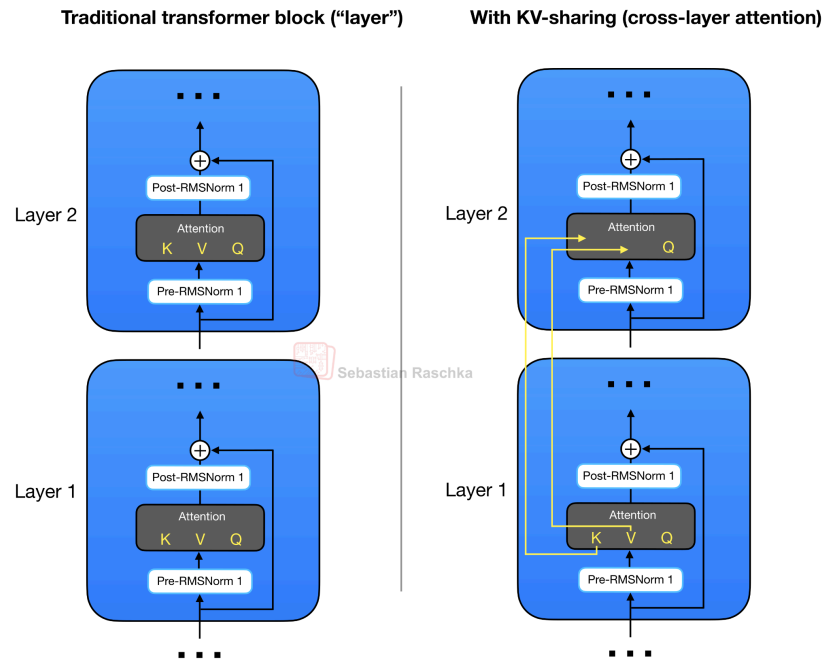
GQA에서 Cross-Layer Attention으로



GQA: 여러 Q head가 가ㅌ은 K/V head를 공유 → KV cache 절감의 표준 기법.

Cross-layer attention은 한 발 더 나아가 **다른 layer의 KV까지 공유**한다.

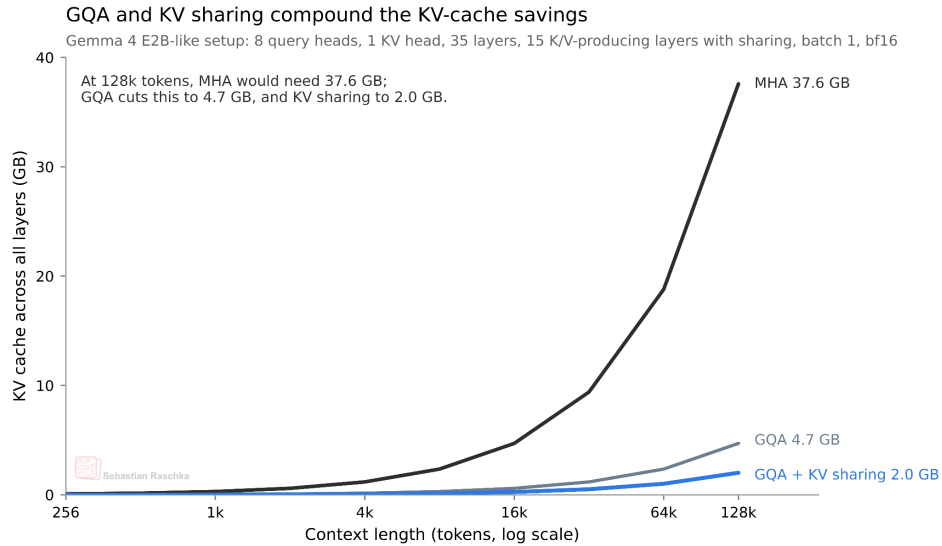
Cross-Layer Attention 메커니즘



왼쪽: 일반 transformer — 각 layer가 Q, K, V를 독립 계산.
오른쪽: Cross-layer — 뒤쪽 layer는 **Q만** 계산하고 K/V는 앞쪽 layer에서 재사용.

Brandon et al. (NeurIPS 2024) 제안, Gemma 4가 주요 모델에 최초 적용.

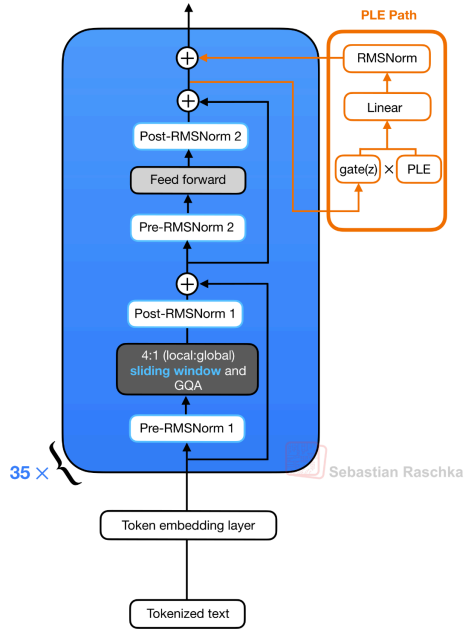
KV Cache 메모리 절감



Gemma 4 variant	총 layers	자체 KV 계산	공유 layers	128K context KV cache
E2B	35	15	20	~2.7 GB (bfloat16)
E4B	42	24	18	~6 GB (bfloat16)

GQA + cross-layer sharing 결합 시 **KV cache ~50% 절감**. 소형 모델에서 capacity 손실은 minimal.

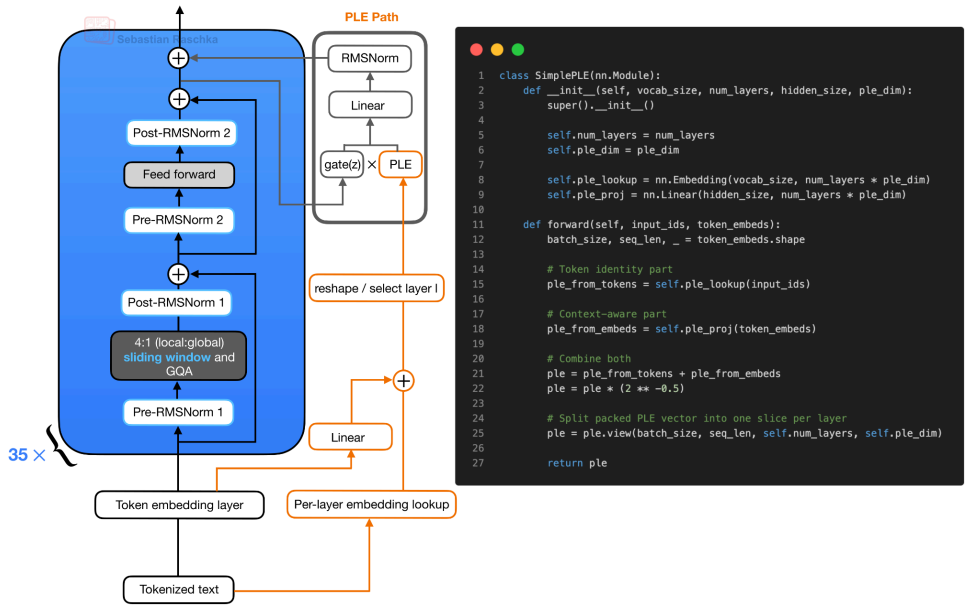
2. Gemma 4: Per-Layer Embeddings (PLE)




```python
class Gemma4BlockWithPLE(nn.Module):
    def __init__(self, dim, ple_dim):
        super().__init__()

        self.pre_attn_norm = RMSNorm(dim)
        self.attn = GQA(dim)
        self.post_attn_norm = RMSNorm(dim)

        self.pre_ffn_norm = RMSNorm(dim)
        self.ffn = FeedForward(dim)
        self.post_ffn_norm = RMSNorm(dim)

        # PLE path
        self.ple_gate = nn.Linear(dim, ple_dim)
        self.ple_up = nn.Linear(ple_dim, dim)
        self.ple_norm = RMSNorm(dim)

    def forward(self, x, ple_l):
        residual = x
        x = self.pre_attn_norm(x)
        x = self.attn(x)
        x = self.post_attn_norm(x)
        h = residual + x

        residual = h
        x = self.pre_ffn_norm(h)
        x = self.ffn(x)
        x = self.post_ffn_norm(x)
        z = residual + x

        # Extra PLE residual update
        ple_update = self.ple_gate(z) * ple_l
        ple_update = self.ple_up(ple_update)
        ple_update = self.ple_norm(ple_update)

        out = z + ple_update
        return out
```

Attention + FFN 뒤에 **layer별 token embedding**을 residual로 추가. Hidden state가 PLE vector를 출력에 합산.

PLE 생성 과정




```python
class SimplePLE(nn.Module):
    def __init__(self, vocab_size, num_layers, hidden_size, ple_dim):
        super().__init__()

        self.num_layers = num_layers
        self.ple_dim = ple_dim

        self.ple_lookup = nn.Embedding(vocab_size, num_layers * ple_dim)
        self.ple_proj = nn.Linear(hidden_size, num_layers * ple_dim)

    def forward(self, input_ids, token_embeds):
        batch_size, seq_len, _ = token_embeds.shape

        # Token identity part
        ple_from_tokens = self.ple_lookup(input_ids)

        # Context-aware part
        ple_from_embeds = self.ple_proj(token_embeds)

        # Combine both
        ple = ple_from_tokens + ple_from_embeds
        ple = ple * (2 ** -0.5)

        # Split packed PLE vector into one slice per layer
        ple = ple.view(batch_size, seq_len, self.num_layers, self.ple_dim)

        return ple
```

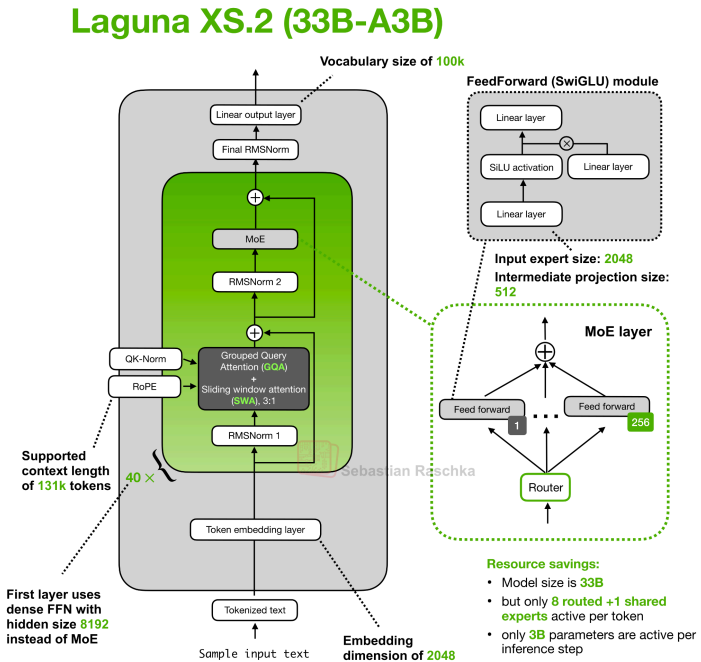
1. **Token ID**로 per-layer embedding lookup
2. 일반 token embedding을 같은 공간으로 projection
3. 합산 + reshape → 각 layer가 **PLE slice**를 받음
4. Hidden state가 PLE vector를 gate → projection → residual 추가

"Effective" Size의 의미

모델	Effective params	총 params (embedding 포함)
E2B	2.3B	5.1B
E4B	4.5B	8.0B
일반적인 용량 확대	PLE 방식	
Layer 수 증가	Layer별 작은 embedding slice 추가	
Hidden size 확대	Main transformer는 작게 유지	
FFN 확대	Lookup-style parameter로 보강	

핵심: **비싼 compute는 "effective" size로 유지, 싼 embedding table로 capacity 보강**. MoE와 유사한 발상.

3. Laguna XS.2: Layer-Wise Attention Budgeting

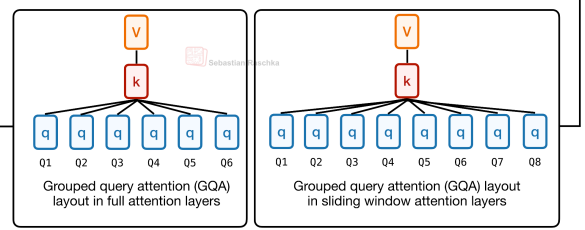


Poolside의 coding-focused 모델. 핵심은 **layer마다 attention 예산과 방식을 다르게 할당**.

40 layers: sliding-window 30개, full/global attention 10개.

Per-Layer Query Head 분배

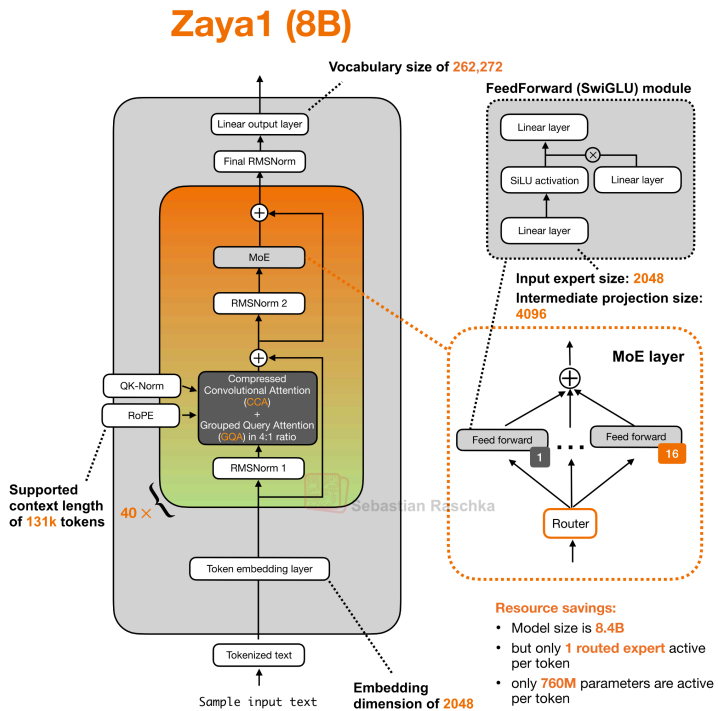
```
Layers 0–39:

0   full      Q heads: 48   KV heads: 8   => 6 Q heads per KV head
1   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
2   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
3   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head

4   full      Q heads: 48   KV heads: 8   => 6 Q heads per KV head
5   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
6   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
7   sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head

... repeated ...

36  full      Q heads: 48   KV heads: 8   => 6 Q heads per KV head
37  sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
38  sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
39  sliding   Q heads: 64   KV heads: 8   => 8 Q heads per KV head
```



Layer type	개수	Window	Q heads / KV head	KV heads
Full attention	10	전체 context	6	8
Sliding-window	30	512 tokens	8	8

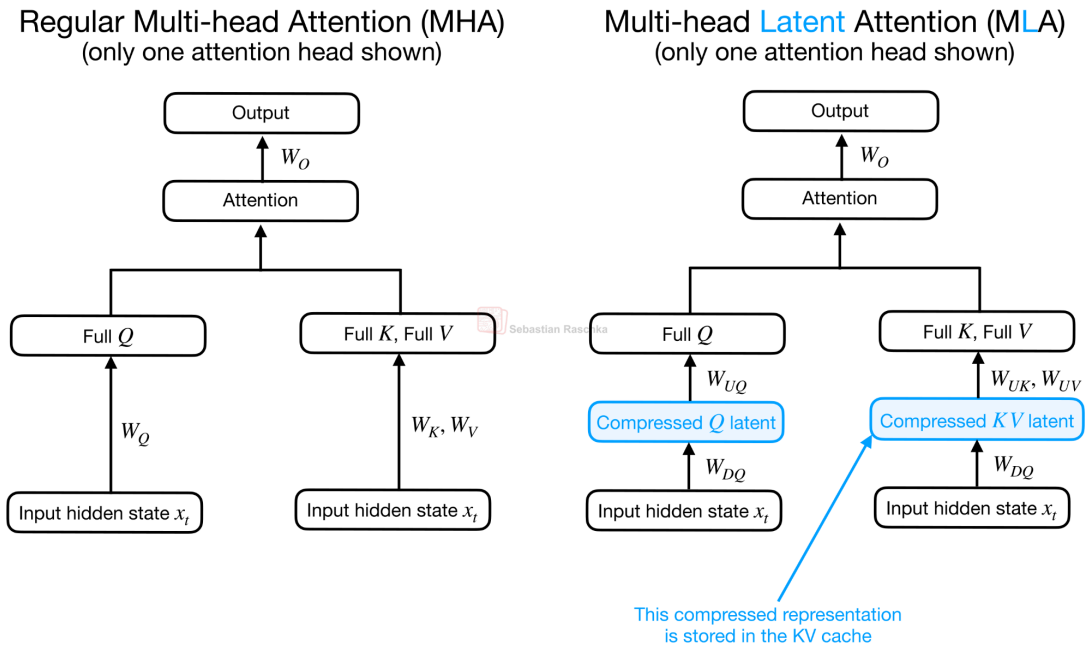
비싼 full attention에 **적은** Q head, 싼 sliding-window에 **많은** Q head 배정

ZAYA1-8B: Compressed Convolutional Attention



Zyphra의 8B MoE 모델 (AMD GPU 학습). 핵심: **CCA** attention을 compressed latent space에서 직접 수행.

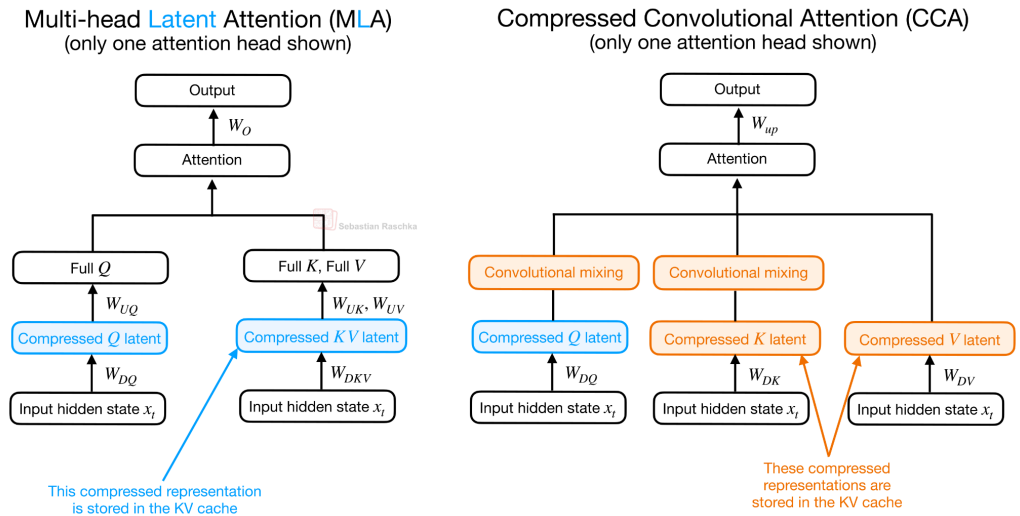
MHA와 MLA 복습



MHA: 각 head가 독립 Q, K, V 계산. Cache에 full K/V 저장.

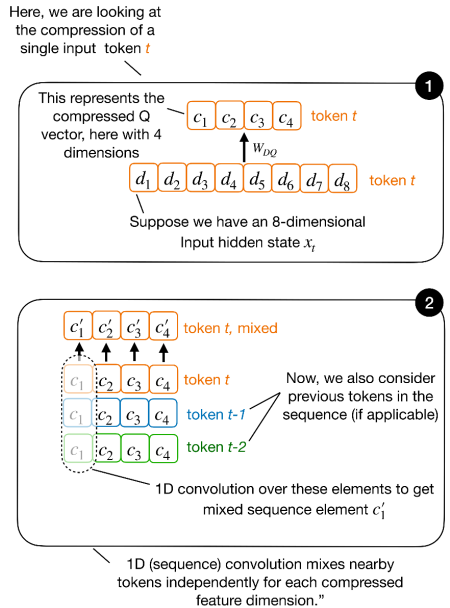
MLA: K/V를 compressed latent로 cache. 추론 시 복원 후 attention 수행. **Cache 절감** O, attention FLOPs 절감 X.

MLA vs CCA: 핵심 차이



관점	MLA	CCA
압축 대상	Cache에 저장할 K/V 표현	Q, K, V **모두** + attention 연산 공간
Attention 계산	Full head space에서 수행	**Compressed latent space**에서 직접 수행
주요 이득	KV cache 절감	KV cache + attention FLOPs 절감
보강 장치	Latent reconstruction	Q/K convolutional mixing

Convolutional Mixing




```python
class CCAConvMixing(nn.Module):
    def __init__(self, head_dim, kernel_size=3):
        super().__init__()

        # mixes nearby tokens independently per channel
        self.seq_conv = nn.Conv1d(
            in_channels=head_dim,
            out_channels=head_dim,
            kernel_size=kernel_size,
            padding=kernel_size - 1,
            groups=head_dim,   # depthwise
        )

        # mixes channels inside the compressed head
        self.channel_conv = nn.Conv1d(
            in_channels=head_dim,
            out_channels=head_dim,
            kernel_size=1,
        )

    def _mix(self, x):
        batch, seq, heads, head_dim = x.shape

        # Conv1d wants [batch, channels, seq]
        x = x.permute(0, 2, 3, 1)                # [batch, heads, head_dim, seq]
        x = x.reshape(batch * heads, head_dim, seq)

        # causal sequence convolution
        x = self.seq_conv(x)
        x = x[..., :seq]                         # remove right padding

        # channel mixing within each head
        x = self.channel_conv(x)

        # back to [batch, seq, heads, head_dim]
        x = x.reshape(batch, heads, head_dim, seq)
        x = x.permute(0, 3, 1, 2)

        return x

    def forward(self, q, k):
        # q, k: [batch, seq, heads, head_dim]

        q = self._mix(q)
        k = self._mix(k)

        return q, k
```

Compressed Q, K에 **sequence-mixing**과 **channel-mixing**를 순차적으로 convolution 적용.

- 압축된 Q/K에 **local context** 부여 후 attention score 계산
- **V에는 미적용** — content 자체를 담당, attention score와 무관
- 동일 압축률에서 **CCA > MLA** 성능 (CCA paper, Oct 2025)

CCA 실험 결과

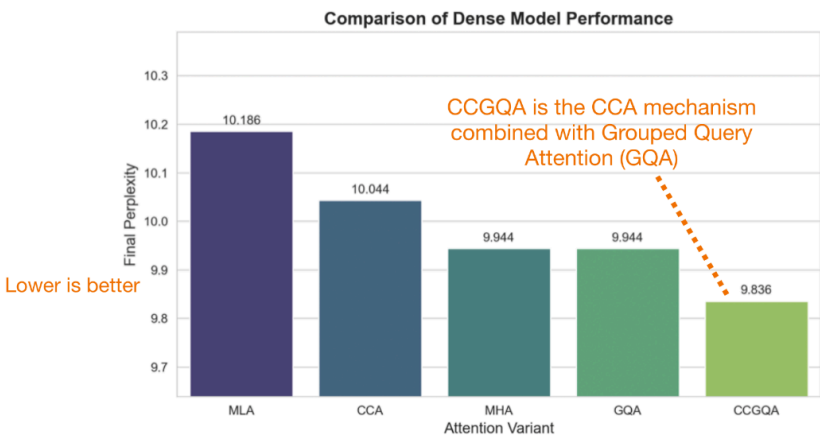


Fig. 3: Comparison of perplexity on the Zyda2 dataset for 1B parameter dense transformer models trained for 300B tokens with different attention mechanisms. CCA beats MLA in the parameter-matched setting with less FLOPs. When matching CCA FLOPs to GQA and MHA via CCGQA, we see a substantial improvement in perplexity.

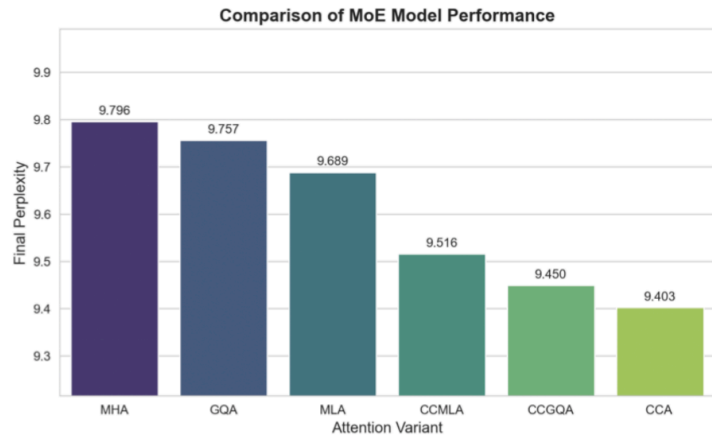


Fig. 4: Comparison of perplexity on 50B tokens of the Zyda2 dataset for 350M/1.5B parameter proprietary MoE models with different attention mechanisms. Our proposed methods, CCA and CCGQA, achieve lower loss than GQA and MLA at equivalent parameter counts with less compute cost, and less training parameters in the case of MLA.

동일 compression setting에서 CCA가 MLA보다 우수.

ZAYA1-8B: **GQA + CCA** 결합, 극단적 sparse MoE (토큰당 routed expert 1개).

DeepSeek V4: 개요

SEBASTIANRASCHKA.COM/LLM-ARCHITECTURE-GALLERY/

Percent Active Parameters per Token

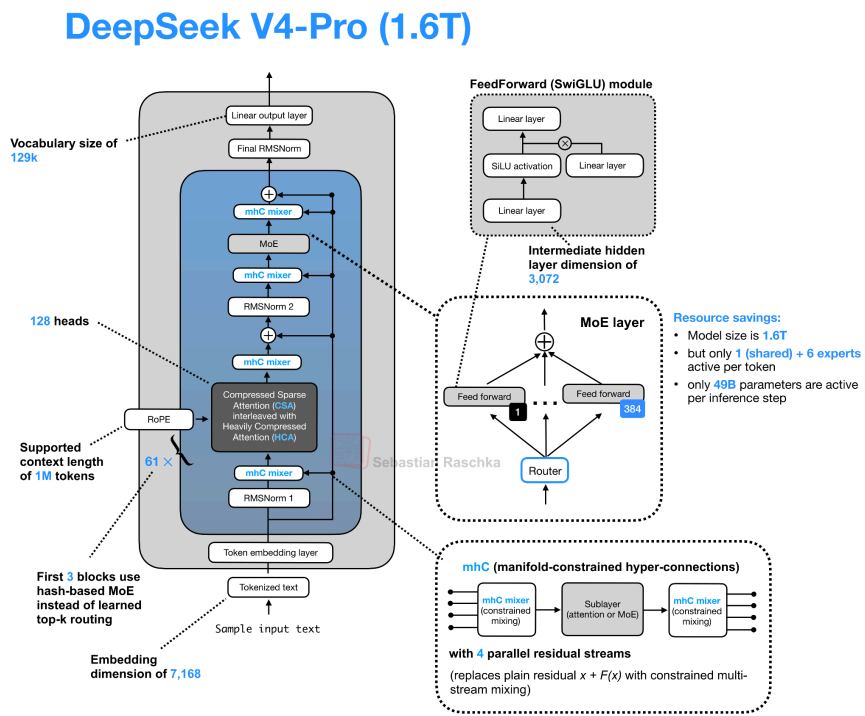
#	MODEL	ACTIVE %	ACTIVE PARAMS	TOTAL PARAMS	TYPE	DATE	ATTENTION
1	DeepSeek V4-Pro	3.1%	49B active	1.6T	MoE	2026-04-24	CSA/HCA
2	Kimi K2	3.2%	32B active	1T	MoE	2025-07-10	MLA
3	Kimi K2.5	3.2%	32B active	1T	MoE	2026-01-27	MLA
4	Kimi K2.6	3.2%	32B active	1T	MoE	2026-04-20	MLA
5	Arcee AI Trinity Large 400B	3.3%	13B active	400B	MoE	2026-01-27	3:1 sliding-window/global gated GQA
6	Qwen3 Next 80B-A3B	3.8%	3B active	80B	Hybrid	2025-09-09	3:1 Gated DeltaNet and Gated Attention
7	Llama 4 Maverick	4.3%	17B active	400B	MoE	2025-04-05	GQA
8	MiniMax M2 230B	4.3%	10B active	230B	MoE	2025-10-23	GQA

2026년 최대 릴리스. **V4-Pro**는 MoE 중 최저 active-parameter 비율.

핵심 아키텍처 혁신:

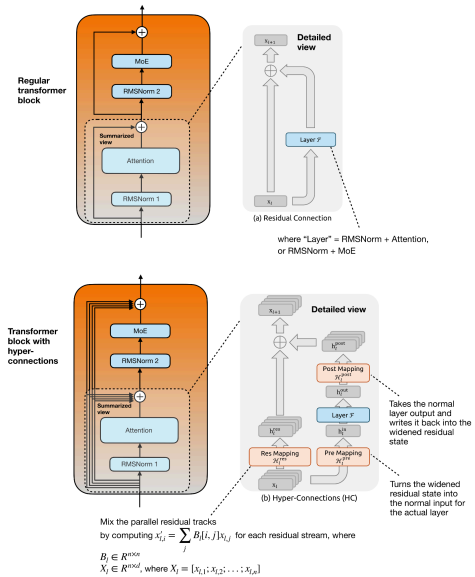
1. **mHC** — residual pathway 개선
2. **CSA/HCA** — attention path 압축

DeepSeek V4-Pro 아키텍처



mHC + CSA/HCA + MoE FFN이 하나의 block에 결합. MLA 대신 **sequence dimension 자체를 압축**하는 새 attention 방식.

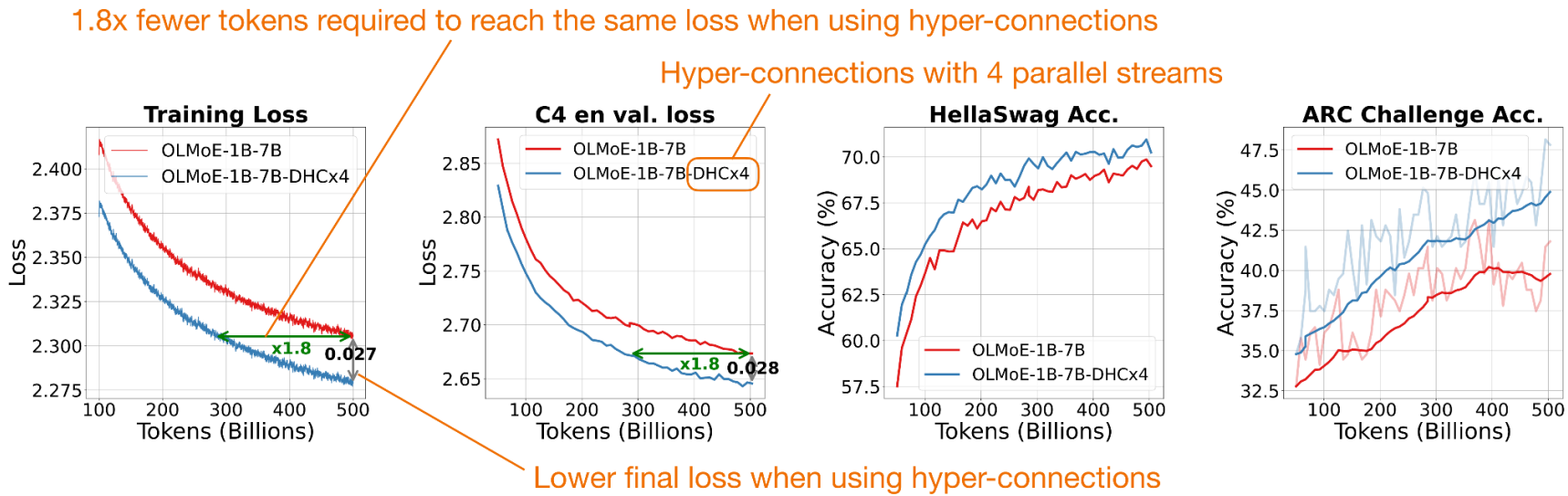
5-1. Manifold-Constrained Hyper-Connections (mHC)



위: 일반 transformer — 단일 residual stream.
아래: HC — **parallel residual streams** + Pre/Post Mapping으로 입출력 분배(Pre는 하나로 합치고 Post는 다시 여러개로 분배)

Hidden size는 유지하면서 **residual pathway만 넓혀** 표현력을 높인다.

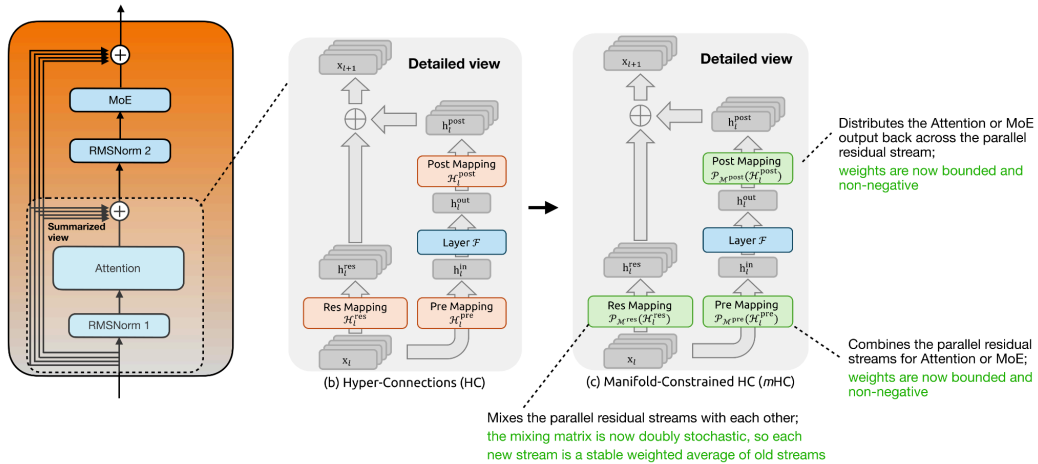
HC의 성능과 효율



Zhu et al. (2024) 결과:

- 7B OLMo: FLOPs 13.36G → 13.38G (거의 무변화)
- **~50% 적은 token**으로 baseline 도달
- 핵심 이득은 학습 효율, compute overhead minimal

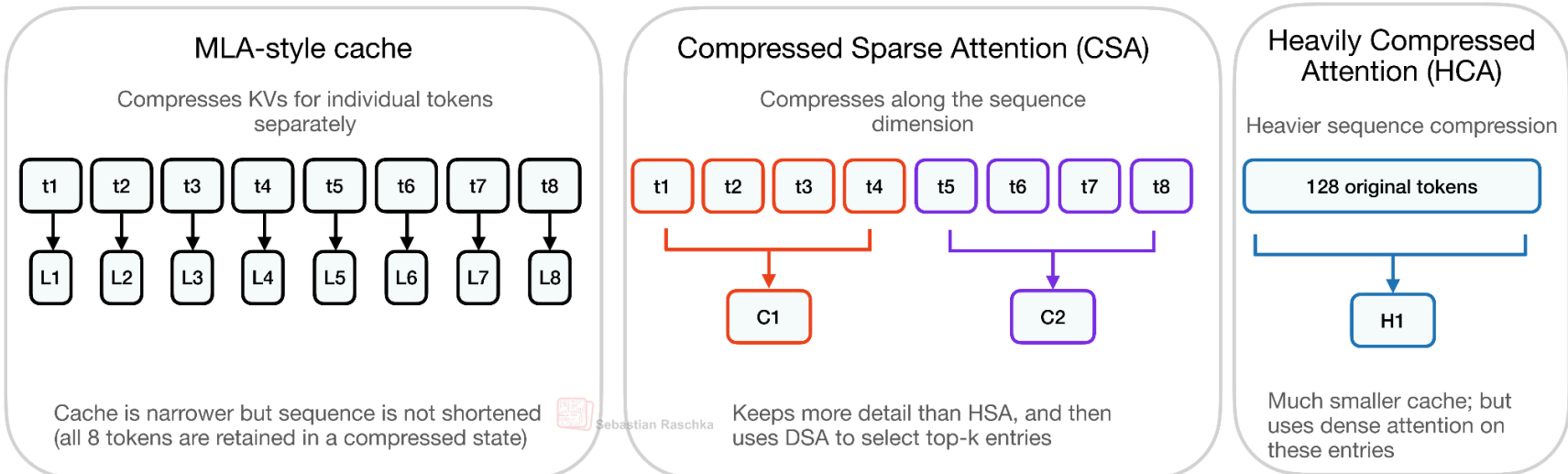
HC에서 mHC로



관점	HC	mHC
Mixing matrix	비제약	**Doubly stochastic** manifold 투영
안정성	신호 증폭/소멸 가능	행/열 합 = 1 → **안정적 재분배**
Pre/Post Mapping	비제약	비음수 + bounded
학습 overhead	—	4 streams 기준 **6.7%**

mHC는 deep/large-scale에서 HC의 안정성 문제를 해결.

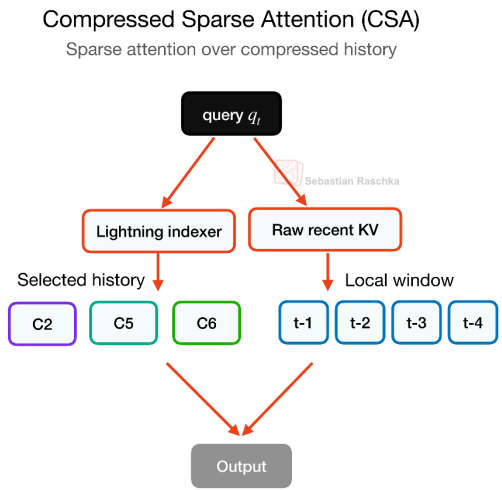
6. DeepSeek V4: CSA/HCA

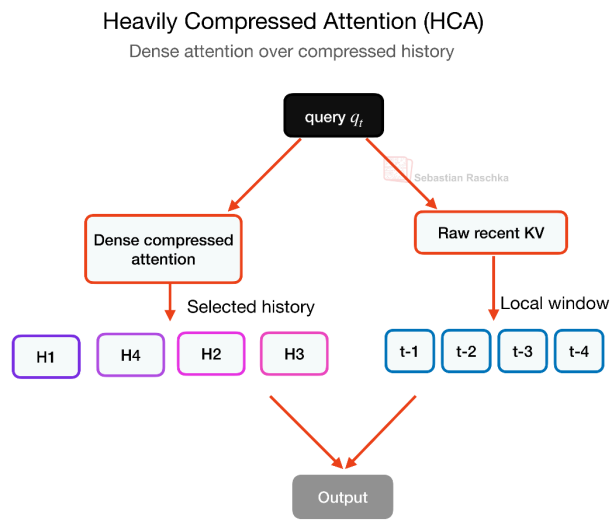


MLA: token별 latent cache → sequence 길이 유지.
CSA: m=4 compression + sparse selection → detail 보존.
HCA: m'=128 heavy compression + dense → global coverage.

MLA는 **per-token representation** 압축, CSA/HCA는 **sequence dimension** 자체를 압축.

CSA와 HCA의 역할 분담

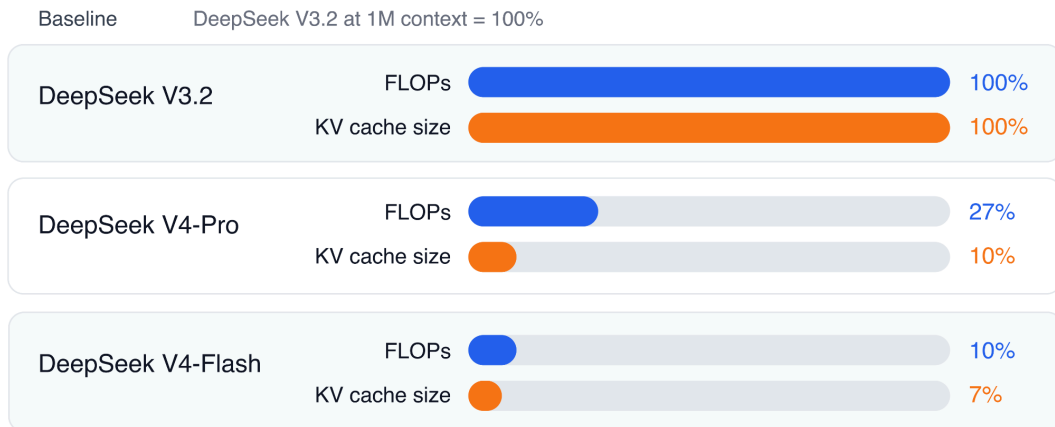




메커니즘	압축 방향	Attention type	Trade-off
CSA	Mild (m=4)	Sparse top-k	Detail 보존 + 비용 절감
HCA	Heavy (m'=128)	Dense	싼 global coverage, 정보 손실 가능

최근 128 token은 sliding-window로 **원본 KV 유지**. CSA/HCA layer를 **번갈아** 배치하여 상호 보완.

1M Context 효율 비교

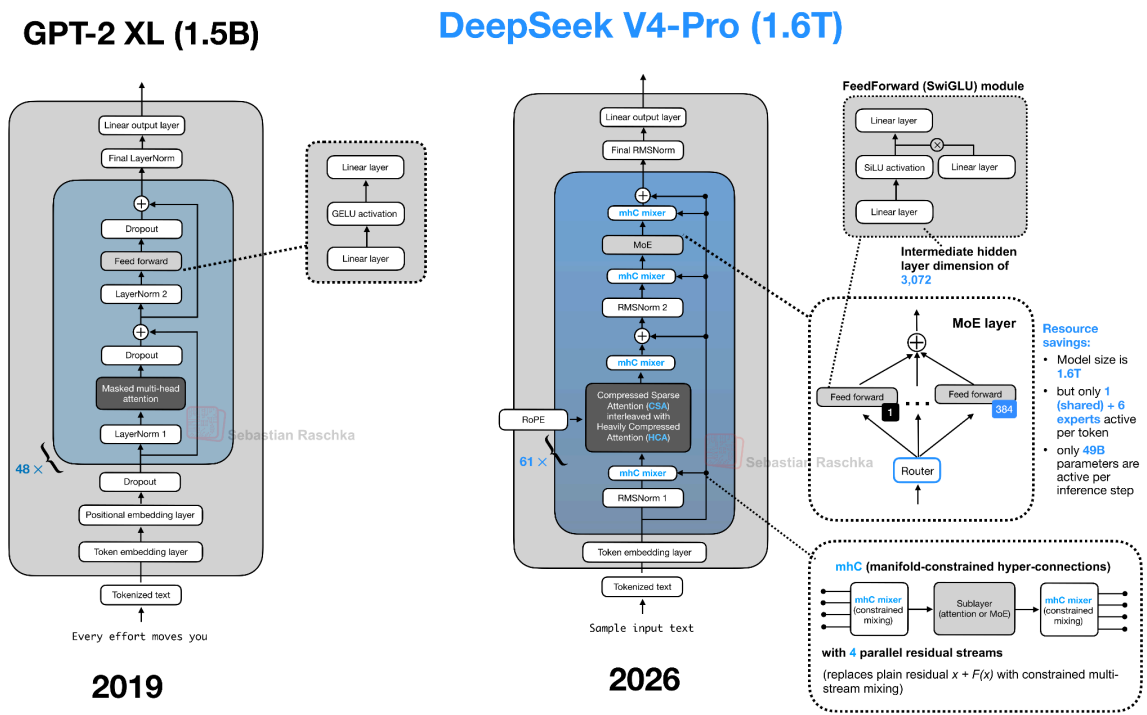


DeepSeek V3.2를 100%로 했을 때, **1M context** 기준:

모델	Single-token FLOPs	KV Cache Size
V4-Pro	27%	10%
V4-Flash	10%	7%

V4-Flash-Base가 V3.2-Base를 대다수 벤치마크에서 상회하면서 비용 극감. 단, data·Muon·mHC·precision 등 전체 레시피 효과 합산이라 개별 기여도 판단은 어렵다.

2019에서 2026으로



GPT-2 XL의 단순 block → DeepSeek V4-Pro의 복잡 block. Attention, MoE, residual, cache 모두 특화. **코드 복잡도 ~10배 증가**, 런타임 비용은 감소.

결론: 핵심 정리

1. **Gemma 4** — KV cache를 cross-layer sharing으로 줄이고, PLE로 embedding capacity 보강
2. **Laguna XS.2** — Layer별 attention 예산 차등 배정 (비싼 layer에 적은 Q heads)
3. **ZAYA1-8B** — Compressed latent space에서 attention 자체를 수행 (CCA)
4. **DeepSeek V4** — Residual path는 mHC로, long-context attention은 CSA/HCA로 혁신
5. **공통점** — Transformer를 버리지 않고, **block 내부를 장문맥 서빙에 맞게 세분화**

> "Transformer는 여전히 SOTA의 중심이다. 매 분기 새로운 tweaks가 추가되지만, 기본 레시피는 GPT-2(2019) 이후 지금까지 이어지고 있다."

토론 질문

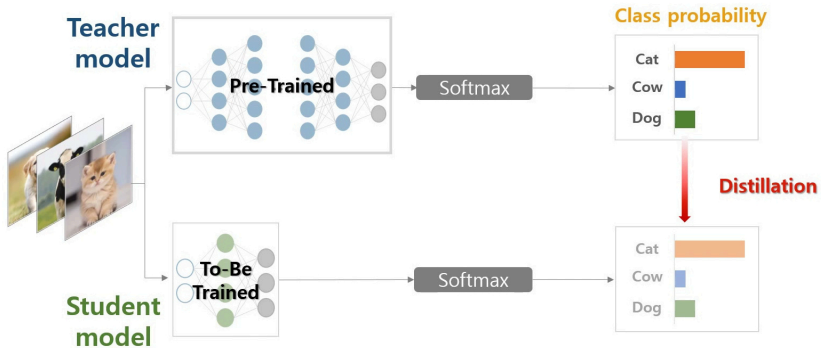
- **Long-context 비용**을 품질만큼 중요하게 측정하고 있는가?
- KV cache 절감 vs attention FLOPs 절감, **어떤 워크로드**에서 차이가 큰가?
- mHC처럼 residual path를 바꾸면 **어떤 failure mode**가 생기나?
- 코드 복잡도 10배 vs 런타임 비용 절감, **수용 한계**는?

참고 자료

- Sebastian Raschka, *"Recent Developments in LLM Architectures"*, Ahead of AI, May 2026
- Brandon et al., *"Reducing Transformer Key-Value Cache Size with Cross-Layer Attention"*, NeurIPS 2024
- Zhu et al., *"Hyper-connections"*, 2024
- DeepSeek, *"mHC: Manifold-Constrained Hyper-Connections"*, Dec 2025
- Zyphra, *"Compressed Convolutional Attention"*, Oct 2025
- LLM Architecture Gallery: `sebastianraschka.com/llm-architecture-gallery/`

Knowledge Distillation

Knowledge Distillation



핵심 아이디어

큰 **Teacher Model**이 만든 출력 분포를 작은 **Student Model**의 학습 목표로 사용한다.

- Teacher는 이미 학습된 크고 깊은 모델
- Student는 더 작고 빠른 모델
- 목표: Teacher의 예측 정확도와 일반화 패턴을 Student로 이전
- 기본 지식(Knowledge)은 Teacher의 **soft output distribution**

> 정답뿐 아니라 Teacher의 “헷갈림 패턴”까지 학습한다.

왜 Distillation인가?

Hard Label vs Soft Label

Hard Label (one-hot): 정답만 1, 나머지 0

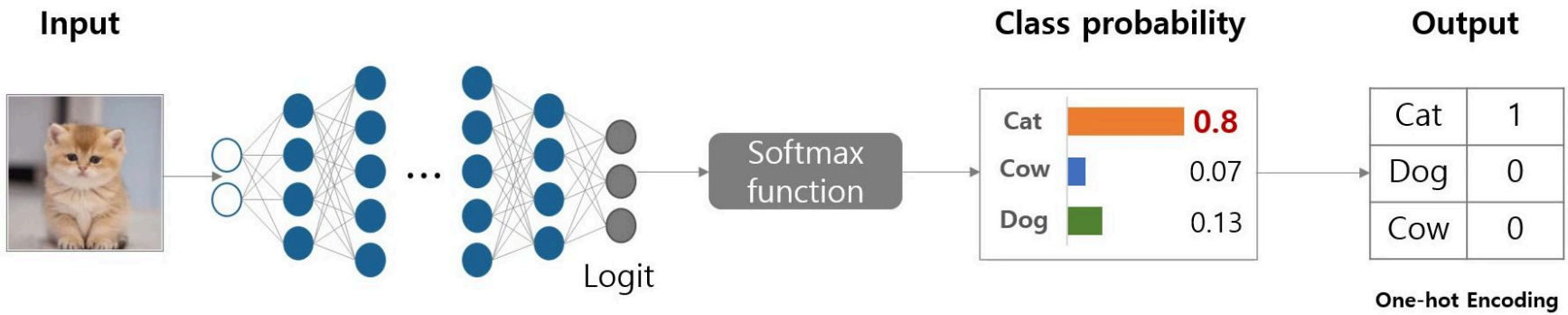
```
"고양이" → [0, 0, 1, 0, 0, ...]
```

Soft Label (Teacher 출력): 정답 외에도 유사 토큰에 확률 분배

```
"고양이" → [0.01, 0.05, 0.70, 0.15, 0.02, ...]
                          ↑      ↑
                        고양이   강아지 (유사 개념에도 확률 부여)
```

> Soft Label에는 **Dark Knowledge**(암묵지)가 담겨 있다 — "고양이 ≈ 가ㅈ아지 ≠ 자동차"라는 관계 정보.

Hard Target: 정답만 남기는 학습 신호



흐름

`input` → `logit` → `softmax` → `one-hot label`

학습 신호

정답만 1, 나머지 0

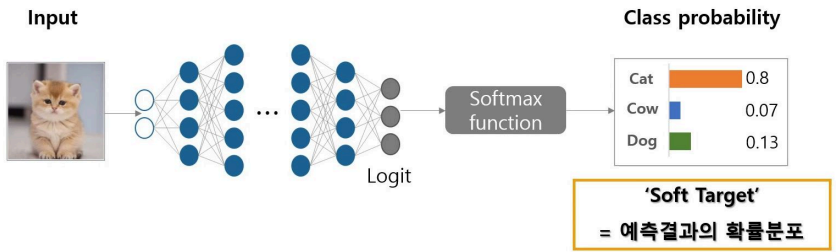
```
Cat = 1
Dog = 0
Cow = 0
```

한계

softmax 출력의 상대적 관계가 one-hot 변환 시 사라진다.

> 정답은 알려주지만, 오답 간 유사도(dark knowledge)는 버린다.

Hard Target vs Soft Target



왜 Soft Target인가?

Hard Target은 정답만 알려준다.

```
Cat = 1, Cow = 0, Dog = 0
```

Soft Target은 Teacher의 예측 확률 분포를 보존한다.

```
Cat = 0.80, Cow = 0.07, Dog = 0.13
```

- Hard Target: 클래스 간 관계 정보가 거의 없음
- Soft Target: 오답 클래스의 상대적 가능성까지 포함
- 이 확률 분포가 바로 **Dark Knowledge**
- Student는 정답뿐 아니라 클래스 간 유사도까지 학습

Soft KD에서 같은 토크나이저가 권장되는 이유

> *표준 **토큰 단위 Soft KD**는 각 위치 t에서 Teacher와 Student의 확률 분포를 비교한다.*

$$L_{\text{KD}} = \frac{1}{N} \sum_{t=1}^{N} \text{KL}\left(P_{\text{teacher}}^{(t)} \parallel P_{\text{student}}^{(t)} \right)$$

① 어휘 차원과 의미 일치

- 출력 로짓의 차원인 **어휘 크기 $|V|$**가 가ㅌ아야 한다.
- 같은 인덱스 i가 **동일한 토큰**을 가리켜야 한다.
- 어휘 크기만 같아도 토큰 ID의 의미가 다르면 직접 비교할 수 없다.

② 시퀀스 길이와 토큰 경계 일치

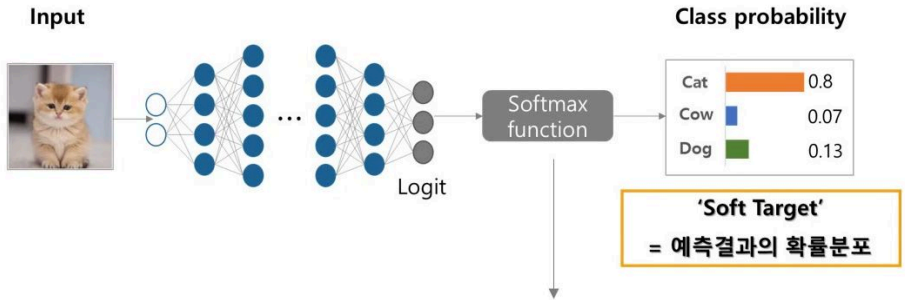
- 같은 텍스트의 **토큰 수 N**과 **분할 경계**가 가ㅌ아야 한다.
- 각 위치 t가 **같은 문맥의 다음 토큰 예측**에 대응해야 한다.

토큰 분할 예시:

```
Teacher: [나는] [학교에] [간다]     → 3개
Student: [나는] [학교] [에] [간다] → 4개
```

두 번째 토큰까지 읽으면 Teacher는 **“나는 학교에”**, Student는 **“나는 학교”** 다음을 예측한다. 문맥이 달라 **1:1 비교가 어긋나며, 패딩으로 길이만 맞춰도 해결되지 않는다.**

Temperature: 확률 분포를 부드럽게 만들기



$$Softmax(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad \Rightarrow \quad Softmax(z_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

문제: softmax가 너무 뾰족해짐

최대 logit이 확률 대부분을 차지하고, 나머지는 거의 0이 된다.

```
Before: [8.0, 2.0, 1.5]
After softmax: [0.994, 0.002, 0.001]
```

오답 클래스 사이의 상대적 관계가 잘 드러나지 않는다.

개선: Temperature 사용

logit을 T로 나눈 뒤 softmax를 적용한다.

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Temperature Scaling

> *Softmax에 **temperature** T를 적용하여 확률 분포의 평탄도를 조절*

$$p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

Temperature	효과	분포 형태
$T = 1$ (기본)	일반 softmax	뾰족한 분포 (peak에 집중)
$T > 1$ (증류용)	분포 평탄화	부드러운 분포 (dark knowledge 노출)
$T \to \infty$	균등 분포에 수렴	모든 토큰에 비슷한 확률

MiniMind 기본값: `temperature = 1.5` (권장 범위 1.0 ~ 2.0)

> T를 높이면 오답 토큰 간 **상대적 관계**까지 Student에게 전달된다.

실제 학습은 CE + Soft Distillation

> *정답 토큰을 맞히는 **CE**와 Teacher 분포를 따르는 **KL**을 함께 사용한다.*

$$\mathcal{L} = \alpha \mathcal{L}_{\text{CE}} + (1 - \alpha)\, T^2\, D_{\text{KL}}(p_T^T \parallel p_S^T)$$

```
# distillation_loss 내부에 temperature ** 2 보정 포함
loss = alpha * ce_loss + (1 - alpha) * distill_loss
```

`alpha`	학습 신호
`0`	Soft distillation만 사용
`0.5` (기본값)	CE와 증류 손실에 각각 0.5의 계수 적용
`1`	정답 기반 CE만 사용

기본 온도는 ** `temperature=1.5` **이다.

왜 T^2를 곱할까? — 온도를 높였을 때 약해지는 증류 학습의 기울기를 보정하기 위해

- T를 높이면 확률 분포가 부드러워져, Student가 Teacher의 **정답 외 후보 토큰에 대한 상대적 선호**도 배우기 쉬워진다.
- 이때 Student logits에 전달되는 KL 손실의 기울기는 대략 $1/T^2$로 작아진다(높은 온도에서의 근사).
- 따라서 KL 손실에 **T^2를 곱해 학습 신호의 크기를 보정**한다.

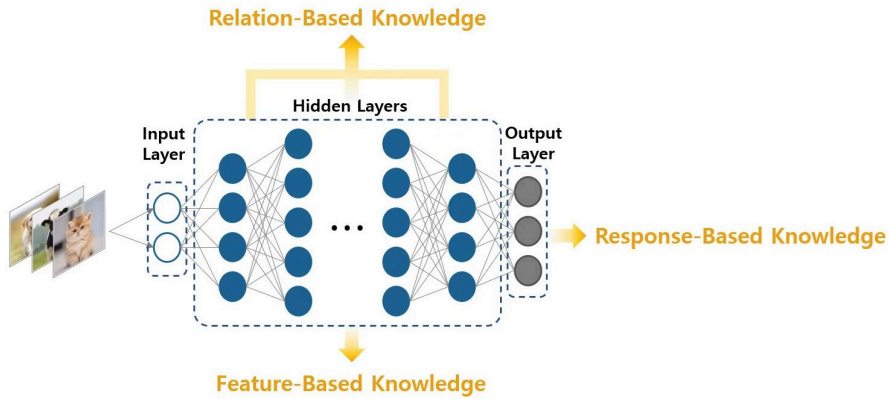
Distillation Loss 함수

```python
def distillation_loss(student_logits, teacher_logits, temperature=1.0):
    with torch.no_grad():
        teacher_probs = F.softmax(teacher_logits / temperature, dim=-1)

    student_log_probs = F.log_softmax(student_logits / temperature, dim=-1)

    kl = F.kl_div(
        student_log_probs,
        teacher_probs,
        reduction='batchmean'
    )
    return (temperature ** 2) * kl
```

왜 logits을 입력으로 받는가?

`logits / T` 는 softmax **이전**에 적용해야 하므로, 반드시 raw logits을 입력으로 받는다.

어떤 지식을 넘길 것인가?



Response-Based

출력층의 logit/soft target 전달. 기본 KD 방식

Feature-Based

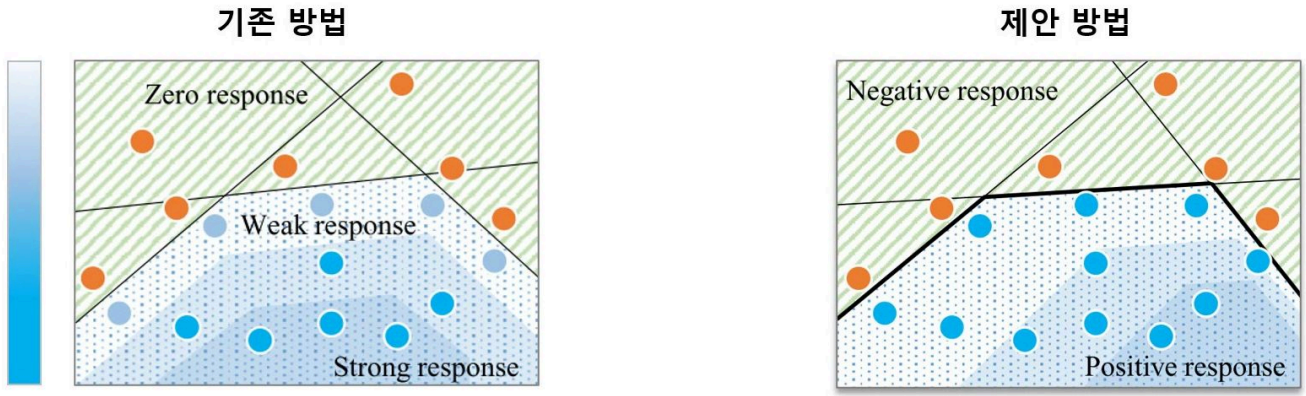
hidden layer의 activation, feature map을 맞춘다.

Relation-Based

샘플 간 거리·각도·embedding 구조를 전달한다.

> 기본 KD는 response-based이지만, 더 강한 증류는 feature와 relation까지 활용한다.

Feature-Based Knowledge: Activation Boundary



Activation Boundary란?

hidden neuron이 입력 공간을 나누는 경계. class 소속 강도와 경계 근처의 모호함을 담는다.

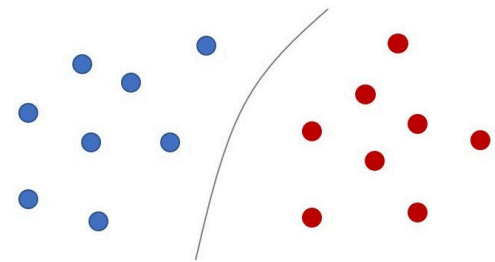
왜 지식인가?

출력 확률이 아닌, Teacher의 hidden layer decision structure를 Student가 모방한다.

> 정답 분포를 넘어, Teacher의 내부 표현 방식까지 전달하는 접근이다.

왜 Activation Boundary를 가져오는가?

좋은 Decision boundary → 좋은 일반화 성능



좋은 Decision Boundary

잘 나눈 경계는 unseen 데이터에서도 안정적 예측을 만든다.

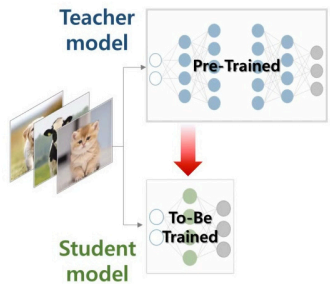
좋은 일반화 성능

Teacher의 activation boundary를 따라가면, 정답 암기 대신 공간 분할 방식을 학습한다.

> Hidden layer의 decision structure를 전달하여 Student의 일반화 성능을 높인다.

Activation Boundary Matching Loss

목표: Teacher와 Student의 Activation Boundary만 같아지도록 학습



$$\rho(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

- $\mathbf{1}$: 모든 성분이 1로 구성된 벡터
- $\odot$: element-wise 곱
- $T(x_i)$: Teacher 모델의 히든 레이어의 반응벡터
- $S(x_i)$: Student 모델의 히든 레이어의 반응벡터
- μ : 분류경계면의 margin

$$L_{activation} = \left\| \rho(T(x_i)) - \rho(S(x_i)) \right\|_1$$

목표

Teacher와 Student의 hidden layer 반응이 같은 쪽 boundary에 놓이도록 학습한다.

$$\rho(x) = \begin{cases} 1, & x > 0 \\ 0, & otherwise \end{cases}$$

손실

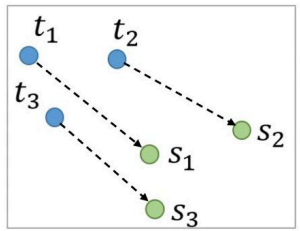
$T(x_i)$와 $S(x_i)$를 이진 패턴으로 변환 후 차이를 최소화한다.

$$L_{activation} = ||\rho(T(x_i)) - \rho(S(x_i))||_1$$

> Activation 값 복사가 아닌, 같은 boundary를 갖도록 맞추는 것이 핵심이다.

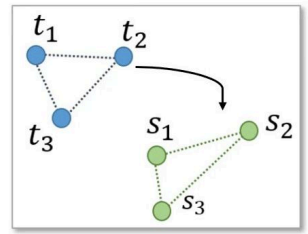
Relation-Based Knowledge: 구조를 전달하기

기존 방법



Individual wise

제안 방법



Structure wise

Individual-wise

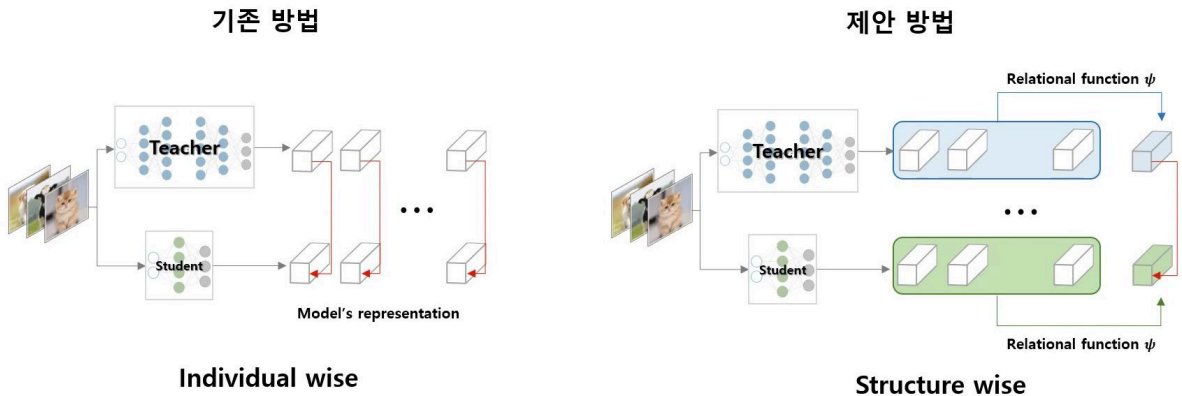
Teacher t_i와 Student s_i를 일대일로 맞춘다.

Structure-wise

개별 점보다 샘플 간 거리·각도·배치 구조를 맞춘다.

> 개별 위치보다 샘플 간 관계 보존에 초점을 둔다.

Relation Function: 표현의 구조 비교



기존 방식

같은 위치의 feature를 직접 맞추는 방식. 개별 값에 초점.

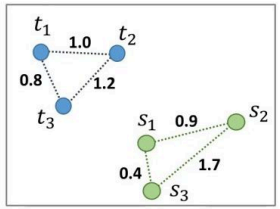
제안 방식

ψ로 representation 구조를 계산하고, Teacher-Student 구조를 일치시킨다.

> Feature 값 복사가 아닌, 샘플 간 거리와 배치 형태를 보존한다.

Distance-wise vs Angle-wise Relation

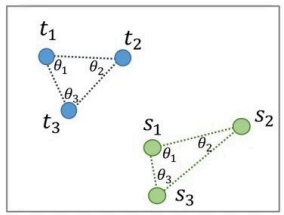
Distance-wise Relational function



$$\psi_D(t_i, t_j) = \frac{1}{\mu}\|t_i - t_j\|_2$$

$$\mu = \frac{1}{|\chi^2|}\sum_{(x_i, x_j)\in\chi^2}\|t_i - t_j\|_2$$

Angel-wise Relational function



$$\psi_A(t_i, t_j, t_k) = cos\angle t_i t_j t_k = \langle e^{ij}, e^{kj}\rangle$$

$$where\ e^{ij} = \frac{t_i - t_j}{\|t_i - t_j\|_2},\ e^{kj} = \frac{t_k - t_j}{\|t_k - t_j\|_2}$$

Distance-wise

쌍 (t_i, t_j)의 거리 비율을 맞춘다. Teacher에서 가까운 샘플은 Student에서도 가깝게 유지.

Angle-wise

세 샘플 (t_i, t_j, t_k)의 각도를 맞춘다. 거리보다 풍부한 구조 정보 전달.

> Embedding 공간의 거리·각도를 보존하여 Teacher의 데이터 geometry를 전달한다.

MiniMind: 같은 입력으로 두 모델 평가

> *학생의 다음 토큰 확률 분포를 교사의 분포에 가깝게 만듭니다.*

```
res = model(input_ids)
student_logits = res.logits[..., :-1, :]

with torch.no_grad():
    teacher_logits = teacher_model(input_ids).logits[..., :-1, :]
```

- `logits` : 각 위치에서 다음 토큰 후보마다 매긴 점수
- 교사: 고정된 분포를 제공하며 기울기를 기록하지 않음
- 학생: 정답 토큰과 교사의 후보별 확률을 이용해 학습

다음 토큰 예측: 각 위치의 역할

> *전체 입력을 `input_ids = [나는, 밥을, 먹었다, 종료]` 라고 가정합니다.*

위치	해당 위치까지 볼 수 있는 내용	출력이 예측하는 대상
0	나는	밥을
1	나는 밥을	먹었다
2	나는 밥을 먹었다	종료
3	나는 밥을 먹었다 종료	종료 다음 토큰

Causal attention mask가 미래 토큰을 가리므로, 다음 토큰을 미리 볼 수 없습니다.

예측과 정답의 위치 맞추기

입력 위치의 토큰	모델 출력	비교할 다음 토큰 정답
나는	$logits_0$	밥을
밥을	$logits_1$	먹었다
먹었다	$logits_2$	종료
종료	$logits_3$	없음

```
student_logits = res.logits[..., :-1, :]  # 마지막 출력 제외
shift_labels = labels[..., 1:]           # 첫 정답 위치 제외
```

전체 입력으로 순전파한 뒤, 손실 계산에서 마지막 출력을 제외합니다.

증류: 같은 위치의 두 분포 비교

```
student_logits = student_outputs.logits[..., :-1, :]
teacher_logits = teacher_outputs.logits[..., :-1, :]
```

예를 들어 위치 1은 두 모델 모두 **“나는 밥을” 다음 토큰**을 예측합니다.

후보	교사 확률	학생 확률
먹었다	70%	40%
먹는다	20%	35%
기타	10%	25%

같은 문맥에서 교사가 후보마다 준 확률까지 학습 신호로 활용합니다.

마스킹: assistant 응답만 선택

```
loss_mask = (labels[..., 1:] != -100).float()
loss_mask_flat = loss_mask.view(-1)

student_selected = student_logits.view(-1, vocab_size)[loss_mask_flat == 1]
teacher_selected = teacher_logits.view(-1, vocab_size)[loss_mask_flat == 1]
```

질문·패딩 등 `labels == -100` 인 정답 위치는 제외합니다.

단계	텐서 형태
다음 토큰 위치 정렬 후	`[배치 크기, 시퀀스 길이 - 1, 어휘 크기]`
응답 위치 선택 후	`[학습할 토큰 수, 어휘 크기]`

온도 적용: 확률과 로그 확률

```
teacher_probs = F.softmax(teacher_logits / temperature, dim=-1)
student_log_probs = F.log_softmax(student_logits / temperature, dim=-1)
```

변환	결과	용도
교사 `softmax`	확률 P_T	목표 분포
학생 `log_softmax`	로그 확률 $\log P_S$	KL 계산 입력

`F.kl_div()`의 기본 입력 규약에 맞춰 서로 다른 형태로 전달합니다.

`T > 1`이면 분포가 부드러워져 차선 후보의 확률도 드러납니다.

MiniMind 기본값은 `temperature = 1.5`입니다. CE에는 이 온도를 적용하지 않습니다.

KL 계산: 교사 분포와의 차이

```
kl = F.kl_div(
    student_log_probs,
    teacher_probs,
    reduction='batchmean'
)
```

각 응답 토큰 위치에서 전체 어휘 후보의 차이를 합산합니다.

$$D_{\mathrm{KL}}(P_T \parallel P_S) = \sum_v P_T(v) \log \frac{P_T(v)}{P_S(v)}$$

교사 분포 P_T가 목표이며, 학생 분포 P_S가 가ㅌ아지면 KL은 `0` 입니다.

KL 계산 예제

> *온도 적용 후, 후보가 세 개인 분포를 가정합니다.*

후보	교사 P_T	학생 P_S
서울	0.7	0.4
부산	0.2	0.4
제주	0.1	0.2

```
KL = 0.7 × ln(0.7 / 0.4)
   + 0.2 × ln(0.2 / 0.4)
   + 0.1 × ln(0.1 / 0.2)
   ≈ 0.184
```

교사와 학생의 확률이 모두 같으면 각 로그 비율이 0이 됩니다.

토큰 평균과 온도 제곱 보정

> *입력이 `[학습할 토큰 수 N, 어휘 크기 V]` 이므로 `batchmean` 은 토큰별 KL의 평균입니다.*

$$L_{\text{distill}} = T^2 \times \frac{1}{N} \sum_{n=1}^{N} D_{\text{KL}}(P_{T,n} \parallel P_{S,n})$$

```
return (temperature ** 2) * kl
```

```
T = 1.5이면 T² = 2.25
평균 KL = 0.184라고 가정하면
distillation loss ≈ 2.25 × 0.184 ≈ 0.414
```

온도로 작아지는 기울기 크기를 보정하기 위해 T^2를 곱합니다.

정답 학습과 증류 손실 혼합

```
loss = (
    alpha * ce_loss
    + (1 - alpha) * distill_loss
) / args.accumulation_steps
```

손실	학생이 학습할 내용
`ce_loss`	데이터의 정답 토큰 맞히기
`distill_loss`	교사의 전체 후보 확률 분포 따라 하기

```
alpha = 0.5, CE loss = 1.2, Distillation loss = 0.4
혼합 손실 = 0.5 × 1.2 + 0.5 × 0.4 = 0.8
```

혼합 손실을 기울기 누적 횟수로 나눈 뒤 역전파합니다.

주요 하이퍼파라미터

파라미터	기본값	설명
`alpha`	0.5	CE vs Distill 비율
`temperature`	1.5	증류 온도 (1.0 ~ 2.0 권장)
`student_hidden_size`	768	Student 모델 차원
`teacher_use_moe`	1	Teacher에 MoE 사용
`student_use_moe`	0	Student는 Dense

정리: Distillation 핵심 요약

세 줄 요약

1. **Teacher의 soft output**을 temperature로 평탄화하여 dark knowledge를 노출시킨다
2. **KL Divergence**로 Student가 Teacher의 출력 분포를 모방하도록 학습한다
3. **CE + Distill 혼합 손실**로 정답 학습과 지식 전달을 동시에 수행한다

Speculative Decoding

Speculative Decoding

> *LLM 추론 지연(Latency)을 줄이는 핵심 전략*

GPU는 막대한 연산 능력을 갖고 있지만,
Autoregressive 생성은 본질적으로 **순차적**이기 때문에
대부분의 연산 자원이 **유휴 상태**로 남는다.

출처: NVIDIA Developer Blog — "An Introduction to Speculative Decoding" (Sep 2025)

Speculative Decoding이란?

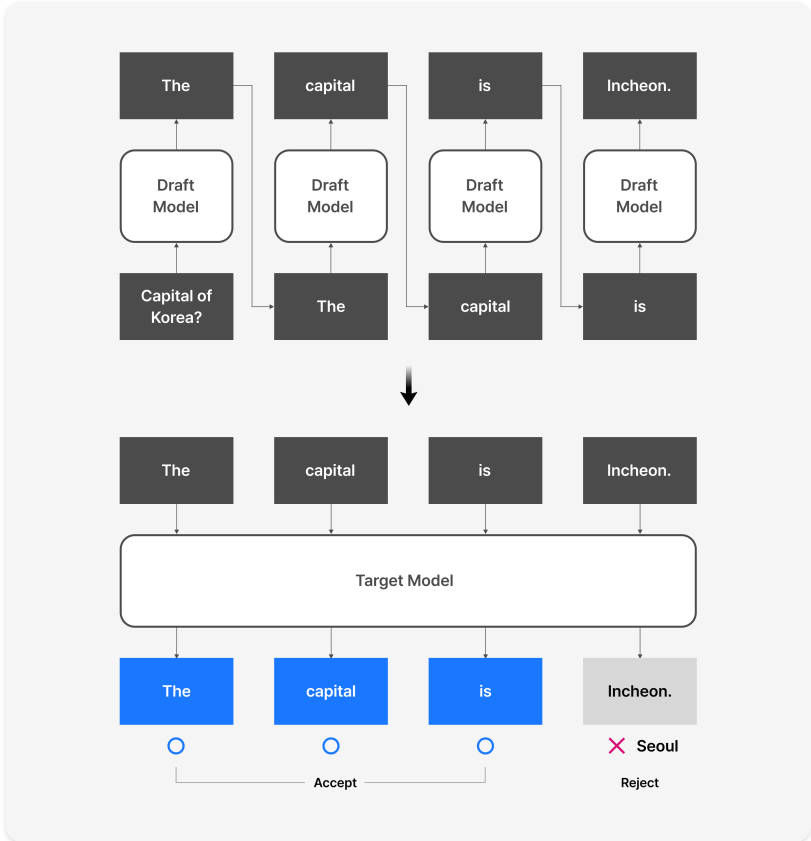
> ***Target 모델 + 경량 Draft 메커니즘**을 결합하여 여러 토큰을 동시에 생성*

1. **Draft 단계**: 작고 빠른 모델이 여러 후보 토큰을 빠르게 제안 (3~12개)
2. **Verify 단계**: Target 모델이 **단일 forward pass**로 모든 후보를 동시 검증
3. **Accept/Reject**: 가장 긴 일치 prefix를 수락하고 계속 진행

비유: 수석 과학자(Target)가 경험이 적지만 효율적인 조수(Draft)에게 일상적인 실험을 맡기는 것과 같다. 조수가 빠르게 체크리스트를 처리하고, 과학자는 검증과 수정에 집중한다.

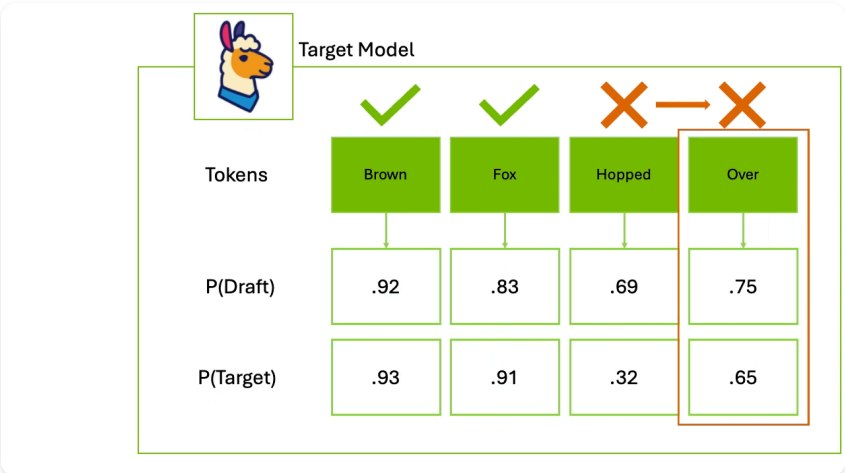
Draft-Target: 동작 원리

> *Draft 모델이 후보 토큰을 제안하고, Target 모델이 단일 forward pass로 검증*



Rejection Sampling: 수락 논리

Draft 토큰의 수락/거부를 결정하는 핵심 메커니즘



각 토큰에서 P_{target}과 P_{draft}를 비교하여 수락 여부 결정

수락: $P_{\text{target}}(x_i) \geq P_{\text{draft}}(x_i)$
Target도 높은 확률 부여 → Draft 예측이 정확

거부: $P_{\text{target}}(x_i) < P_{\text{draft}}(x_i)$
Draft가 과신 → **이후 모든 Draft 토큰도 폐기**

Best: Draft 전부 수락 → 1회 forward pass로 K+1개 토큰 생성
Worst: Draft 전부 거부 → 1개 토큰만 생성 (= AR과 동일)
→ **AR보다 절대 느려지지 않는다**

Draft 모델 선정 기준

> *어떤 Draft를 쓰느냐가 Speculative Decoding의 성능을 결정한다*

같은 계열 소형 모델

- Target과 **동일한 학습 데이터·토크나이저** → 높은 채택률
- Llama-3-70B → **Llama-3-8B**
- GPT-4 → **GPT-3.5**

가장 일반적이고 안정적인 선택

크기 비율: 1/5 ~ 1/10

- **너무 작으면**: 채택률 ↓, 거부 빈번
- **너무 크면**: Draft 자체가 느림, 가속 효과 ↓
- **균형점**: Target의 1/5~1/10 크기

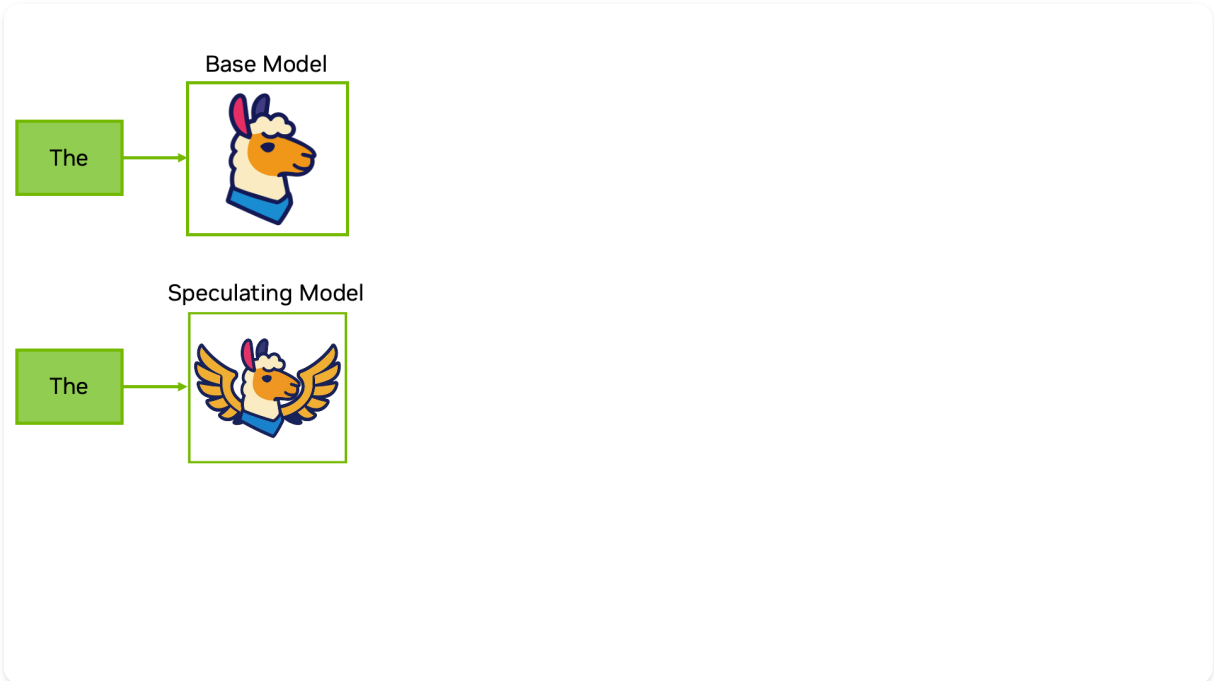
"빠른 모델"이 아닌 "균형 잡힌 모델"

Draft 모델 없는 방식

- **EAGLE**: Target 내부 feature에서 경량 head로 draft 생성
- **MTP**: 학습 시 multi-head를 함께 훈련

별도 모델 운영 불필요 → 최신 트렌드

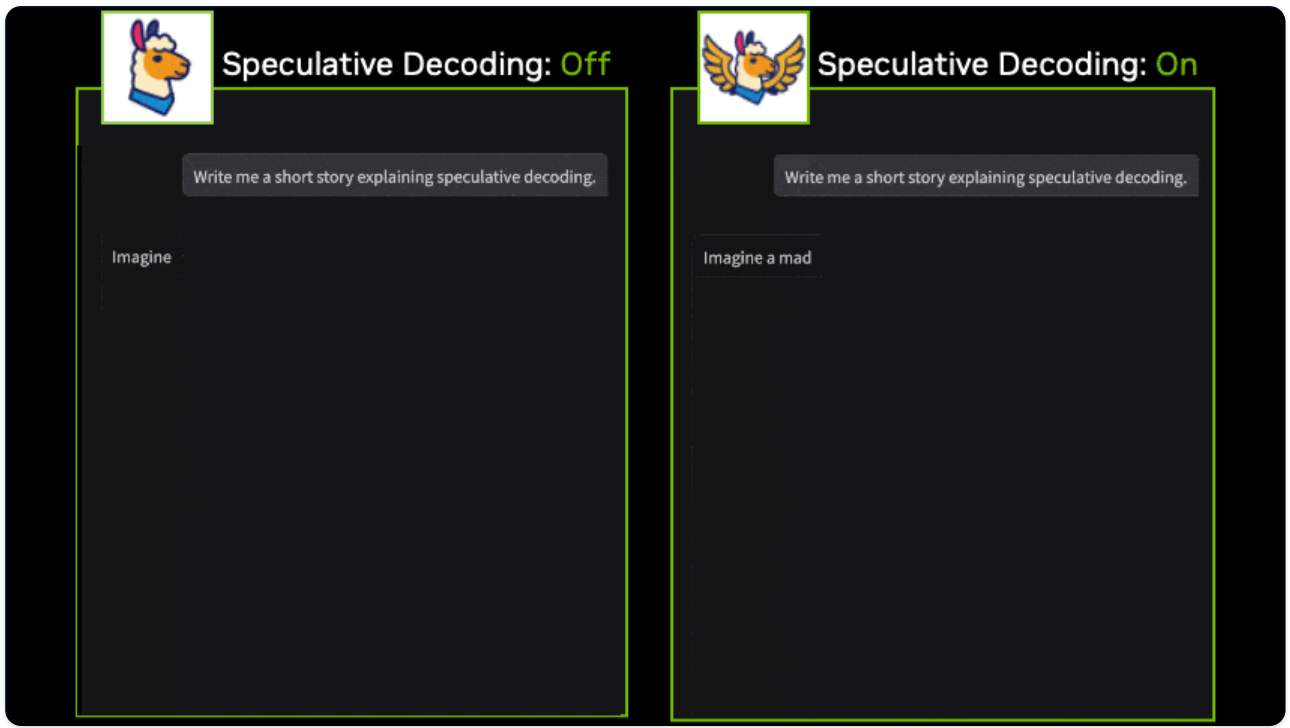
Latency 비교: AR vs Speculative Decoding



600ms → 250ms — 2.4× 속도 향상

사용자 경험의 변화

> *챗봇에서 Speculative Decoding ON/OFF 비교*



왼쪽: Speculative Decoding OFF — 토큰이 하나씩 느리게 나타남 / 오른쪽: ON — 여러 토큰이 한꺼번에 나타나 더 자연스러운 대화

기존 압축 기법과의 비교

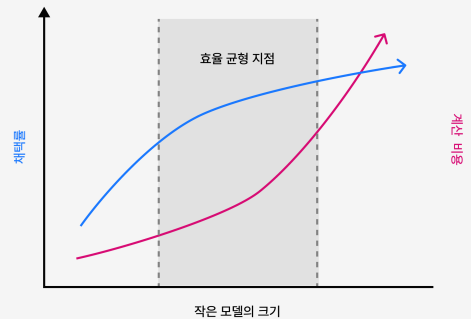
> *Speculative Decoding은 Pruning, Quantization과 달리* ***출력 품질을 저하시키지 않는다***

	Pruning / Quantization	Speculative Decoding
생성 분포	달라질 수 있음	**원래 분포 보존**
출력 품질	저하 가능성	변화 없음
서비스 도입	추가 검증 필요, 보수적	바로 적용 가능
가속 원리	모델 자체를 축소	생성 구조를 병렬화

Speculative Decoding은 **모델을 건드리지 않고** 생성 과정만 최적화하므로
기존 서비스에 **리스크 없이** 적용할 수 있다

성능 결정 요인 ①: 채택률과 Draft 모델 속도

> *Draft 모델이 **얼마나 빠르게, 얼마나 높은 채택률로** Target과 유사한 토큰을 생성하느냐가 관건*



x축: Draft 모델 크기 / 파랑: 채택률 / 핑크: 연산 비용 — 효율 균형 지점이 존재

모델이 너무 작으면

→ 채택률 하락 → 검증 단계에서 자주 거부

→ 오히려 전체 성능 저하

모델이 너무 크면

→ 채택률은 높지만 Draft 생성 자체가 느림

→ Target과의 속도 차이가 줄어 가속 효과 감소

최적 지점

→ 채택률과 연산 비용이 **균형을 이루는 구간**

→ 단순히 "빠른 모델"이 아니라 "균형 잡힌 모델"을 선택

성능 결정 요인 ②: 예측 길이(K) 설정

> *Draft 모델이 한 번에 몇 개의 토큰을 예측할지 — 고정값이 아닌 **동적 조정**이 필요*

시스템 상황	예측 길이 (K)	기대 효과	고려할점
CPU/GPU 여유 있음	K = 3~5	더 많은 토큰을 빠르게 생성 가능	채택률이 낮으면 비용이 크고 처리속도가 느려질 수 있음
리소스 혼잡, 요청 많음	K = 1~2	비용을 더 키우지 않고 안정적인 생성	속도 향상 효과가 제한될 수 있음

여유로운 환경 (K = 3~5)

- 더 많은 토큰을 빠르게 생성 가능
- 가속 효과 극대화
- 단, 채택률 낮으면 비용만 커짐

혼잡한 환경 (K = 1~2)

- 비용을 키우지 않고 안정적 생성
- 속도 향상 효과는 제한적
- 서비스 안정성 우선

지원 프레임워크

> *Speculative Decoding을 지원하는 주요 추론 프레임워크*

TensorRT-LLM

NVIDIA의 고성능 LLM 추론 엔진

- FP8/INT4 양자화 통합
- 최적화된 CUDA 커널

SGLang

빠른 LLM serving 프레임워크

- RadixAttention 최적화
- 유연한 프로그래밍 모델

vLLM

오픈소스 LLM serving

- PagedAttention 기반
- Draft model 방식 지원
- 넓은 모델 호환성

Speculative Decoding은 LLM 추론 가속의 **기본 전략**으로 자리잡고 있다

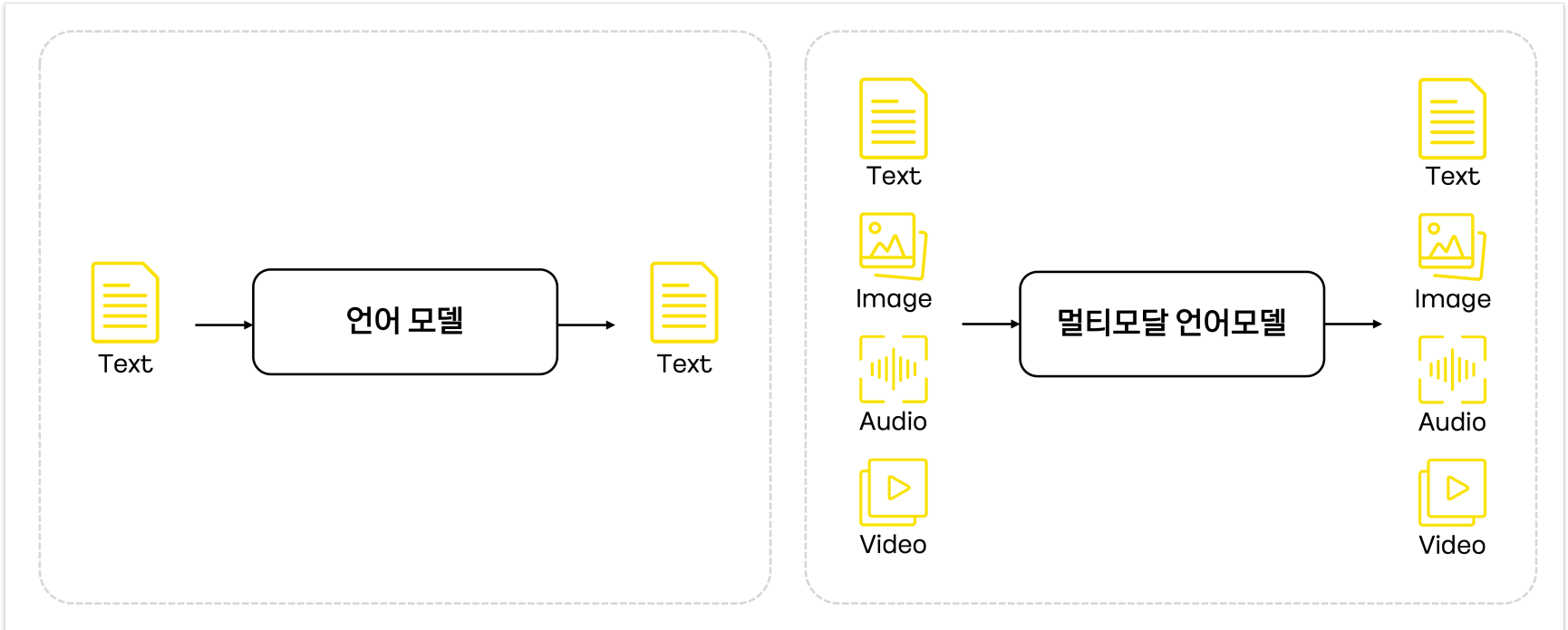
Vision LLM

오늘의 목표

구분	내용
필수 이해	`vision encoder + projector + LLM` 구조, image feature가 text embedding 일부를 교체
코드 실습	`get_image_embeddings`, `vision_proj`, `count_vision_proj`
선택 심화	grounding head 설계, multi-image edge case

decoder block은 그대로 두고, 입력 embedding이 어떻게 바뀌는지에 집중합니다.

멀티모달 언어 모델


Text
언어 모델
Text
Text
Image
Audio
Video
멀티모달 언어모델
Text
Image
Audio
Video

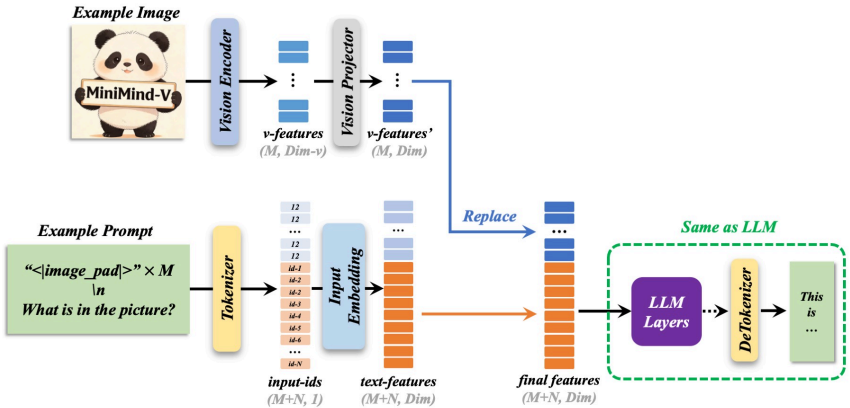
VLM이 다루는 Task

Task	설명
VQA (Visual Question Answering)	이미지에 대한 질문에 답변
Captions	이미지를 자연어로 설명
Doc Understanding	문서·차트·표 이미지 분석
Graph Analysis	그래프·다이어그램 해석

모든 task가 동일한 forward path를 공유합니다. 차이는 데이터 포맷과 출력 형식입니다.

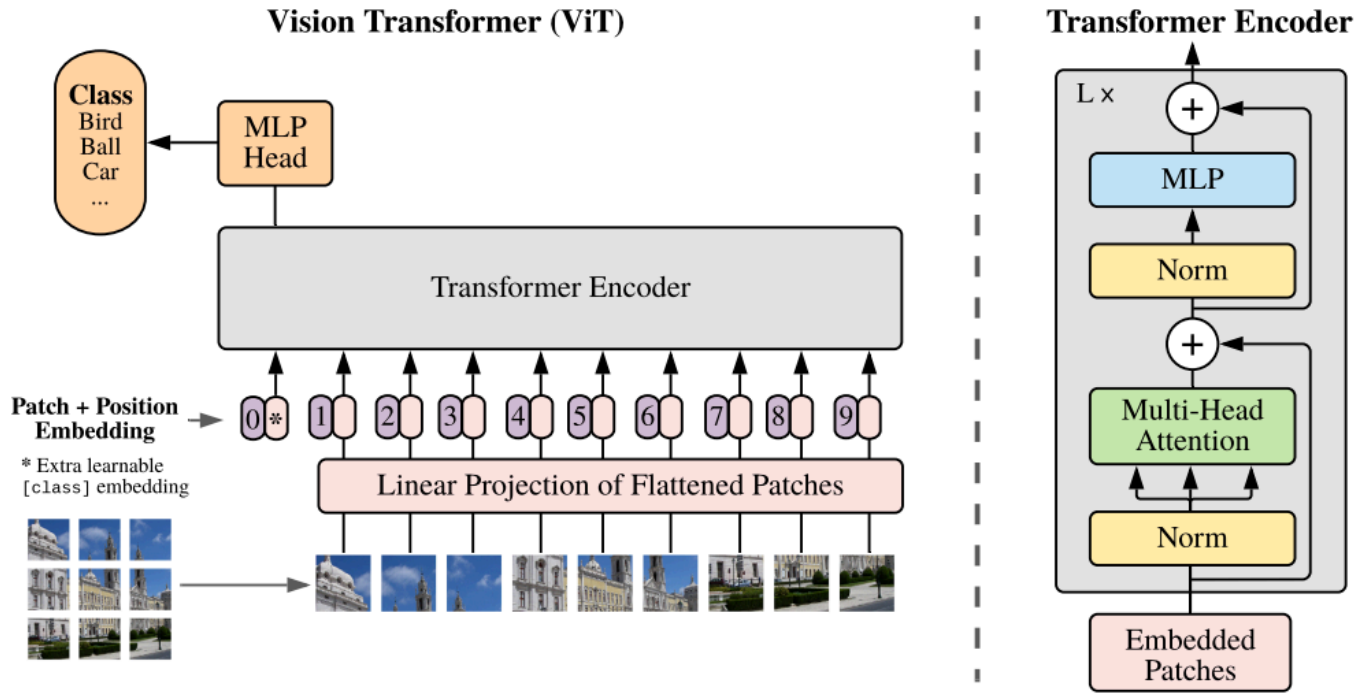
VLM 입력 구성과 Prompt Format

> *image slot 64개가 text embedding에 자리 잡고, projected visual feature로 교체*



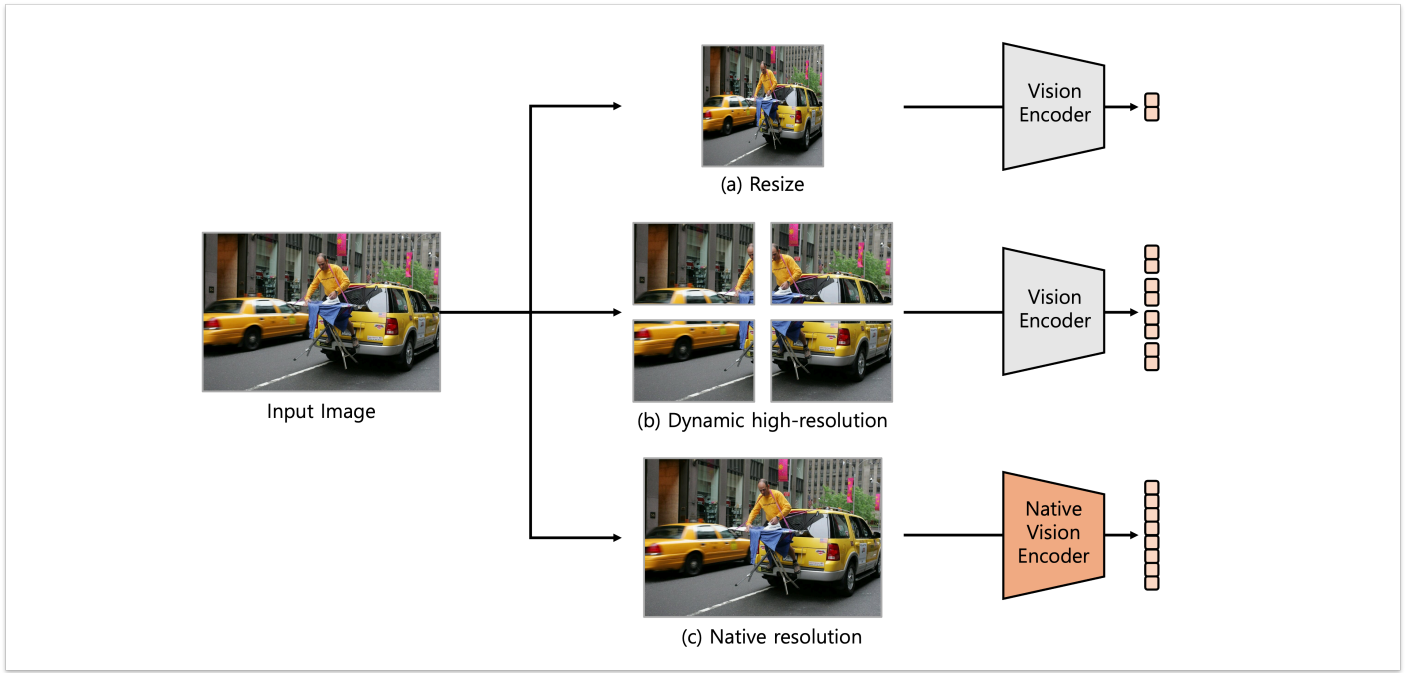

```
image_slot = "<|image_pad|>" * 64
prompt = f"{image_slot}\n이미지를 자세히 설명해 주세요."
```

Vision Transformer (ViT)



고해상도 이미지 처리

> *Resize, Dynamic tiling, Native resolution 세 가지 방식으로 고해상도 입력을 처리*



이미지: 64 토큰

> *MiniMind-V는 `siglip2-base-p32-256-ve` (ViT-B/32, 256×256 입력)를 사용합니다.*

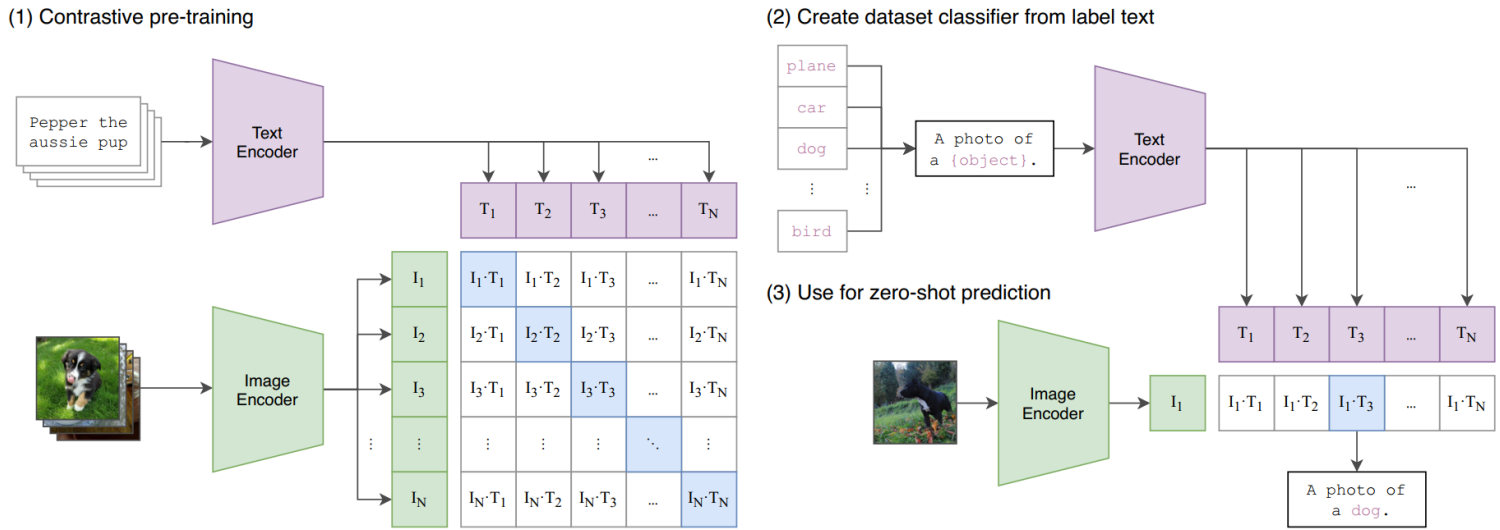
$$\text{patch tokens} = \left(\frac{256}{32}\right)^2 = 8^2 = 64$$

단계	shape	설명
입력 이미지	`[3, 256, 256]`	256×256 RGB
patch grid	`8 × 8`	32px 크기 patch 64개
encoder 출력	`[64, 768]`	patch별 768-dim feature
projector 입력	`[64, 768]`	LayerNorm 적용
projector 출력	`[64, hidden]`	LLM hidden size로 변환

- `768` : SigLIP2 ViT-B hidden dimension
- `[CLS]` 없이 **64개 patch feature 전체**를 LLM에 투영

CLIP은 이미지를 텍스트와 같은 공간에 놓는다

> *image-text pair의 similarity를 높이고, 틀린 pair는 낮추도록 학습*



- 이미지와 문장이 각각 encoder를 지남 → visual feature로 변환
- feature 간 similarity matrix → 이미지-텍스트 공간 정렬
- 정답 pair는 대각선 → "어울리는 문장"을 고르는 능력

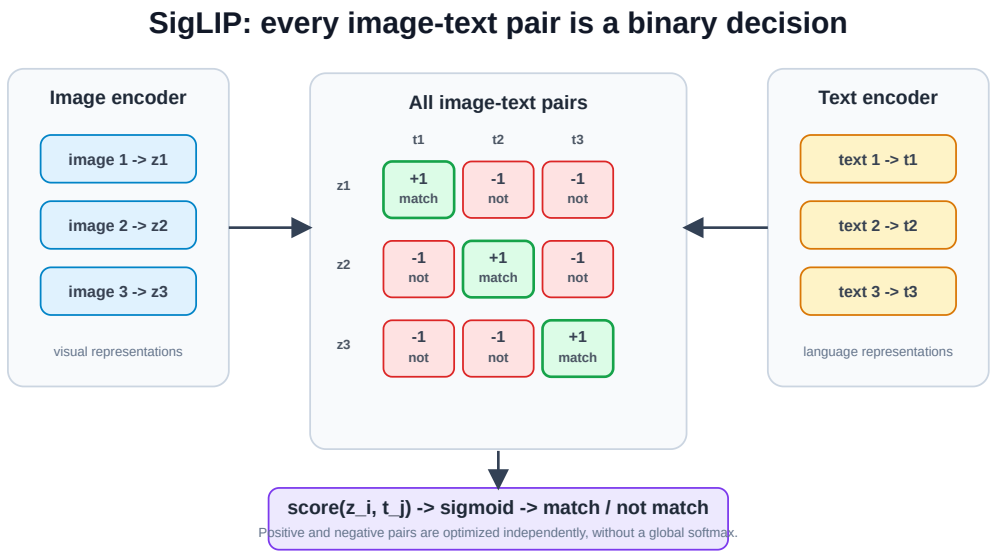
CLIP 구현: 핵심 코드

```
# I: [n, c, h, w]  T: [n, l]
# W_i, W_t: learned projection layers  t: temperature

# 1. Extract feature representations
I_f = image_encoder(I)   # [n, d_i]
T_f = text_encoder(T)    # [n, d_t]

# 2. Project to joint multimodal embedding space
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# 3. Scaled pairwise cosine similarities
logits = np.dot(I_e, T_e.T) * np.exp(t)
```

`logits[i, j]`는 i번째 이미지와 j번째 텍스트의 similarity입니다. 정답은 대각선(`i == j`)이고 나머지는 negative 입니다.

SigLIP은 positive/negative pair를 만든다

> *CLIP은 softmax로 묶고, SigLIP은 pair 각각을 독립 binary decision으로 학습*



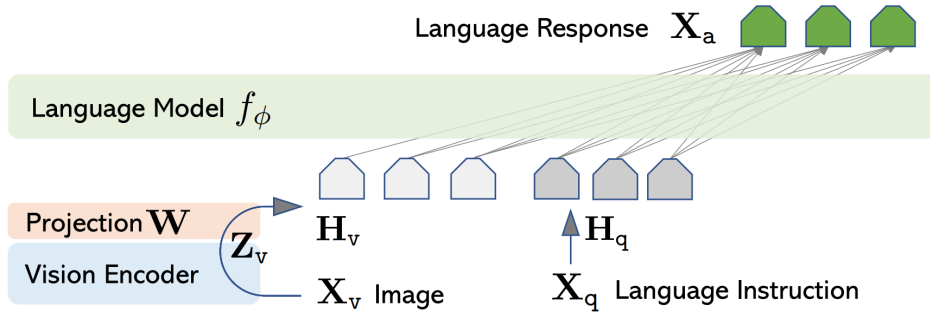
- 모든 image-text 조합을 pair로 생성
- 대각선 = positive, 나머지 = negative
- 각 pair에 sigmoid → "이 문장과 맞는 이미지인가"를 독립 판단

Pairwise Sigmoid가 CLIP 대비 좋은 점

> *SigLIP은 CLIP의 alignment 목표를 유지하면서 scaling을 편하게 만든 방식*

비교	CLIP softmax	SigLIP sigmoid
학습 단위	batch 전체를 분류 문제로 봄	pair 각각 독립
batch 의존성	큰 batch 필요	부담 적음
분산 학습	feature 동기화 필요	pair loss 분리 용이
직관	"정답 문장은?"	"이 pair가 맞나?"

Projector 진화



모델	Projector 방식	특징
LLaVA-1	Linear projection	단순 선형 변환, alignment 기본 수준
LLaVA-1.5	2-layer MLP	비선형 변환으로 정렬 품질 향상
MiniMind-V	LayerNorm + Linear → GELU → Linear	LLaVA-1.5 방식 채택, 32 patch, 64 tokens 그대로 유지

이미 64개로 compact하여 resampler 불필요. 대신 OCR 등 세부 인식에는 약함

MiniMind-V Projector 구현

> *feature 차원 정렬에 집중, token 수는 64개 유지*

```
class MMVisionProjector(nn.Module):
    def __init__(self, in_dim, out_dim, source_tokens=64, target_tokens=64):
        self.mlp = nn.Sequential(
            nn.LayerNorm(in_dim),
            nn.Linear(in_dim, out_dim),
            nn.GELU(),
            nn.Linear(out_dim, out_dim),
        )
```

- 입력: `image_hidden_size` (768)
- 출력: `hidden_size` (LLM hidden dim)
- token 수는 64개 그대로 유지

Vision Encoder Output Shape

> *핵심은 이미지 feature의 마지막 차원을 LLM hidden size로 바꾸는 것이다.*

단계	shape 예시	의미
image input	`[B, 3, H, W]`	processor가 만든 pixel tensor
vision encoder feature	`[B, 64, image_hidden]`	patch별 visual feature
projected visual token	`[B, 64, hidden]`	LLM embedding 공간
text embedding	`[B, T, hidden]`	token id의 embedding
combined sequence	`[B, T, hidden]`	image marker 구간 교체 후
logits	`[B, T, vocab]`	다음 token 분포

Attention과 학습: Image Token은 어떻게 처리되나?

> *self-attention은 image/text token을 구분하지 않고 같은 Q/K/V로 처리한다*

```
[image features][question tokens][answer tokens]
```

- causal mask만 적용: 미래 token은 못 보고, 이전 image + text token에 자유롭게 attend
- 학습 시 loss는 **assistant 답변 token에만** 계산

구간	attention 참여	loss 대상	label
image slot	O	X	`-100`
user question	O	X	`-100`
assistant answer	O	O	token id

Freeze 전략: `--freeze_llm` 옵션

> *중간 레이어를 freeze하여 언어 능력 보존, 첫/끝 레이어만 학습*

`--freeze_llm`	학습 대상	사용 단계
`2`	Projection만	Pretrain
`1`	Projection + LLM 첫/끝 레이어	SFT
`0`	전체 (Vision Encoder 제외)	Full fine-tuning

- **첫 레이어**: visual token 진입 지점
- **끝 레이어**: 답변 형식 결정
- **중간 레이어**: 기존 언어 지식 보존을 위해 freeze

학습 경로: SFT only vs Pretrain → SFT

> *Pretrain을 하지 않았을 때와 했을 때*

경로 A: **SFT only** — 빠르고 단순, alignment 불안정 위험

```
python train_sft_vlm.py --epochs 2 --from_weight llm
```

경로 B: **Pretrain → SFT** — 안정적 정렬, 품질 우위

```
python train_pretrain_vlm.py --epochs 2 --from_weight llm      # 1. alignment
python train_sft_vlm.py --epochs 2 --from_weight pretrain_vlm  # 2. instruction
```

단계	freeze	학습 대상
Pretrain	`--freeze_llm 2`	projector만
SFT	`--freeze_llm 1`	projector + LLM 첫/끝 레이어

- 장점: 안정적 정렬, 복잡한 VQA에서 품질 우위, 모듈러 개발
- 단점: pretrain 추가로 학습 시간 증가, checkpoint 관리 복잡

두 경로 비교와 선택

비교	SFT only	Pretrain → SFT
projector 시작	random	pretrain 정렬됨
비용	낮음	높음
안정성	중간	높음
추천	빠른 재현, 데모	벤치마크, 품질

65M급 소형 모델은 image gradient가 언어 능력을 쉽게 덮으므로, 2단계 분리가 특히 중요. 2단계는 "더 오래 학습"이 아니라 **학습 신호의 역할을 분리하는 설계**

VLM 학습 다이어그램

Stage 1: Training VL Adaptor

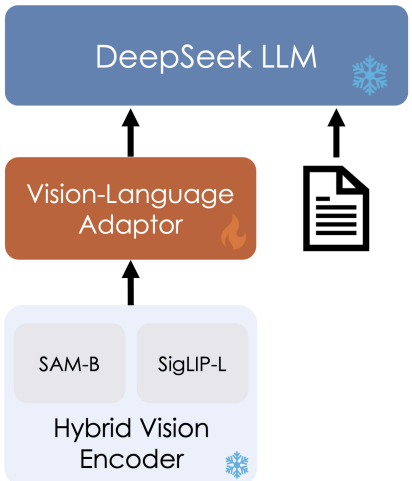
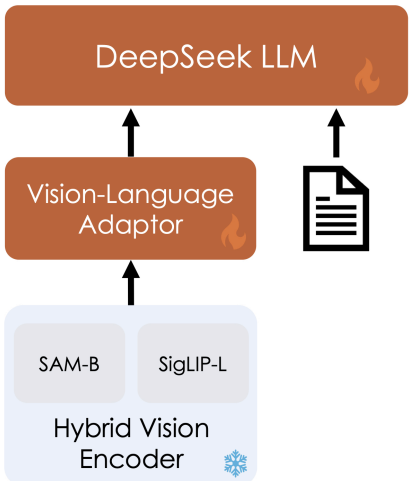


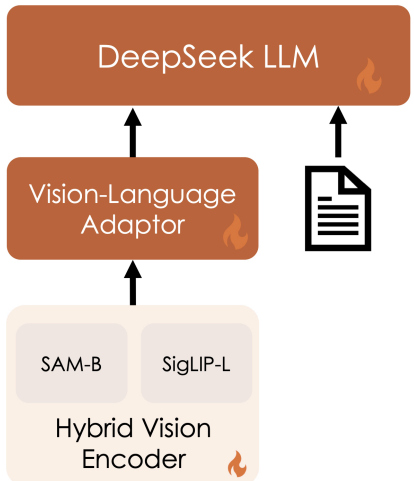
Image-Text Pairs

Stage 2: Joint VL Pre-training



Interleaved VL +
Pure Language Sequences

Stage 3: Supervised Finetuning



VL Chat Data +
Pure Language Chat Data

Failure Cases

- 객체 착각: 작은/가려진 물체 오인식
- OCR 실패: 이미지 속 글자 인식 불가
- hallucination: 이미지에 없는 내용 생성
- prompt mismatch: slot 수 ≠visual token 수

증상	의심 지점
큰 물체도 못 봄	vision encoder
답변 형식 어색	projector / SFT
없는 물체 언급	LLM hallucination
특정 prompt만 실패	image slot 구성

향후 개선 방향

방향	설명
Dynamic resolution	Tile-based encoding으로 고정 해상도 극복
강한 Vision Encoder	세밀한 image feature
Multi-image / Video	여러 이미지·video frame 확장

현재 한계: 64 patch token으로 세부 디테일 부족, 65M backbone hallucination, 고정 256×256 해상도

Self-attention: 하나의 순서열 안에서

Cross-attention: 두 흐름 사이에서

Q와 K·V는 서로 다른 흐름에서 나온다. 이 코드에서는 Q가 텍스트, K·V가 정지 영상이다.

```
import torch

def cross_attention(text, image, W_q, W_k, W_v):
    Q = text @ W_q
    K = image @ W_k
    V = image @ W_v
    scale = K.size(-1) ** 0.5
    scores = Q @ K.transpose(-2, -1) / scale
    weights = torch.softmax(scores, dim=-1)
    return weights @ V
```

1. 질문할 흐름과 참조할 흐름을 구분한다
2. 텍스트 위치마다 시각 정보의 관련성을 구한다
3. 선택한 시각 정보를 텍스트 위치에 모은다

Q의 길이와 K·V의 길이가 달라도 된다.

self-attention에서는 Q·K·V가 가ㅌ은 시퀀스에서 나오므로 길이가 가ㅌ다.

이미 학습된 두 모델을 연결하는 두 설계

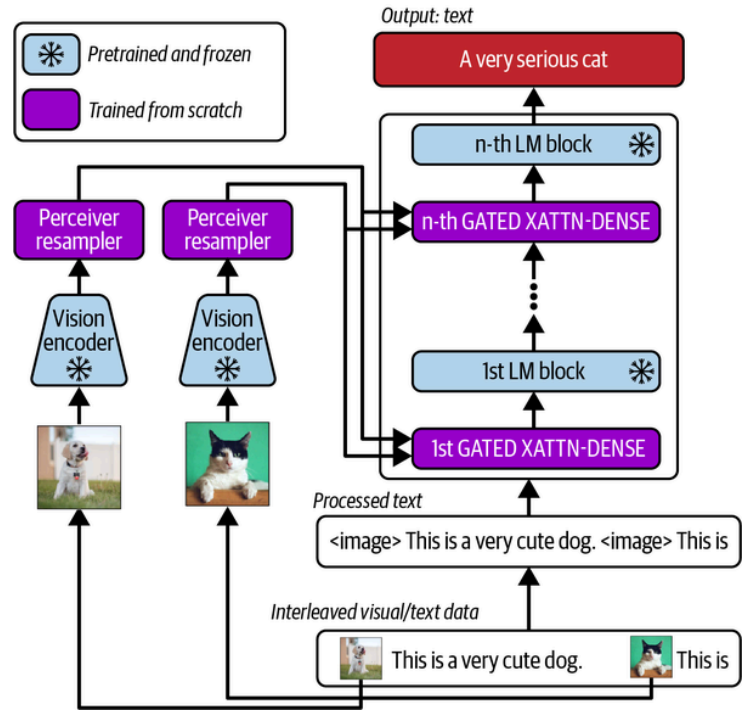
질문	cross-attention adapter 구조	통합 순서열 구조
시각 정보는 어디에 있는가	별도 시각 흐름	텍스트와 같은 입력 순서열
연결은 어디서 하는가	LLM 내부의 새 cross-attention 층	LLM 입력의 투영층
주로 쓰는 attention	새 cross-attention	기존 self-attention
대표 예시	Flamingo	SmolVLM

공통 출발점: **사전학습(pretraining)한 시각 부호기 + LLM**

Flamingo: 고정한 모델 사이에 새 층을 넣는다

- 왼쪽, 아래 → 위: 정지 영상 → 시각 부호기 → resampler
- 오른쪽, 아래 → 위: 텍스트 → gate가 있는 cross-attention → LLM
- 파랑: 사전학습 후 **고정**
- 보라: 새로 **학습**하는 연결부

LLM의 원래 언어 능력을 보존하면서 시각 정보를 주입한다.



출처: 『Vision Language Models』 그림 6-1, Alayrac 외

Perceiver resampler: 많은 특징 → 64개 요약

```
queries = learned_queries.expand(
    vision_features.size(0), -1, -1)
summary, _ = cross_attn(
    query=queries, key=vision_features,
    value=vision_features)
visual_tokens = projector(summary)
```

1. 학습되는 **64개 query**를 배치에 맞춰 복제한다
2. 전체 시각 특징을 K·V로 참조해 요약한다
3. LLM 차원으로 투영한다

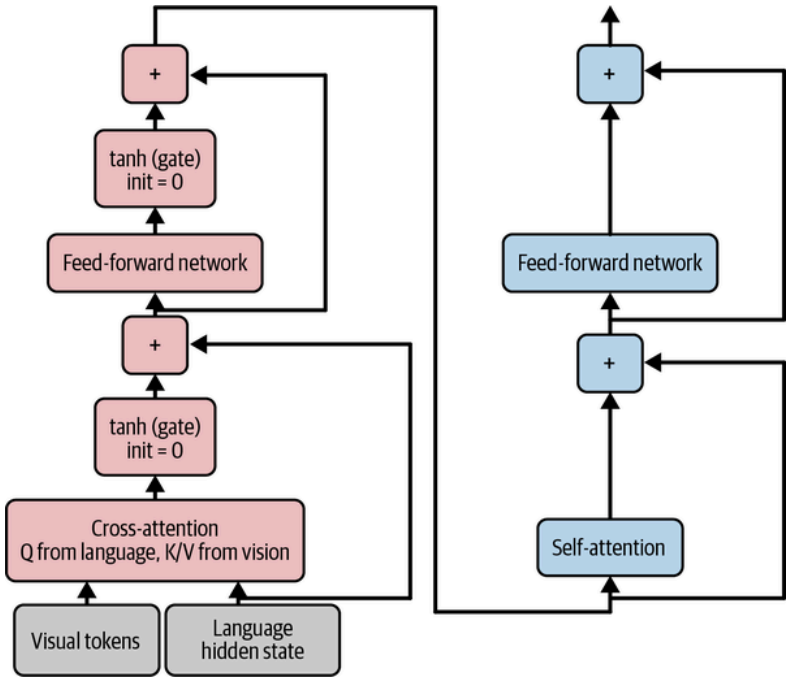
예시: `[B, 256, 768]` → `[B, 64, 576]`

- 입력 해상도가 달라도 요약 개수는 고정

Gate가 필요한 이유: 처음에는 원래 LLM 그대로

- 왼쪽 아래: 언어 은닉 상태가 Q, 시각 token이 K·V
- 분홍: 학습하는 cross-attention과 feed-forward
- 파랑: 고정한 기존 self-attention과 feed-forward
- 우회 화살표: 잔차 연결(residual connection)

두 gate를 **0으로 초기화**하면 새 경로의 기여도 0에서 시작한다.



출처: 『Vision Language Models』 그림 6-2

코드: 잔차 경로에 gate를 곱한다

```
attn_gate = nn.Parameter(torch.zeros(1))
ffn_gate = nn.Parameter(torch.zeros(1))
attn_out, _ = cross_attn(
    query=attn_norm(text_hidden),
    key=image_features, value=image_features)
text_hidden = text_hidden + attn_gate.tanh() * attn_out
ffn_out = ffn(ffn_norm(text_hidden))
text_hidden = text_hidden + ffn_gate.tanh() * ffn_out
```

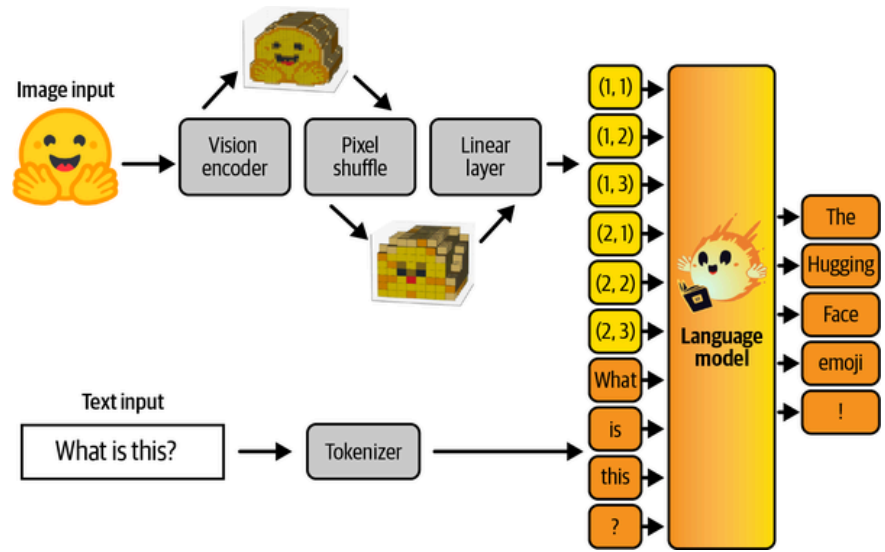
1. Attention·feed-forward gate를 각각 영 초기화한다
2. 시각 정보를 가져와 gate를 곱해 더한다
3. Feed-forward 결과도 gate로 조절해 더한다

초기에는 $\tanh(0) = 0 \rightarrow$ `text_hidden` 보존

SmolVLM: 같은 순서열에 시각 token을 넣는다

- 왼쪽 → 오른쪽: 정지 영상 → 부호기 → pixel shuffle → 선형층
- 아래: 텍스트 → tokenizer
- 노란 순서열: 시각·텍스트 embedding을 함께 입력
- 기존 LLM self-attention이 두 양식의 관계를 학습

연결부는 간단한 투영층, 융합은 LLM이 수행한다.



출처: 『Vision Language Models』 그림 6-3

코드: 투영층과 기존 LLM 호출로 연결한다

3장 [12,13] 복습: 정지 영상 자리를 시각 벡터로 교체한다.

```
image_embd = vision_encoder(pixel_values).last_hidden_s
image_tokens = modality_projector(image_embd)
logits = llm(inputs_embeds=token_embd).logits
```

1. 시각 부호기 출력을 LLM 차원으로 투영한다
2. 자리 교체한 결합 순서열를 기존 LLM에 입력한다

기존 LLM 입력에서 결합하므로 내부에 새 cross-attention 층이 없다.

점검: 자리 수·정지 영상 순서·자료형을 맞춘다.

어떤 구조가 좋은가? Backbone 학습 조건부터

> *Idefics 2 비교: **Mistral-7B** + **SigLIP***

조건	Cross-attention	Self-attention	해석
Backbone 고정	**66.7**	60.3	전용 융합 층의 이점
Backbone을 LoRA로 미세조정(fine-tuning)	67.3	**69.5**	기존 모델이 적응하면 순위 변화

점수: **VQAv2·TextVQA·OKVQA·COCO**의 평균

구조 이름만으로 승자를 정할 수 없다.

같은 Backbone이어도 구현 비용은 다르다

> *작은 실험: **SigLIP2 base + SmolLM2-135M-Instruct***

항목	통합 순서열	Cross-attention
학습 매개변수	227,888,256	262,192,336
새 연결부	선형 투영층	Resampler + gated 층
LLM 실행	`inputs_embeds` 로 호출	복호기 층을 직접 순회

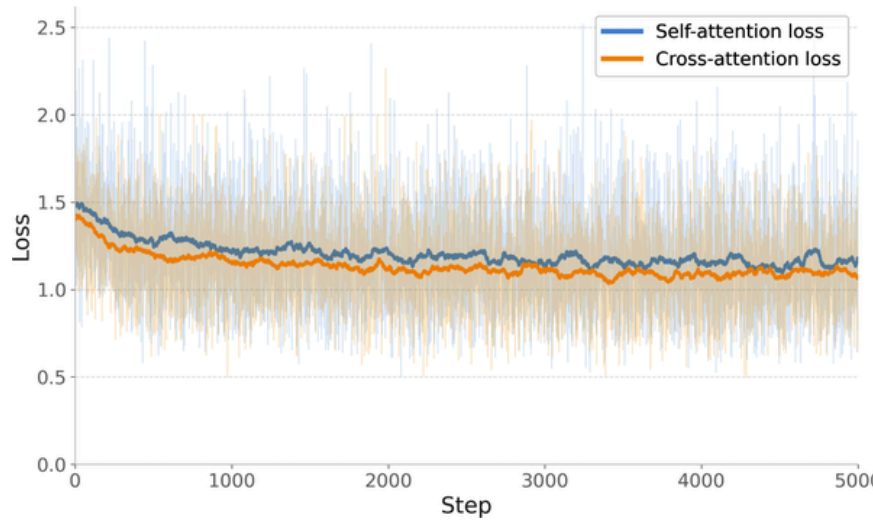
- Cross-attention: 약 **3,400만 개, 15%** 더 많은 학습 매개변수
- 수동 순회에서는 causal mask·위치 표현·KV cache도 관리
- Backbone 미세조정에는 메모리와 망각 위험도 고려

첫 실험: Cross-attention 손실이 조금 낮다

- 같은 FineVision 부분집합, **5,000단계**
- 두 구조 모두 **전체 매개변수** 학습
- 주황 곡선이 파랑보다 약간 낮음

그런데 **한 단계에서 본 정답의 양도 같은가?**

Packing의 정지 영상 자리	개수
통합 순서열	256
첫 cross-attention 실험	1



출처: 『Vision Language Models』 그림 6-4

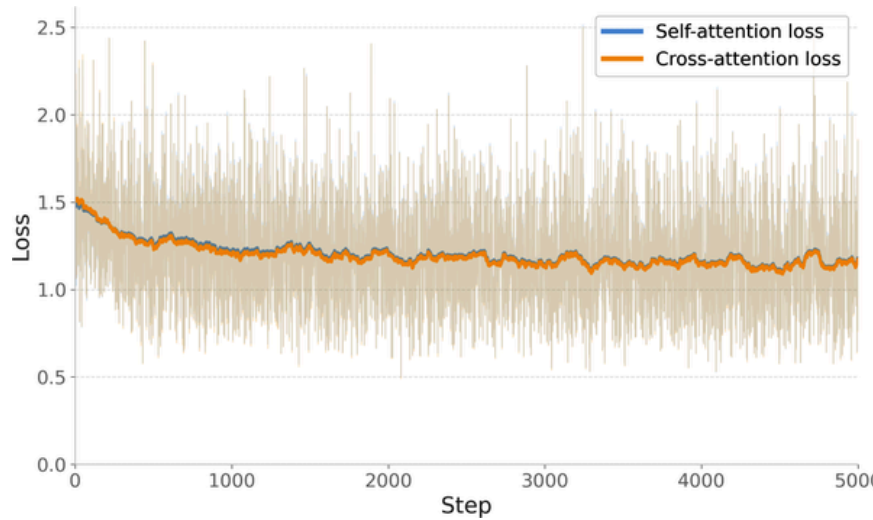
- 파랑: self-attention
- 주황: cross-attention
- 같은 순서열 예산에서 텍스트 수가 달랐다

Token 예산을 맞추면 차이가 거의 사라진다

- Cross-attention 쪽도 정지 영상 자리 **256개** 유지
- 그림에서 파랑과 주황 곡선이 거의 겹침
- 이 작은 규모·학습 조건에서는 연결 구조의 차이가 작음

훈련 손실만으로 시각 이해 능력 전체를 판정하지 않는다.

- 텍스트 대부분은 정지 영상 없이도 예측 가능
- 데이터·학습률·배치·학습량도 함께 살펴야 함

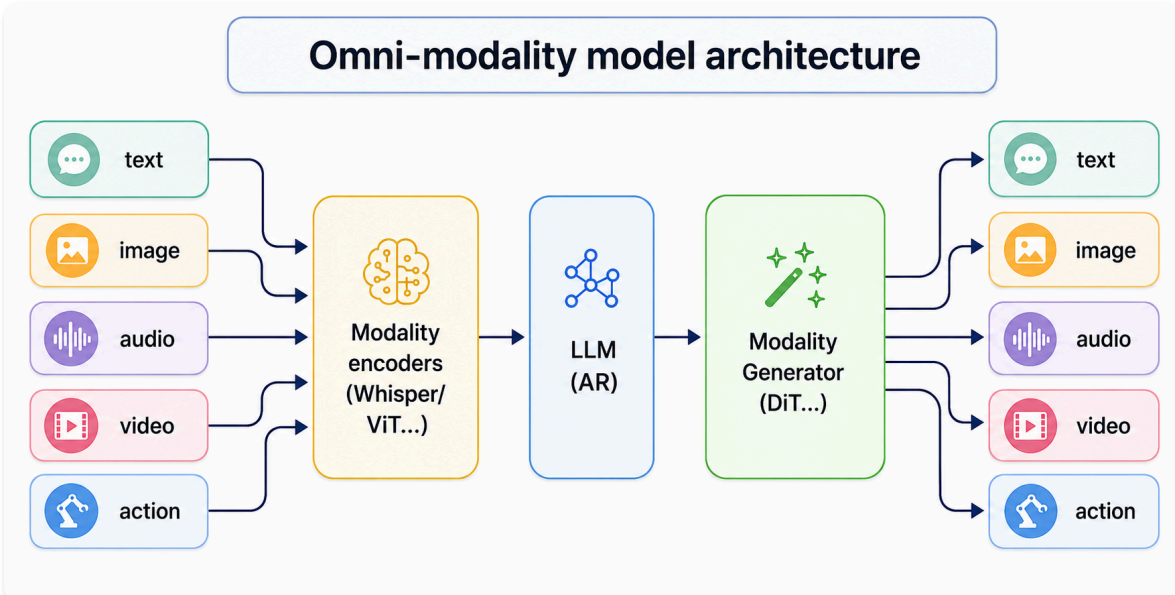


출처: 『Vision Language Models』 그림 6-5

- 정지 영상 자리 예산을 맞추자 학습 손실 차이가 거의 없어짐

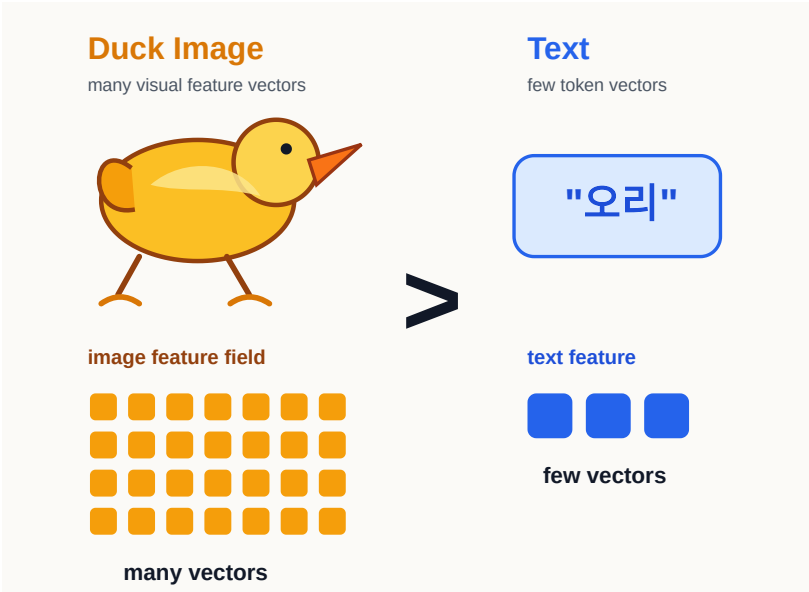
Omni Model

멀티 모달의 Challenges



문제	왜 어려운가
grounding	LLM은 텍스트만 학습 → 이미지·오디오는 낯선 신호
표현 크기 불균형	image/audio feature가 text signal을 압도
시간축	speech/video는 토큰 순서 외에 실시간 정렬 필요
출력 modality	text token + audio code 동시 생성 필요

표현 크기 불균형



오리 이미지 → 수백 개 patch feature / `오리` 텍스트 → 1~2개 token feature

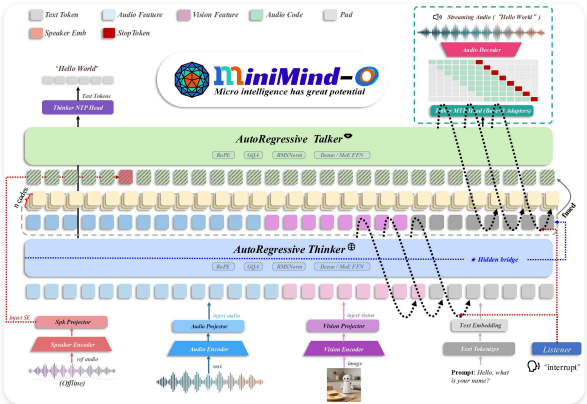
attention 공간에 넣기 전에 projector, gate, loss balance로 스케일을 맞춰야 한다

MiniMind-O 전체 I/O 흐름

포인트	코드 대응
text, image, audio 모두 입력 가능	tokenizer, vision encoder, audio processor
입력 feature → hidden/token 공간으로 정렬	projector, placeholder injection
text + speech 동시 출력	Thinker text logits, Talker audio logits
모델 밖 realtime 제어 필요	VAD, ASR, streaming loop

MiniMind-O Architecture

> *Thinker = 텍스트 이해·생성 / Talker = Thinker hidden을 조건으로 speech code 생성*



포인트	코드 대응
모든 입력 → Thinker로 수렴	`MiniMindOmni.forward`
projector가 hidden size 정렬	`MMAudioProjector` , `MMVisionProjector`
Thinker → text logits + bridge hidden	`self.thinker` , `bridge_layer`
Talker → audio code logits	`TalkerModule` , `audio_logits`
codec code → waveform 복원	Mimi decoder

Thinker Stream 입력 Layout

> *placeholder = feature injection 위치를 표시하는 token 자리*

포인트	의미
user / assistant 대화 형식	Thinker의 기본 입력 = 대화 token sequence
`<|audio_pad|>` 구간	audio feature injection 위치
`<|image_pad|>` 구간	image feature injection 위치
답변 token 구간	text CE loss 적용 대상

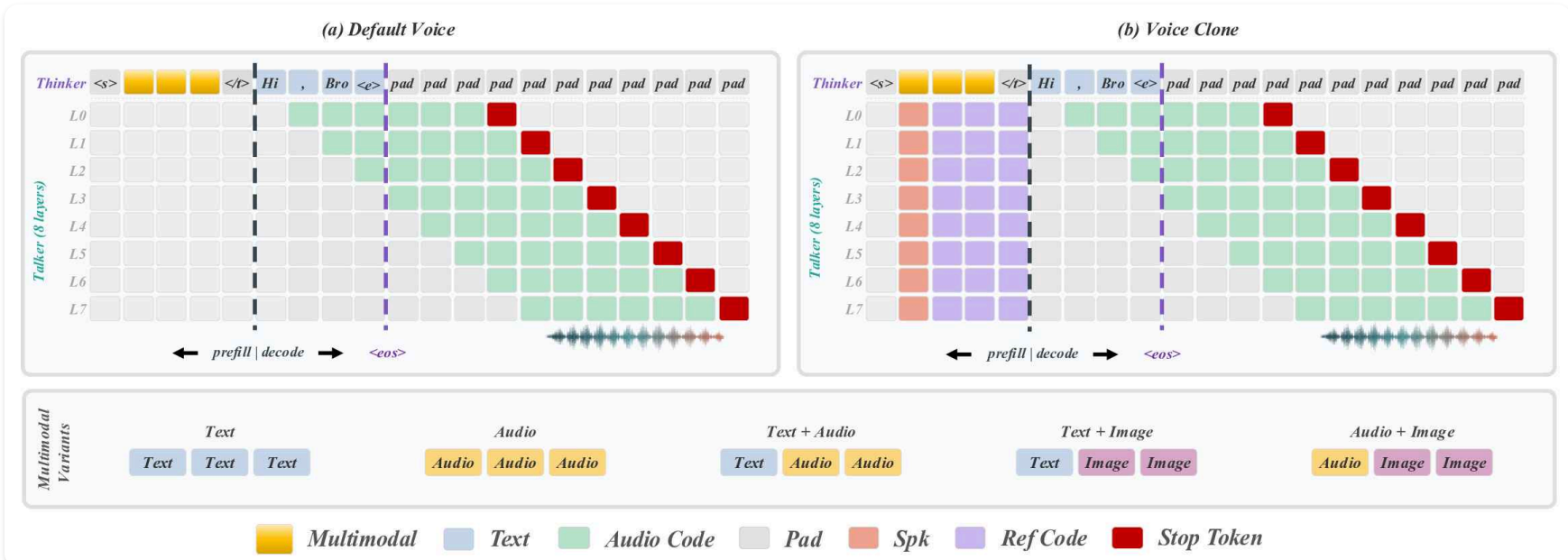
Speech 용어 먼저 분리하기

> *audio 관련 용어는 입력(이해)과 출력(생성) 경로로 나뉜다*

용어	경로	의미
waveform	입력/출력 바깥	실제 소리 sample
`audio_inputs`	speech understanding	**fbank/SenseVoice** 입력 feature
projected audio feature	speech understanding	Thinker hidden에 주입되는 vector
`audio_ids`	speech generation	이전 audio codec code stream
`audio_logits`	speech generation	다음 codec codebook id 점수
codec code	speech generation	**Mimi decoder**가 waveform으로 복원할 discrete id

들어오는 speech feature와 **나가는 speech code**는 같은 audio라는 이름을 쓰지만 다른 경로입니다.

Sequence Format



그림에서 볼 것	의미
text token row	Thinker가 처리하는 text stream
8 audio-code rows	Talker가 예측할 Mimi codebook streams
conditioning 구간	speech/image/voice 조건이며 reconstruction target이 아님
reply 이후 구간	text loss와 audio code loss가 실제로 걸리는 위치

Sequence Format 읽기: 코드북별 한 스텝 지연

> *텍스트와 음성을 함께 생성하되, **L0~L7의 시작을 한 칸씩 늦춘다** — 8개 행은 Transformer 층이 아닌 코드북*

생성 스텝 →	0	1	2	3	4	5	6
Thinker	Hi	,	Bro	`<e>`	pad	pad	pad
코드북 L0	·	**A0**	B0	C0	STOP	·	·
코드북 L1	·	·	**A1**	B1	C1	STOP	·
코드북 L2	·	·	·	**A2**	B2	C2	STOP

3개 코드북·3개 음성 프레임으로 단순화한 예시 · 실제는 L0~L7의 8개 스트림

같은 프레임은 대각선으로 완성

- 프레임 A = **[A0, A1, A2]**; 같은 열을 묶지 않음
- 실제는 첫 프레임의 코드 8개가 모인 뒤 음성 복원
- 텍스트 종료 후에도 음성 생성 지속 → 종료도 계단형

기본 목소리 / 참조 목소리

- 검은 점선: 입력 읽기(prefill) → 응답 생성(decode)
- 참조 조건: `Spk` 화자 벡터 + Mimi `RefCode`
- 조건을 먼저 읽고 **새 응답 코드**를 같은 지연 방식으로 예측

현재 열의 L0~L7 → TalkerEmbedding + Thinker 표현 → Talker → 8개 코드북 점수 → 다음 열
8개 점수는 함께 계산하지만, 같은 음성 프레임은 여러 생성 스텝에 걸쳐 완성된다.

왜 코드북을 한 칸씩 늦출까?

> *TalkerHead는 매 스텝 **8개 코드북의 점수를 함께 계산**한다. 지연 배치는 **코드북 간 의존성**을 활용한다.*

같은 프레임을 동시에 예측

코드북	스텝 1	스텝 2	스텝 3
L0	**A0**	B0	C0
L1	**A1**	B1	C1
L2	**A2**	B2	C2

이전 문맥은 공유하지만, A1은 **이번에 선택될 A0**를 직접 참고할 수 없다.

코드북별 한 스텝 지연 — 현재 방식

코드북	스텝 1	스텝 2	스텝 3	스텝 4
L0	**A0**	B0	C0	D0
L1	·	**A1**	B1	C1
L2	·	·	**A2**	B2

A0 선택 → 이를 입력으로 A1 예측 → **A0·A1을 문맥으로 A2 예측**

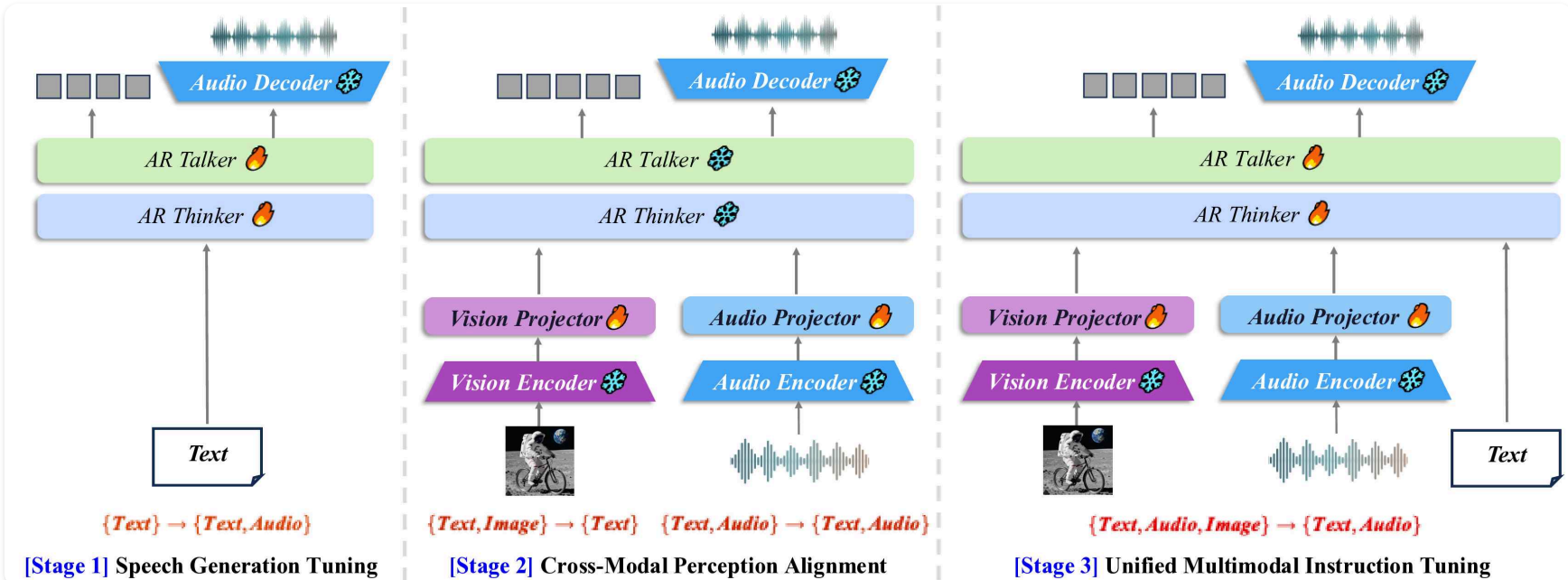
여러 프레임을 겹쳐 처리: 스텝 3에서 C0(시작) + B1(진행) + A2(완성)를 동시에 예측

방식	같은 프레임에서 앞서 선택한 코드 참고	프레임 완성 속도
같은 프레임의 8개 코드 동시 예측	직접 참고 불가	처음부터 스텝당 1프레임
코드북별 지연 예측	이전 스텝의 선택 결과 참고 가능	초기 지연 후 스텝당 1프레임

코드북 간 의존성을 활용하면서, 초기 지연 이후에는 매 스텝 한 프레임씩 완성한다.

학습 순서

> *SFT 데이터 흐름을 따라 능력을 순서대로 추가*



단계	목적	주로 맞추는 경로
`sft_t2a`	text → speech 정렬	Thinker → Talker
`sft_a2a`	speech → reply 정렬	audio projector → Thinker

LLM / VLM / Omni 비교

모델별 입출력 구조

모델	입력 확장	backbone	출력
LLM	text token	MiniMind decoder	text logits
VLM	image feature injection	MiniMind decoder	text logits
Omni	audio/image feature + audio codes	Thinker + Talker	text logits + audio logits

내부 처리 관점 비교

관점	LLM	VLM	Omni
내부 sequence	text hidden	text + image hidden	text + audio/image hidden
예측	vocabulary token	vocabulary token	vocabulary token + codec code
loss/output	CE over text	CE over text	text CE + audio code loss

- 공통: 모든 입력을 hidden space로 정렬
- 차이: modality가 늘면 encoder, projector, head, loss도 증가

요약

1. Omni는 raw audio/image를 그대로 LLM에 넣지 않고, **encoder/projector**를 거쳐 hidden sequence에 주입한다.
2. **Thinker**는 text 의미 이해와 text logits를 담당한다.
3. **Talker**는 Thinker hidden state를 조건으로 audio code logits를 만든다.
4. VAD, ASR, CAM++, codec은 모델 내부 layer가 아니라 **시스템 외부 컴포넌트**로 구분해서 읽는다.
5. 코드 독해는 `forward` shape 분기에서 시작하면 길을 잃지 않는다.

Linear Attention

오늘의 목표

- **왜 O(T) 복잡도가 실제로는 $O(T^2)$보다 느린가?** 직관적 이유 이해
- **Linear Attention의 핵심**: 유사도 분해와 상태 행렬
- **연산 순서 재배열**: $S \times S$ 병목을 $d \times v$ 상태로 축소
- **Chunk-wise 병렬화**: 재귀를 행렬 곱셈으로 변환
- **MiniMind-Linear 구현**: Full Attention → Gated DeltaNet 4가지 변경

Full Attention의 계산 구조

> *입력길이 T가 길어질수록 계산량이 증가함*

- 문맥 백터는 모든 토큰의 가중 합계

$$o_t = \sum_{i=1}^{T} \text{softmax}(q_t^T k_i) \cdot v_i$$

- 행렬 형태로 쓰면

$$O = \text{softmax}(QK^T)V$$

특성	Full Attention
복잡도	$O(T^2 d)$
병렬성	T^2개 원소 전부 동시 계산
GPU 활용률	극대화 (행렬 연산 1회)

Linear Attention의 핵심 아이디어

> *$\phi(Q_i)^T$는 j에 의존하지 않으므로 합 밖으로*

Attention

- 유사도(sim)로 value를 가중 평균하는 연산

$$V_i' = \frac{\sum_{j=1}^{N} \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^{N} \text{sim}(Q_i, K_j)}$$

softmax attention

- 유사도를 확률분포로 정규화

$$\text{sim}(Q_i, K_j) = \exp\left(\frac{Q_i K_j^T}{\sqrt{d_k}}\right)$$

Linear Attention

- 유사도를 feature map(ϕ) 내적으로 분해

$$\text{sim}(Q_i, K_j) = \phi(Q_i)^T \phi(K_j)$$

Feature Map ϕ의 선택: ELU

> *softmax의 exp를 근사하는 간단한 함수*

softmax attention의 유사도는 $\exp(Q_i K_j^T)$인데, 이를 $\phi(Q)^T\phi(K)$로 분해하려면 적절한 ϕ가 필요하다

ELU + 1 Feature Map

$$\phi(x) = \text{ELU}(x) + 1 = \begin{cases} x + 1 & x > 0 \\ e^x & x \leq 0 \end{cases}$$

- $x > 0$: 선형 → 상대적 크기 순서 보존
- $x \leq 0$: 지수 함수 → 연속성 유지, 기울기 소실 방지
- **+1**: 모든 출력이 양수 → 유사도가 음수가 되지 않음

왜 ELU인가?

후보	장단점
$\text{ReLU}(x)$	간단하지만 음수 영역 정보 소실
$\text{ELU}(x) + 1$	exp 근사 + 항상 양수 + 연속
Random Fourier	softmax 정확 근사, 고차원 필요

핵심은 ϕ의 정확도보다 **분해 가능성** — $\phi(Q)^T\phi(K)$로 쪼갤 수 있으면 결합법칙 적용 가능

상태 행렬로 바뀌는 Attention

> *유사도 분해를 대입하면 계산 순서를 바꿀 수 있다. 새 토큰의 Q가 들어때마다 고정된 S,Z와 계산*

유사도

$$\frac{\phi(Q_i)^T \sum_j \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_j \phi(K_j)}$$

상태를 두 개의 누적량으로 정의

$$S = \sum_j \phi(K_j) V_j^T, \qquad Z = \sum_j \phi(K_j)$$

$$V_i' = \frac{\phi(Q_i)^T S}{\phi(Q_i)^T Z}$$

Causal / recurrent 형태

$$S_t = S_{t-1} + \phi(k_t) v_t^T, \quad Z_t = Z_{t-1} + \phi(k_t)$$

- 전체 $T \times T$ attention matrix를 만들지 않음
- 추론: KV Cache 대신 고정 크기 상태 S_t, Z_t 유지
- 총 복잡도: sequence length에 대해 $O(T)$

연산 순서 재배열로 $O(S)$ 달성

> *여기서 $S(=T)$는 **시퀀스 길이**, d는 feature map 차원, v는 value 차원*

Full Attention 순서

$$\underbrace{QK^T}_{S\times S}V$$

- 먼저 $S \times S$ attention score 생성
- 중간 행렬이 sequence length의 제곱으로 증가
- 복잡도: $O(S^2 d)$

Linear Attention 순서

$$\phi(Q)\underbrace{\left(\phi(K)^T V\right)}_{d\times v}$$

- $\phi(K)^T V : (d, S) \times (S, v) \to (d, v)$
- sequence 축 S가 먼저 합쳐져 사라짐
- 복잡도: $O(Sdv)$, d, v 고정이면 $O(S)$

핵심은 QK^T**를 먼저 만들지 않고,** $K^T V$ **계열의 작은 상태를 먼저 만든다**는 점이다.

결합법칙으로 복잡도 줄이기

> *괄호 순서만 바꿔도 $O(T^2) \to O(T)$*

$(QK^T)V$ — 원래 순서

$$\underbrace{Q}_{T\times d} \cdot \underbrace{K^T}_{d\times T} = \underbrace{T \times T}_{\text{거대 행렬}}$$

복잡도: $O(T^2 d)$

$Q(K^T V)$ — 재배열 순서

$$\underbrace{K^T}_{d\times T} \cdot \underbrace{V}_{T\times d} = \underbrace{d \times d}_{\text{작은 행렬}}$$

복잡도: $O(Td^2)$

$T = 4096,\ d = 128$ 일 때 → $T^2 d = 2.1\text{B}$ vs $Td^2 = 67\text{M}$ — **약 32배 차이**

긴 시퀀스에서의 구조적 이점

> *현대 언어 모델에서 **긴 시퀀스 처리 능력은 곧 모델 파워**와 연결된다*

Full Attention

- S가 커질수록 $S \times S$ score matrix 생성
- 메모리와 연산량이 제곱으로 증가
- 긴 컨텍스트에서 attention matrix 자체가 병목

$$\text{memory} \propto S^2$$

Linear Attention

- 중간 상태는 $d \times v$로 고정
- sequence 축 S는 누적 계산에서 사라짐
- 긴 입력에서도 상태 크기는 feature/value 차원에 의해 결정

$$\text{state size} \propto d \times v$$

구분	Full Attention	Linear Attention
중간 행렬	$S \times S$ (가변)	$d \times v$ (고정)
복잡도	$O(S^2)$	$O(S)$
S 증가 시	메모리 폭증	상태 크기 유지

이 장점은 **긴 컨텍스트 추론의 메모리 구조**에 대한 장점이며, 뒤에서 볼 recall 정확도 문제와는 별개다.

Gated DeltaNet으로 확장

> *MiniMind-Linear는 단순 누적에 **게이트와 쓰기 강도**를 추가*

$$\boxed{S_t = g_t \cdot S_{t-1} + \beta_t \cdot k_t v_t^T, \quad o_t = q_t^T \cdot S_t}$$

파라미터	역할
g_t	이전 상태 보존 비율, 얼마나 잊어야 하는가?
β_t	현재 토큰 쓰기 강도, 새로운 속성을 얼마나 반영하는가?

Gated DeltaNet은 Qwen3-Next / Qwen3.5 계열에서 채택된 Linear Attention 변형이다.

왜 O(T)이 $O(T^2)$보다 느린가?

> *결합법칙을 이용해 순서를 바꾸고 싶은데...*

$$O = \underbrace{\text{softmax}(QK^T)}_{\text{먼저 패턴을 찾아야 함}} V$$

softmax가 걸림돌: 전체 행을 봐야 귀납화 가능 → $T \times T$ 점수 행렬을 먼저 만들어야 함 → $Q(K^T V)$로 재배열 불가

순진한 O(T) 구현의 문제점

각 토큰마다 PyTorch kernel을 하나씩 launch → **launch/동기화 오버헤드**가 지배적

→ 실제로 Full Attention보다 수십 배 느릴 수 있음

Chunk-wise 병렬화

> ***핵심 아이디어**: 시퀀스를 청크로 잘라 청크 내부는 행렬 곱셈으로 처리*

```
token:  [1 2 3 4] [5 6 7 8] [9 10 11 12]
chunk:    chunk 0    chunk 1    chunk 2
```

chunk 2 (token 9~12) 내부 상태 전개 — S_8은 이전 청크에서 전달

$$S_9 = S_8 + k_9 v_9^T$$

$$S_{10} = S_8 + k_9 v_9^T + k_{10} v_{10}^T$$

$$S_{11} = S_8 + k_9 v_9^T + k_{10} v_{10}^T + k_{11} v_{11}^T$$

$o_{11} = q_{11}^T S_{11}$ 을 전개하면:

$$o_{11} = q_{11}^T S_8 + (q_{11}^T k_9) v_9 + (q_{11}^T k_{10}) v_{10} + (q_{11}^T k_{11}) v_{11}$$

청크 내부 = 하삼각 행렬 곱셈

> *4개 출력을 한꺼번에 쓰면 ($q^T S_8$ 항 제외)*

$$\begin{bmatrix} o_9 \\ o_{10} \\ o_{11} \\ o_{12} \end{bmatrix} = \underbrace{\begin{bmatrix} q_9^T k_9 & 0 & 0 & 0 \\ q_{10}^T k_9 & q_{10}^T k_{10} & 0 & 0 \\ q_{11}^T k_9 & q_{11}^T k_{10} & q_{11}^T k_{11} & 0 \\ q_{12}^T k_9 & q_{12}^T k_{10} & q_{12}^T k_{11} & q_{12}^T k_{12} \end{bmatrix}}_{\text{하삼각 행렬, } 4\times 4} \begin{bmatrix} v_9 \\ v_{10} \\ v_{11} \\ v_{12} \end{bmatrix}$$

재귀와 행렬 곱셈은 동일한 공식의 두 가지 표현 — 결과는 완전히 동일, 계산 순서만 다름

GPU가 이 $L \times L$ 행렬 곱셈을 한 번에 처리

청크 크기 선택

왜 청크 크기 = T로 설정하지 않는가?

- 청크 내 행렬이 $T \times T \rightarrow O(T^2)$로 퇴화
- Linear Attention은 상태 업데이트 추가 오버헤드 존재
- Full Attention엔 FlashAttention 같은 극도로 최적화된 구현 존재 → 양쪽 다 손해

적절한 청크 크기 효과 (예: $L = 128$)

T	L	직렬 단계 수
384	1	384
384	**128**	**3**
384	384	1 (퇴화)

직렬 단계 $384 \rightarrow \mathbf{3}$으로 감소 각 단계는 128×128 행렬 곱셈 → GPU 가득

왜 PyTorch만으론 여전히 느린가?

> *청크 내부에 행렬 곱셈 외에도 다음 연산들이 섞여 있음*

- 상태 업데이트 (S 갱신)
- 게이팅 (g_t, β_t)
- Causal mask 적용

PyTorch는 이를 **여러 개의 작은 kernel로 분해** → 중간 결과가 HBM(显存)에 쓰고 읽기 반복 → 실제 병목은 연산이 아니라 **메모리 왕복**

FlashAttention이 빠른 이유도 동일: 전체 계산을 kernel 하나로 fuse, 중간 결과를 SRAM에 유지

FLA: Flash Linear Attention

> *FLA는 새로운 수식이 아니라, Linear Attention의 상태 업데이트를 GPU에 맞게 fuse한 고속 커널/라이브러리*

해결책: FlashAttention처럼 중간 결과를 HBM에 반복 저장하지 않도록, Linear Attention의 chunk 계산을 하나의 fused kernel 안에서 처리

```
chunk_gated_delta_rule  ←  훈련용 (chunk 내 행렬 곱셈 + 상태 + 게이팅 통합)
fused_recurrent_gated_delta_rule  ←  추론용 (token 단위 재귀, 고정 S 유지)
```

훈련은 청크 단위 병렬 계산, 추론은 토큰 단위 recurrent 계산이 유리

모드	방식	KV Cache
훈련 (chunk)	$L \times L$ 행렬 곱 + fused 상태	불필요
추론 (recurrent)	토큰 단위 재귀	불필요 (고정 S)

중간 결과를 SRAM에 유지 → HBM 왕복 최소화

MiniMind-Linear 구현

> *Full Attention → Gated DeltaNet으로 4가지 변경*

1. Softmax Attention → Gated DeltaNet

- 상태 행렬 S_t 유지
- chunk / recurrent 이중 모드

2. Short Convolution 추가

- 로컬 믹싱 레이어
- `causal_conv1d` 가속 활용

3. QK 스코어링: softmax → L2 Norm

```
def l2norm(x, dim=-1, eps=1e-6):
    return x * torch.rsqrt(
        (x * x).sum(dim=dim, keepdim=True) + eps
    )
```

4. RMSNormGated 출력 게이팅

```
x_normed * F.silu(gate.float())
```

레이어 타입 구성

> *층마다 Full Attention과 Linear Attention을 교대 배치*

```
# full_attention_interval = 4 일 때
for i in range(num_hidden_layers):
    if (i + 1) % full_attention_interval == 0:
        layer_types.append("full_attention")   # 4, 8, 12, ...
    else:
        layer_types.append("linear_attention") # 1,2,3, 5,6,7, ...
```

이유: 선형 레이어만으로는 정확한 recall이 어려운 태스크 존재 → 일정 간격으로 Full Attention을 유지해 보완

한계와 과제

> *복독(반복 생성) 문제*

선형 모델의 고정 크기 상태 S는 긴 시퀀스 생성 시 "이미 쓴 내용"을 잊어버리기 쉬움 → 반복 루프에 빠지는 경향

Full Attention 대비 현재 격차

관점	Full Attention	Linear Attention
병렬 학습 편의성	✅ 매우 좋음	⚠️ 청크 관리 필요
생태계/라이브러리	✅ 완성도 높음	🚧 FLA 등 초기
Recall 정확도	✅ 강점	⚠️ 약점
긴 시퀀스 추론	🐢 KV Cache 비례 증가	✅ 고정 상태 S
추론 속도 (long)	⚠️ 메모리 병목	✅ 유리

구현 비교: Traditional Attention

```python
class TraditionalAttention(nn.Module):
    def __init__(self, d_k):
        super(TraditionalAttention, self).__init__()
        self.d_k = d_k

    def forward(self, Q, K, V):
        Z = torch.sqrt(torch.tensor(self.d_k, device=Q.device, dtype=torch.float32))
        scores = torch.matmul(Q, K.transpose(-2, -1)) / Z   # (N, T, T) ← O(T²)
        attention_weights = F.softmax(scores, dim=-1)         # (N, T, T) ← 전체 행 정규화
        output = torch.matmul(attention_weights, V)           # (N, T, d)
        return output
```

병목: `Q @ K.T` → $T \times T$ 행렬 생성 → softmax → 다시 V와 곱셈
시퀀스 길이 T의 **제곱**에 비례하는 메모리와 연산

구현 비교: Linear Attention

```
class LinearAttention(nn.Module):
    def __init__(self):
        super(LinearAttention, self).__init__()
        self.eps = 1e-6

    def elu_feature_map(self, x):
        return F.elu(x) + 1                                    # φ(x) = ELU(x) + 1

    def forward(self, Q, K, V):
        Q = self.elu_feature_map(Q)                            # φ(Q)
        K = self.elu_feature_map(K)                            # φ(K)
        KV = torch.einsum("nsd,nsd->ns", K, V)                 # K⊙V 먼저 계산
        Z = 1/(torch.einsum("nld,nd->nl", Q, K.sum(dim=1))+self.eps)  # 정규화
```

핵심: `K⊙V` 를 먼저 계산 → $T \times T$ 행렬을 **만들지 않음**

시퀀스 길이에 **선형** 비례

벤치마크: 속도 비교

> *단일 Attention 레이어, $d_k = 64$, batch = 32, Nvidia T4 GPU*

시퀀스 길이	Traditional	Linear	속도 비
128	빠름	빠름	~1×
512	보통	빠름	~3×
1024	느림	빠름	~10×
2048 (GPT-3)	매우 느림	빠름	~25×
4096	극도로 느림	빠름	~45×
6000	한계	빠름	**~60×**

Traditional: 시퀀스 길이에 따라 **가파르게 상승**하는 곡선
$O(T^2)$ — 길이가 2배면 시간 4배

Linear: 거의 **수평에 가까운** 완만한 곡선
$O(T)$ — 길이가 2배면 시간 ~2배

복잡도 상세 분석

Traditional Softmax — $O(n^2)$

단계	연산	복잡도
QK^T	n^2개의 d_k차원 내적	$O(n^2 d_k)$
softmax	n^2개 점수 정규화	$O(n^2)$
$\times V$	n^2 행렬과 V 곱셈	$O(n^2 d_v)$

$$\text{총} : O(n^2 d_k) \approx O(n^2)$$

$d_k \ll n$ 이므로 n^2 항이 지배

Linear Attention — $O(n)$

단계	연산	복잡도
$\phi(Q), \phi(K)$	ELU 적용	$O(n \cdot d_k)$
$\phi(K)^T V$	$(d_k, n) \times (n, d_v)$	$O(n \cdot d_k \cdot d_v)$
$\phi(Q) \times$ 결과	$(n, d_k) \times (d_k, d_v)$	$O(n \cdot d_k \cdot d_v)$

$$\text{총} : O(n \cdot d_k^2) \approx O(n)$$

d_k 고정이므로 n에 선형

조건: $d_k \ll n$ (모델 차원 ≪ 시퀀스 길이)일 때 유효

$d_k = 64, n = 4096 \rightarrow n^2 = 16.7\text{M}$ vs $n \cdot d_k = 262\text{K}$ — **64배 차이**

Diffusion Language Model

개요

> *왜 Diffusion LM인가? 어떻게 구현하는가?*

이론 (YouTube: "Diffusion LMs")

- AR의 세 가지 한계
- 텍스트 Diffusion의 세 접근 방식
- Discrete Diffusion 학습/추론 원리
- LLaDA 성능, 스케일링, 데이터 효율

구현 (MiniMind Discussion #618)

- AR → dLM 핵심 변경 3가지
- MDM 학습 코드 (~300줄)
- A2D 미세조정 파라미터 효율
- 실험 결과와 한계

AR 모델의 세 가지 한계

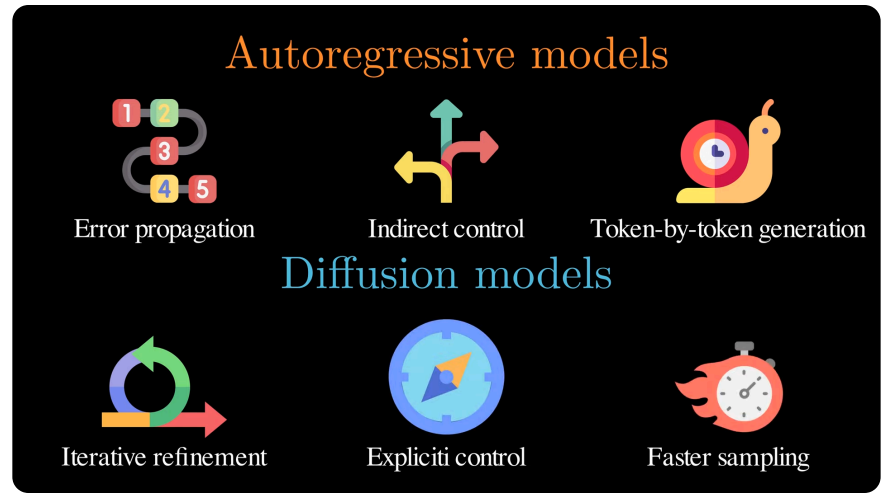
① 오류 전파 (Error Propagation)
한 토큰을 틀리면 이후 모든 예측에 영향

② 제어 어려움 (Indirect Control)
생성 중간에 개입하거나 수정하기 어려움

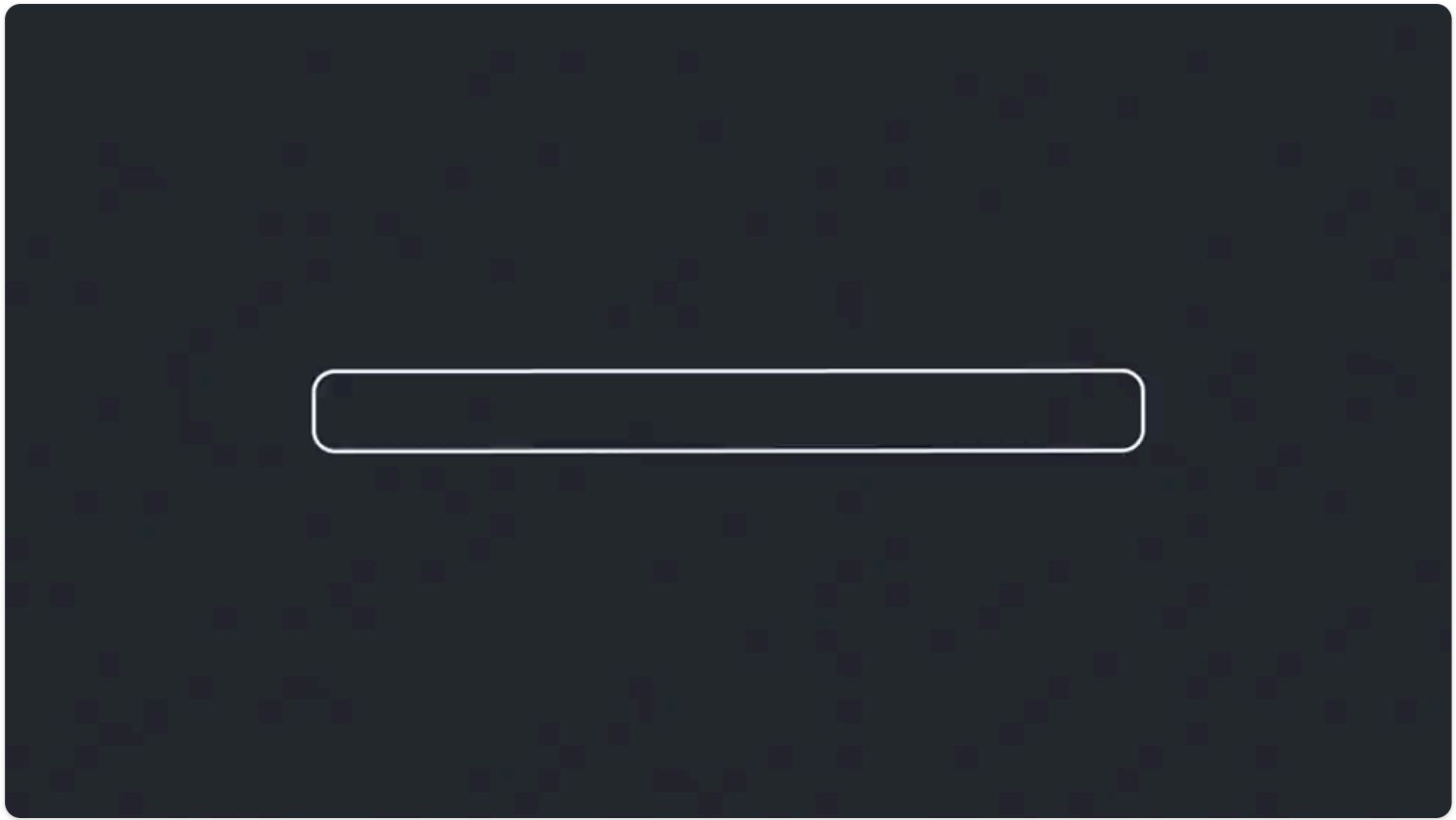
③ 순차 생성의 속도 한계
이전 토큰이 확정되기 전에 다음 토큰 예측 불가

→ **Diffusion 모델**: 반복적 개선, 명시적 제어, 병렬 생성

Mercury (diffusion LM): 1,000 tokens/sec



Diffusion LM 데모: Mercury

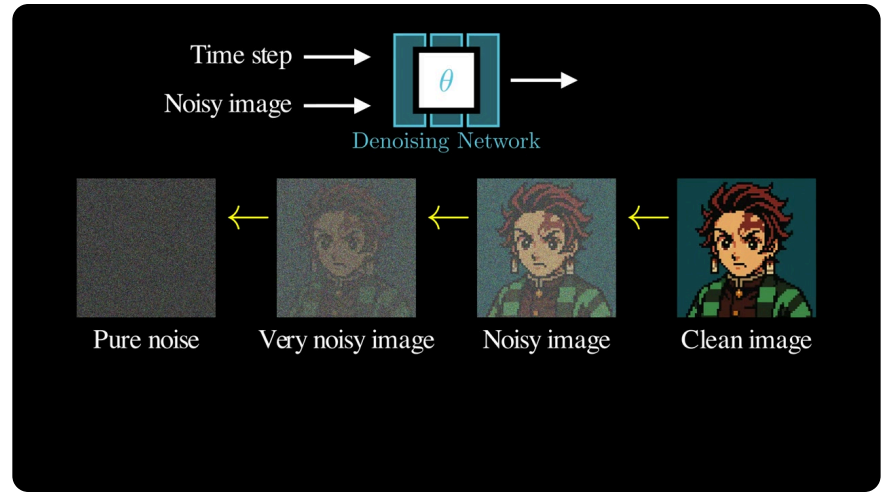


이미지 Diffusion: 복습

Forward process: 깨끗한 이미지 → 점진적 Gaussian noise 추가

Reverse process: Denoising network가 노이즈 → 원본 복원

학습: 노이즈가 낀 이미지를 받아 원본을 예측하도록 훈련
추론: 순수 노이즈에서 시작해 반복적으로 복원



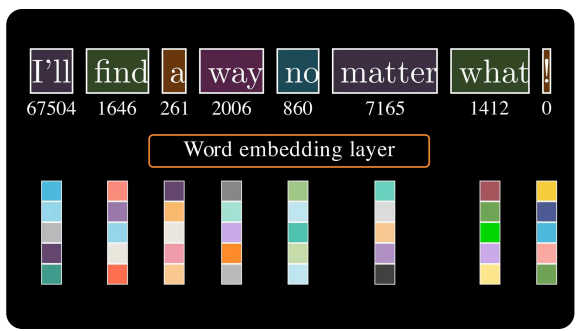
텍스트에 Diffusion을 적용하는 방법

> *텍스트는 이산(discrete) 토큰 ID → Gaussian noise를 직접 더할 수 없음*

접근 1: Embedding Space Diffusion
토큰 → Word embedding → Gaussian noise 추가 → Denoising → 반올림(Rounding) → 토큰

접근 2: Latent Diffusion
문장/단락 → Encoder → Latent embedding → Diffusion → Decoder → 텍스트

접근 3: Discrete Diffusion (Masked)
토큰 → 일부를 `[MASK]`로 치환 → 복원 예측 → 반복

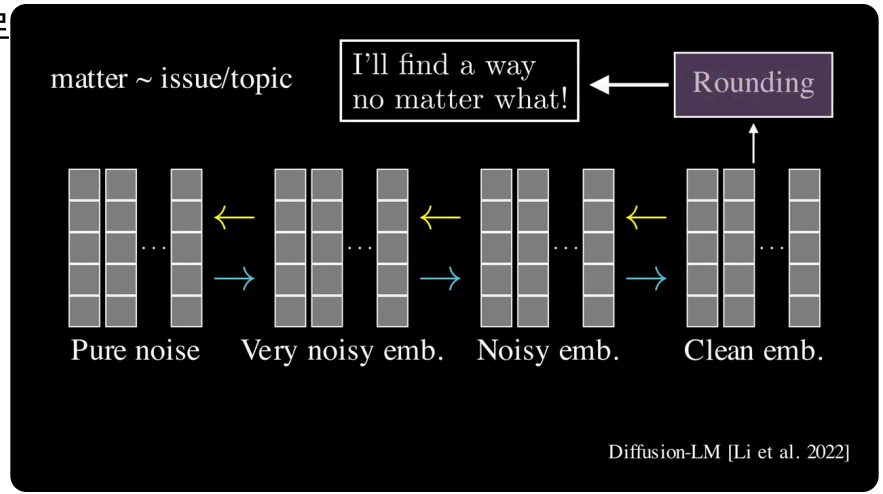


접근 1의 문제: Rounding 오류

Embedding space에서 denoising 후 **가장 가까운 토큰**으로 반올림

문제: `matter` ≈ `issue` ≈ `topic`
→ 반올림 단계에서 의미가 뒤바뀜
→ 문장이 의미를 잃음

Diffusion-LM [Li et al. 2022]



접근 2: Latent Diffusion

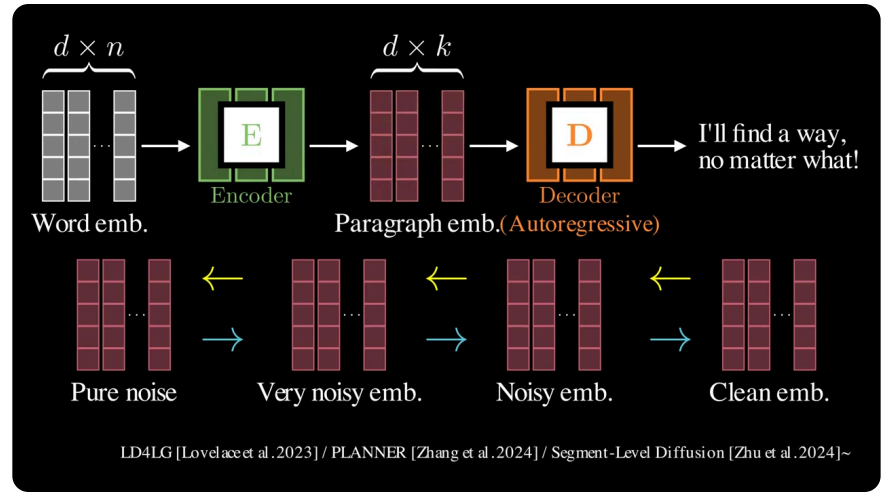
문장 전체를 하나의 latent vector로 압축 후 Diffusion

- Encoder: $d \times n$ 개별 임베딩 → $d \times k$ 단락 임베딩
- Diffusion: Latent space에서 노이즈 추가/제거
- Decoder (AR): 복원된 latent → 텍스트 생성

Latent diffusion은 continuous한 공간에서 작동하므로 Gaussian noise가 잘 맞습니다.

장점: 의미적 단위에서 Diffusion
단점: Discrete ↔ Continuous 변환을 위한 Encoder/Decoder가 추가로 필요

LD4LG, PLANNER, Segment-Level Diffusion



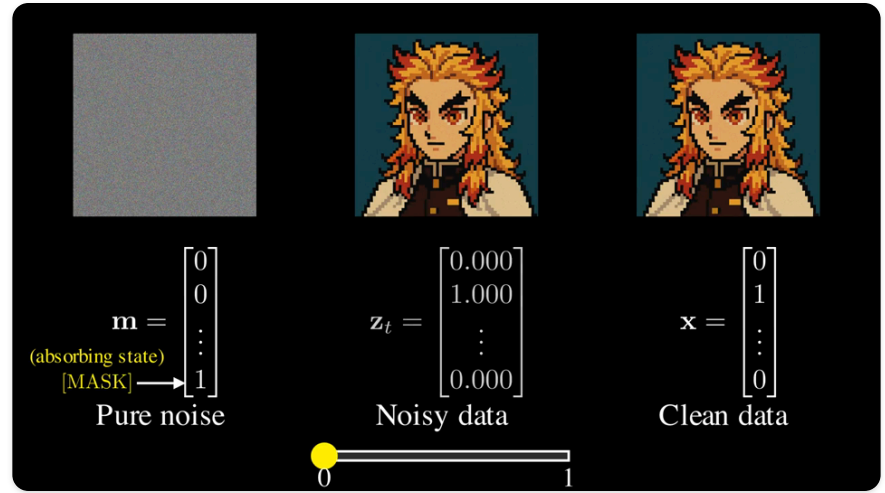
접근 3: Discrete Diffusion

> *[MASK] 토큰 = 흡수 상태 (Absorbing State)*

- Clean token: one-hot vector $\mathbf{x} = [0, 1, 0, \ldots, 0]$
- Pure noise: $\mathbf{m} = [0, 0, \ldots, 0, 1]$ (마지막 = MASK)
- Noisy token: $(1 - t)\,\mathbf{x} + t\,\mathbf{m}$

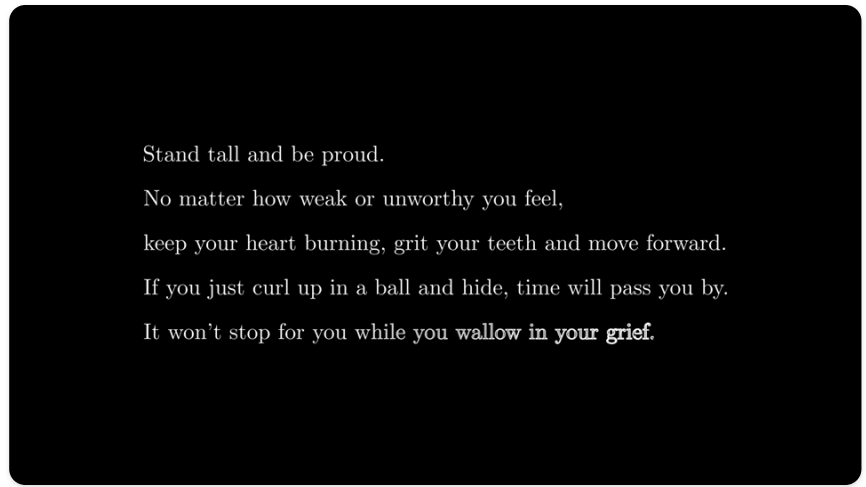
시간 $t \in [0, 1]$이 커질수록 MASK 확률 증가

$t = 1$이면 전체가 MASK



Forward Process: 마스킹

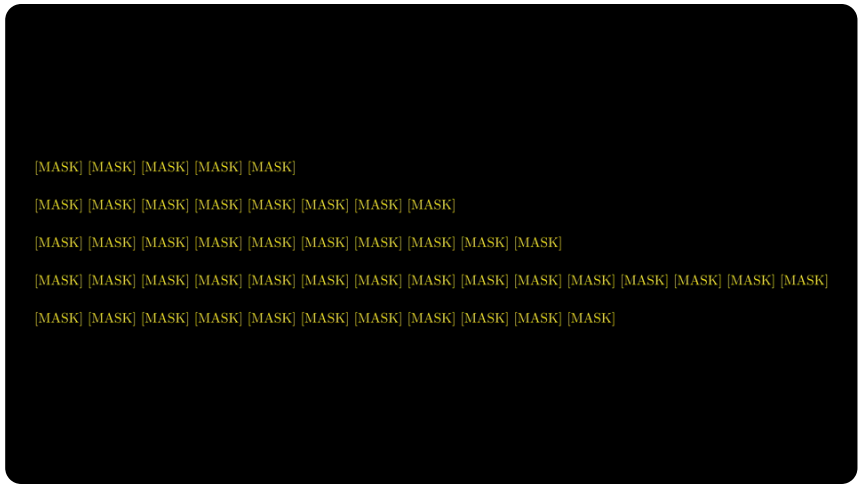
> ***Forward**: 깨끗한 텍스트 → 점점 더 많은 토큰을 `[MASK]` 로 치환*



각 단계에서 일정 확률로 토큰이 MASK 상태로 전이
최종 단계($t = 1$)에서 모든 토큰이 MASK

Reverse Process: 단계적 Unmask

> ***Reverse**: 전체 MASK → 한 번에 한 단계씩 토큰 복원*



각 denoising step에서:

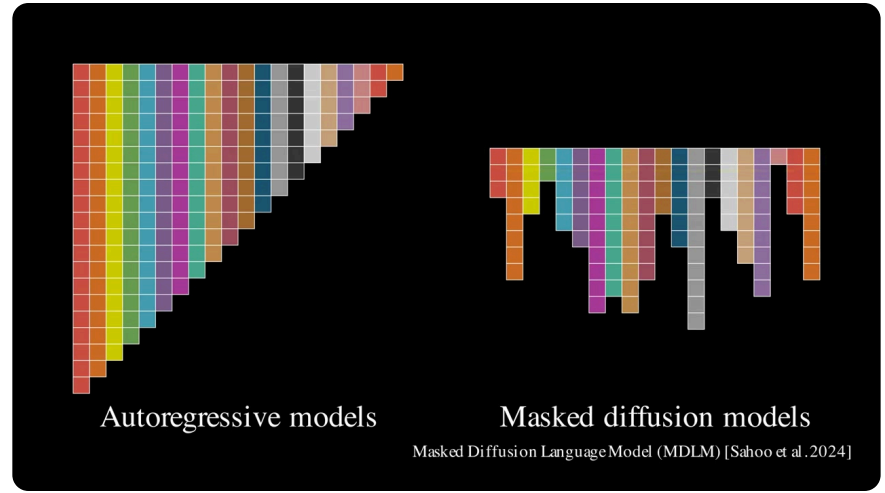
1. 현재 masked sequence를 전체 입력으로 받음 (양방향 주의!)
2. 각 MASK 위치에 대해 vocabulary 분포 예측
3. 확신도 높은 위치부터 순서대로 확정

AR vs MDM: 생성 방식 비교

AR (Autoregressive)
왼쪽 → 오른쪽, 토큰을 순서대로 확정
한 번 결정하면 수정 불가

MDM (Masked Diffusion)
전체 위치를 동시에 보며 점진적 복원
확신도 낮은 위치는 나중에 처리

Remasking 전략 (MDLM):
오류 수정을 위해 이미 예측한 토큰도
다시 MASK로 되돌려 재예측 가능



Diffusion LM의 생성 길이 문제

> *AR은 `<EOS>` 가 나올 때까지 자유롭게 생성하지만, Diffusion LM은 **생성할 길이를 미리 정해야** 한다*

```
AR:        [prompt] → 토큰 하나씩 생성 → <EOS>에서 멈춤 (길이 자유)
Diffusion: [prompt][MASK × N] → 전체를 동시에 언마스킹 (N을 미리 지정)
```

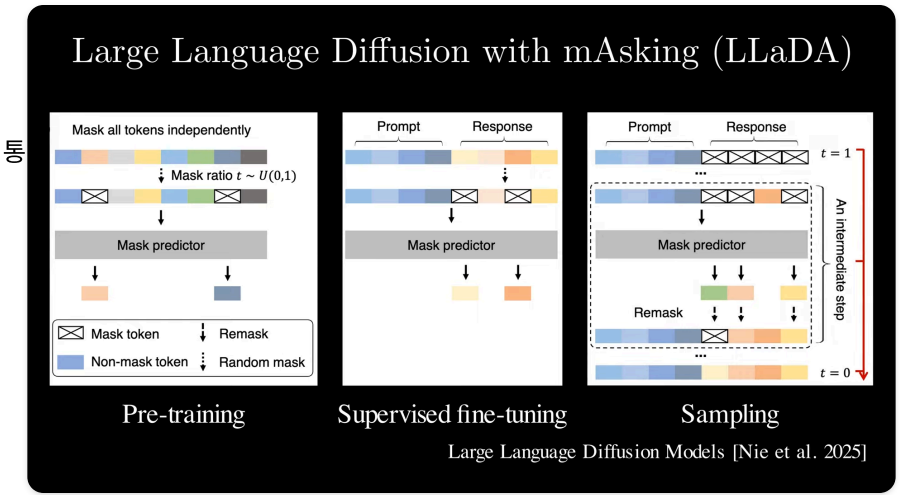
	AR	Diffusion LM
길이 결정	생성 중 `<EOS>` 로 자동 결정	시작 전에 고정
짧은 답변	자연스럽게 처리	남은 MASK가 불필요한 토큰이 될 수 있음
긴 답변	자연스럽게 처리	미리 정한 길이 초과 불가

대응: 충분히 긴 길이를 설정 후 패딩 제거, 또는 별도 length predictor 사용

Remasking

> *이미 확정한 토큰도 다시 `[MASK]`로 되돌려 재예측하는 샘플링 전략*

- 낮은 confidence 토큰을 고정하지 않고 다음 step에서 다시 검토
- `dog → cat`처럼 직접 바꾸지 않고 `dog → [MASK] → cat` 경로를 사용
- 매 iteration마다 remask 개수를 줄여 점점 clean sequence로 수렴
- LLaDA는 pre-training, SFT, sampling 모두 masked prediction 관점으로 통일



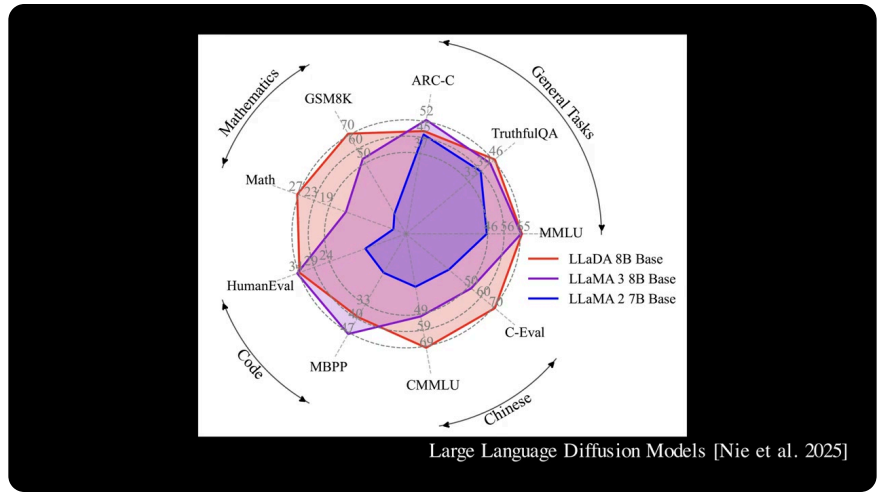
LLaDA: AR 수준의 성능

LLaDA 8B (Large Language Diffusion with mAsking)
= 처음부터 dLM으로 학습 (A2D 아님)

- Pretraining + SFT 패러다임 유지
- In-context learning에서 LLaMA 3 8B와 동등
- **Reversal Curse 극복**: GPT-4 초과 (역방향 완성 벤치마크)

AR이 아니어도 핵심 LLM 능력이 가능함을 보여준 연구

Large Language Diffusion Models [Nie et al. 2025]



스케일링: 단일 epoch에서는 16배 갭

Power-law 스케일링은 MDM에서도 성립:
compute 증가 → validation loss 감소

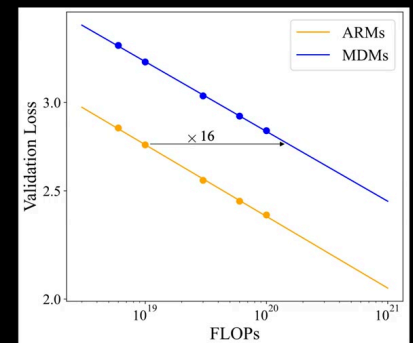
단, **같은 compute에서 MDM은 AR보다 ~16배 더 필요**
(단일 epoch 기준)

이는 한계처럼 보이지만…
→ 데이터 제약 시나리오에서 역전됩니다

Scaling up Masked Diffusion Models on Text [Nie et al. 2025]

Single epoch training

Data-constrained settings?



Scaling up Masked Diffusion Models on Text [Nie et al. 2025]

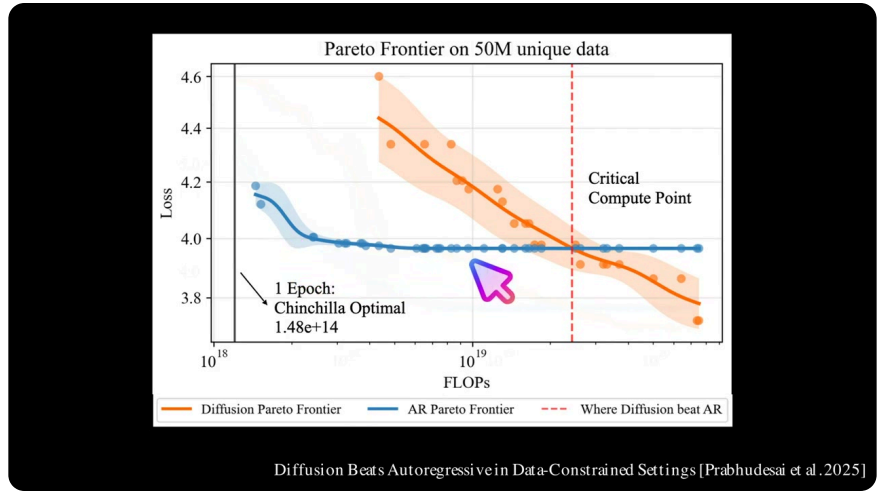
데이터 제약 시나리오: MDM의 역전

실제 학습은 제한된 데이터를 **여러 epoch** 반복

- **AR**: 4~8 epoch 이후 과적합, loss 정체
- **MDM**: 수백 epoch까지 계속 개선

Critical Compute Point 이후 MDM이 AR을 역전
(50M 고유 데이터 기준)

Diffusion Beats Autoregressive in Data-Constrained Settings [Prabhudesai et al. 2025]

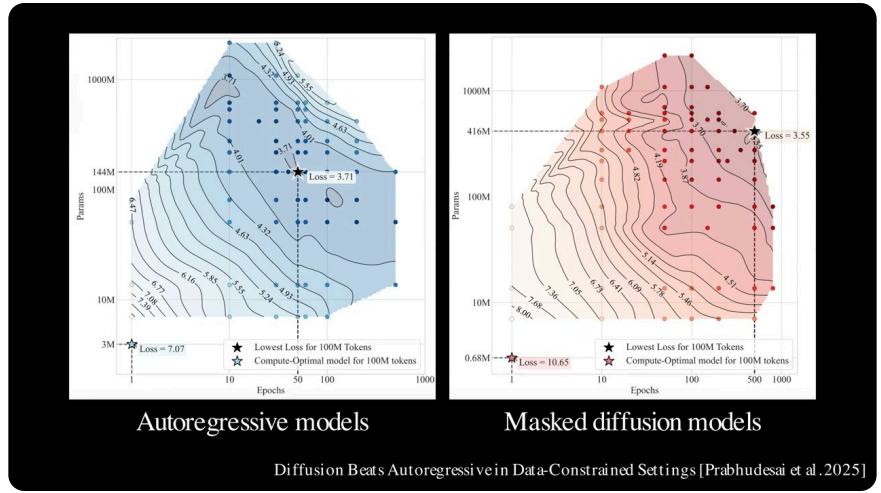


데이터 효율: epoch 스케일링

AR은 4 epoch 반복 ≈ 새 데이터
→ 그 이후는 반복 훈련이 거의 도움 안 됨

MDM은 **100 epoch까지** 반복 훈련이 여전히 유효
→ 데이터 반복에 훨씬 강인

AR: 1 epoch 기준 최저 loss 3.71 → 다 훈련 시 3.71
MDM: 1 epoch 기준 10.65 → 다 훈련 시 3.55
(67% 개선 vs AR 48% 개선)

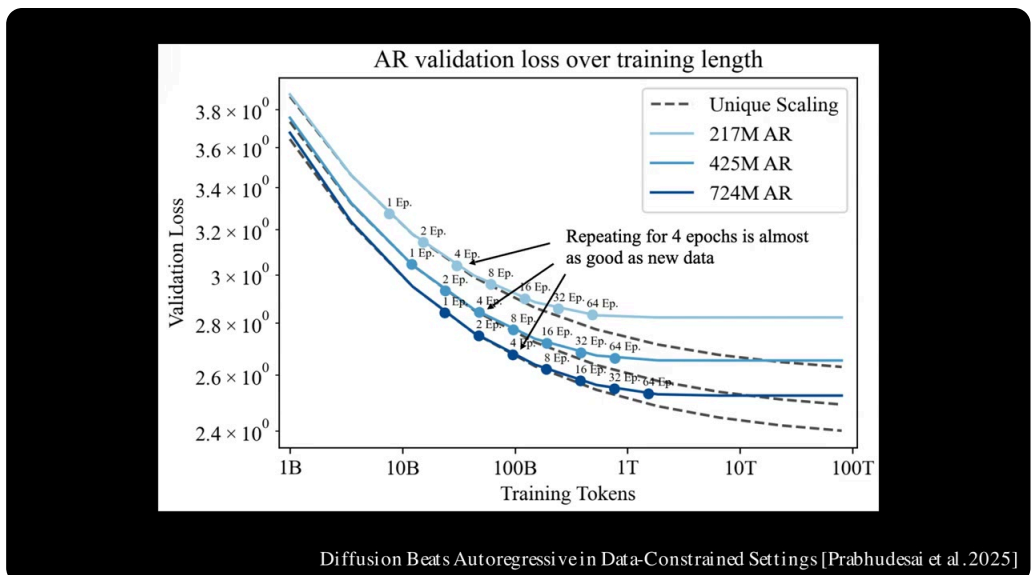


마스킹 조합이 만드는 학습 신호

왜 MDM이 데이터 효율적인가?

AR: 시퀀스 $[x_1, x_2, x_3, x_4]$에서 학습 경우의 수
→ 한 가지 순서 (왼쪽 → 오른쪽)

MDM: 같은 시퀀스에서 학습 경우의 수
→ 마스킹 조합이 $2^4 - 1 = 15$가지
→ 훨씬 다양한 prediction 문제를 학습



AR은 4 epoch 후 새 데이터와 동일한 효과가 사라지지만, MDM은 100 epoch까지 마스킹 조합의 다양성으로 새로운 학습 신호를 얻음

A2D 미세조정: 파라미터 효율

> *AR → dLM 전환에 **전체 파라미터를 학습할 필요 없음***

전략	설명	효율
전체 학습	모든 파라미터	기준
FFN 동결 + Attention 학습	지식 유지, attention 패턴만 변경	~30% 파라미터
Q, K 프로젝션만 학습	극단적 효율	~10% 파라미터

```
for name, param in model.named_parameters():
    param.requires_grad = False
    if 'q_proj' in name or 'k_proj' in name:
        param.requires_grad = True    # Q, K만 학습
```

FFN은 지식 저장 → 변경 불필요
핵심 변화(Causal→Bidirectional)는 Q, K 행렬에 집중

결과 & 한계

MiniMind A2D 결과 (63.91M, ~1.6GB SFT, ~1시간):

- 기본 Q&A 구조는 생성 가능
- 반복 표현, 논리 비약 다수
- AR 대비 명확히 낮은 품질

한계 요인:

- 학습 데이터/시간 부족
- `block_size`, `steps` 하이퍼파라미터 미최적화
- 베이스 AR 모델의 품질에 상한이 있음

개선 방향:

1. 더 많은 데이터와 학습량
2. Noise schedule 최적화 (cosine 등)
3. Remasking 전략 도입
4. 더 강한 베이스 AR 모델
5. 파라미터 효율 미세조정 (Q/K only)

dLM 연구 현황 종합

관점	AR	Diffusion LM
이론 기반	완비된 확률 체인	Masked Prediction 확장 (미성숙)
단일 epoch	효율적	16배 compute 갭
데이터 제약	4 epoch 후 정체	100 epoch까지 개선
추론 속도	KV Cache 가능	매 step 전체 재계산
생성 품질	현재 우세	대규모 시 경쟁력
CoT / 순차 추론	강점	미해결

결론: 데이터 제약 환경 또는 비순차 생성이 중요한 태스크에서 잠재력
단, 이론·엔지니어링 모두 아직 성숙하지 않음

참고 자료

- MiniMind Discussion #618 — A2D 구현 코드
- Discrete Diffusion LMs: Principles & Implementation — 시각화 데모
- MDLM [Sahoo et al. 2024] — Masked Diffusion Language Model
- LLaDA / LaDa [Nie et al. 2025] — Large Language Diffusion with mAsking
- Scaling MDMs [Nie et al. 2025] — 스케일링 법칙 분석
- Diffusion Beats AR [Prabhudesai et al. 2025] — 데이터 제약 시나리오 비교
- Diffusion-LM [Li et al. 2022] — 임베딩 공간 diffusion 선구 연구

Physical AI

Physical AI와 VLA의 정의

> *Physical AI는 무엇을 새롭게 가능하게 하는가?*

Vision

보고

Language

이해하고

Action

행동한다

LeRobot 모델 분류표

학습 방식과 모델 역할 기준으로 분류

Category	Models	설명
Imitation Learning	ACT, Diffusion, VQ-BeT	데모 데이터로 action policy 학습
VLA Models	Pi0, SmolVLA, GR00T N1.5	vision + language + action 통합
World Models	VLA-JEPA	세계 상태 변화를 예측
Reward Models	SARM, TOPReward	행동 품질 평가

Physical AI 용어

> *Physical AI는 아직 하나의 엄밀한 표준 용어가 아닙니다.*

관점	핵심 표현	포함 범위
넓은 정의	몸을 가진 AI	자율주행, 드론, 산업용 로봇, 휴머노이드, 디지털 트윈
좁은 정의	VLA 기반 범용 로봇 지능	end-to-end 학습 로봇, 로봇 파운데이션 모델, cross-embodiment 생태계

넓게 잡으면 산업 전반을 묶는 말이 되고, 좁게 잡으면 **LLM 이후 로보틱스의 전환점**이 보입니다.

로봇 데이터 수집

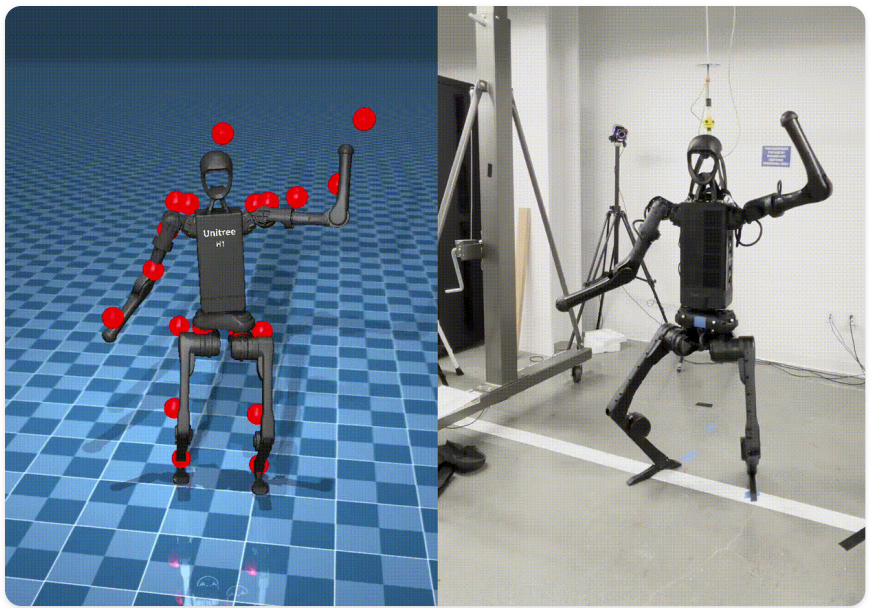
> *로봇으로 직접 수집하면 해당 로봇으로 제한됨, 사람의 행동을 수집하면 변환 문제가 발생함*

대규모 텔레오퍼레이션 데이터 수집 (AgiBot)

VR 기반 일상 동작 데이터 수집

NVIDIA Isaac Lab

> *GPU 가속 로봇 시뮬레이션 프레임워크*

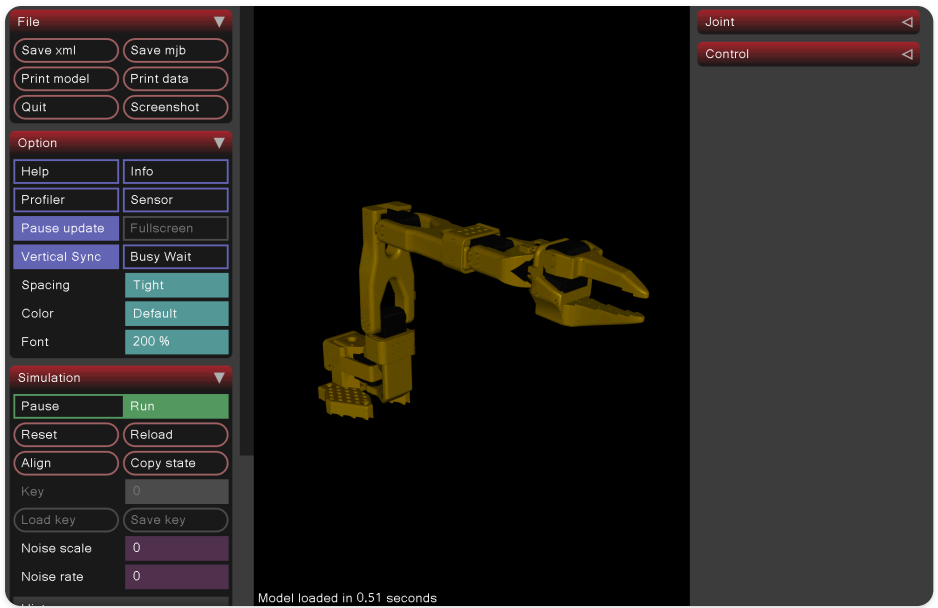


- **Isaac Sim 기반**: NVIDIA Omniverse 위에서 동작하는 로봇 학습 프레임워크
- **대규모 병렬 시뮬레이션**: 수천 개 환경을 GPU에서 동시 실행하여 학습 속도 극대화
- **Sim-to-Real**: 시뮬레이션에서 학습한 정책을 실제 로봇에 바로 전이
- **도메인 랜덤화**: 조명, 텍스처, 물리 파라미터를 무작위로 변경하여 일반화 성능 확보
- **RL/IL 통합**: Stable Baselines3, RSL-RL 등 주요 학습 라이브러리와 연동

실제 로봇 없이도 **수백만 에피소드**를 생성할 수 있어
텔레오퍼레이션 대비 데이터 수집 비용을 크게 절감

MuJoCo

> *Multi-Joint dynamics with Contact — DeepMind 오픈소스 물리 엔진*

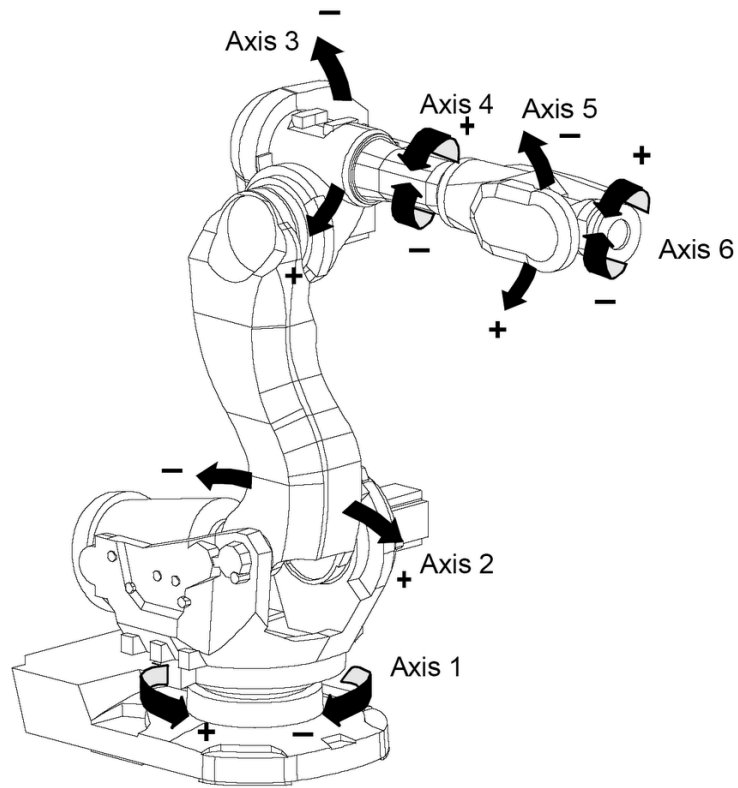


- **정밀 접촉 시뮬레이션**: 관절 마찰, 충돌, 미끄러짐 등 물리적 상호작용을 정확히 모델링
- **빠른 연산 속도**: 경량 C 엔진으로 실시간보다 수백 배 빠른 시뮬레이션 가능
- **로봇 학습 표준 환경**: OpenAI Gym, LeRobot 등 주요 RL/IL 프레임워크가 MuJoCo 기반
- **MJCF/URDF 지원**: XML 기반 모델 정의로 다양한 로봇 형태를 쉽게 구성
- 2022년 DeepMind가 **오픈소스로 전환** → 연구 접근성 대폭 확대

ACT, Diffusion Policy 등 주요 로봇 정책들이
MuJoCo 환경에서 먼저 검증한 뒤 실제 로봇에 배포

로봇 액션이란?

6-DOF 관절 각도 제어



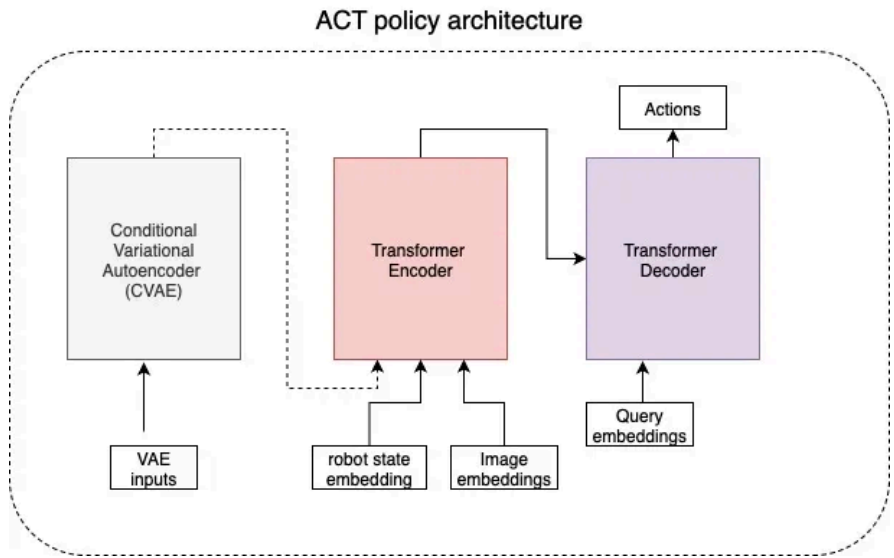
ACT: 실용 로봇 정책의 대표 사례

다음 1개가 아니라 k개 액션 시퀀스를 예측하는 imitation learning 정책

항목	내용
핵심	k개 액션 시퀀스 예측 (Action Chunking)
하드웨어	저비용 양팔 ALOHA
입력	다중 RGB 카메라 + 관절 위치
출력	90 timesteps action sequence, 50Hz

ACT Architecture

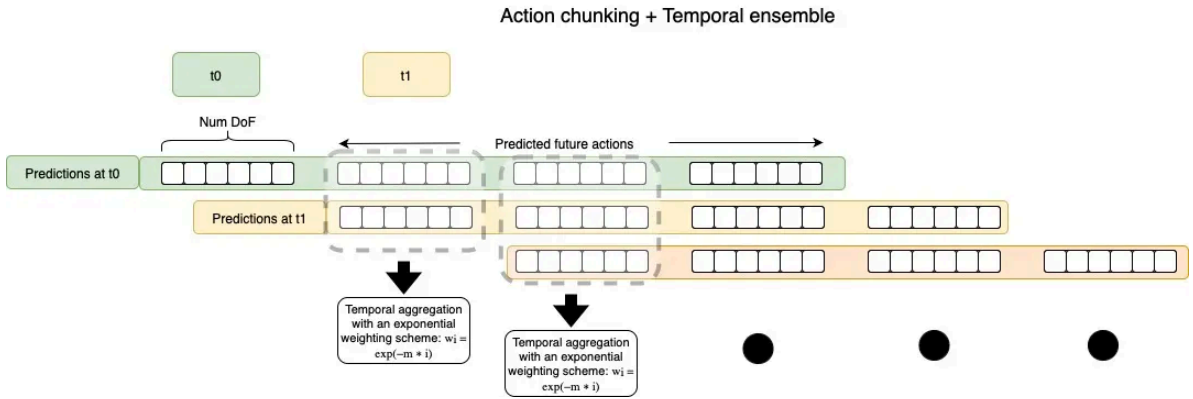
> *CVAE + Transformer Decoder + Temporal Ensembling*



구성	역할
CVAE	시연 스타일을 잠재 변수로 표현
Transformer Decoder	action chunk 병렬 생성
Temporal Ensembling	겹치는 chunk를 평균해 부드럽게

Action Chunking

- ACT는 action timestep query로 k개 미래 액션을 한 번에 예측합니다.
- 겹치는 chunk 예측을 지수 가중 평균하여 안정적 궤적 생성

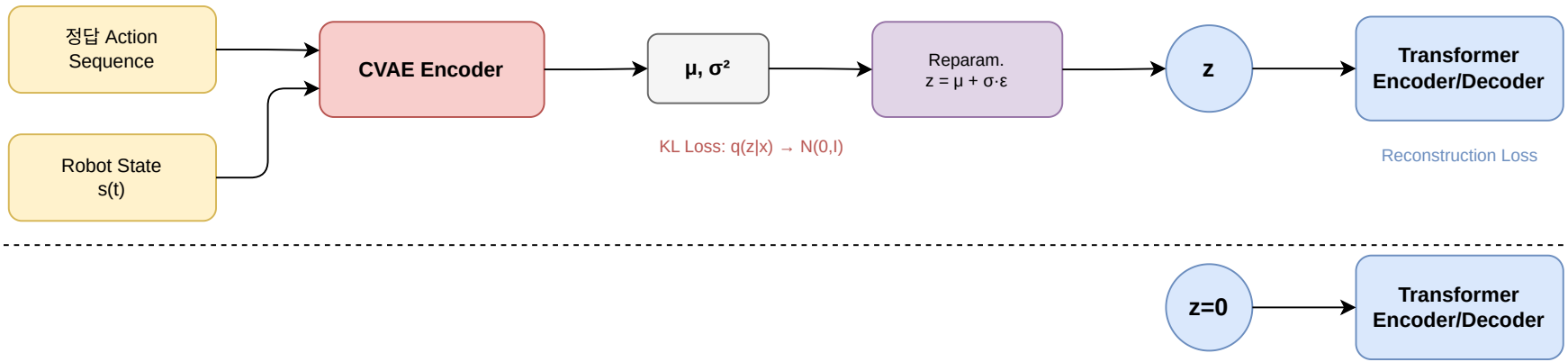


장점

- 결정 횟수 감소: 100step → 10chunk = 10번만 결정
- 오차 누적 완화: chunk 단위로 오차 전파 횟수 감소
- chunk 내부는 한 번에 계획 → 동작이 부드러움

ACT: CVAE의 Latent z

> *z는 전체 모델과 End-to-End로 동시 학습, 추론 시에는 z=0 고정*



Training

- 정답 action + 현재 상태 → CVAE Encoder → $\mu, \sigma \rightarrow z$
- **Reparam. Trick**: $z = \mu + \sigma \cdot \epsilon$ (미분 가능하게)
- **Reconstruction Loss** + **KL Loss** ($z \rightarrow \mathcal{N}(0, I)$)

Inference

- CVAE 없이 $z = 0$ 상수를 점선 경로로 주입
- KL Loss가 $\mu \rightarrow 0$으로 밀어놨으므로 0 = 분포의 중심

z의 역할: 시연의 스타일·모호성을 압축하여 디코더가 안정적으로 학습하도록 돕는 징검다리

ACT의 Latent z와 Loss

> *같은 관측에서 여러 성공 궤적을 latent z로 흡수*

Latent z

- observation-action → Gaussian latent (μ, σ^2)
- reparam trick으로 미분 가능한 sampling
- inference: $z = 0$ 사용

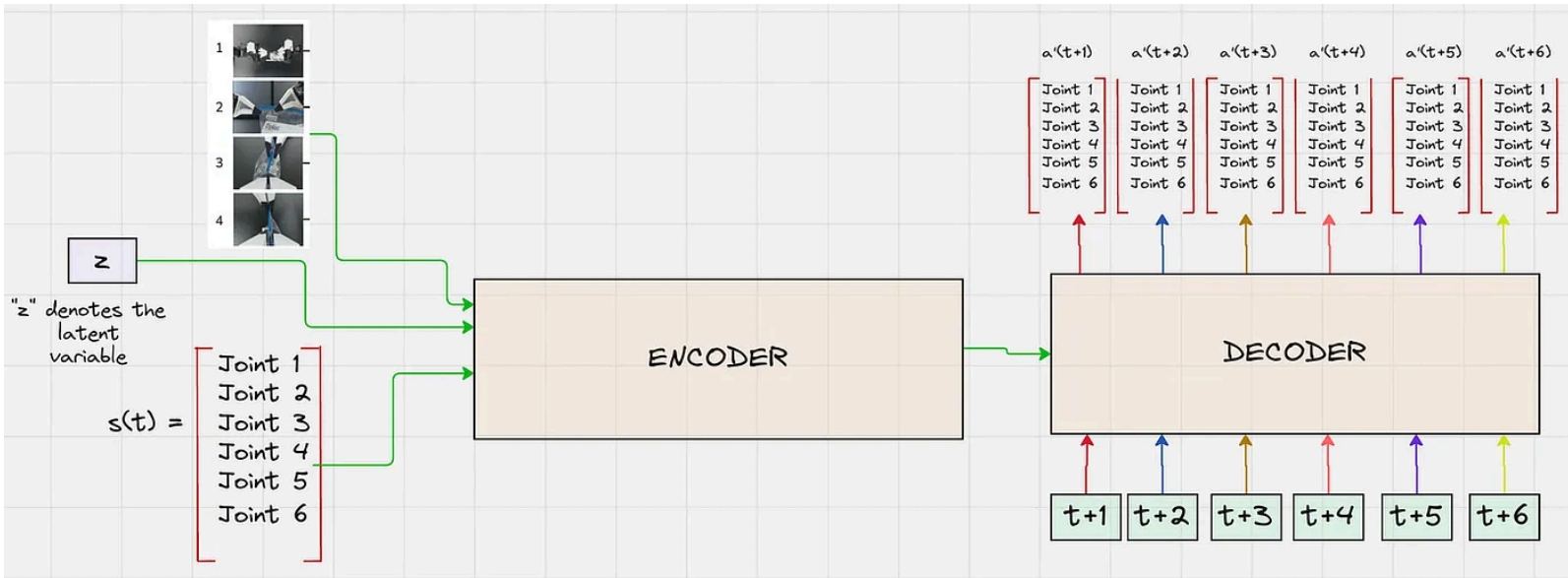
Loss

$$\mathcal{L}_{ACT} = \mathcal{L}_{recon} + \beta D_{KL}(q_\phi(z \mid o, a_{1:k}) \| p(z))$$

- $\mathcal{L}_{recon}$: 예측 vs 시연 joint position MSE
- KL: z가 prior에서 벗어나지 않게 정규화
  - KL 없으면: z가 흩어져 추론 시 $z = 0$이 미지의 영역 → 엉뚱한 action
  - KL 있으면: z가 0 근처에 모여 $z = 0$이 안정적 평균 행동

ACT: 입출력

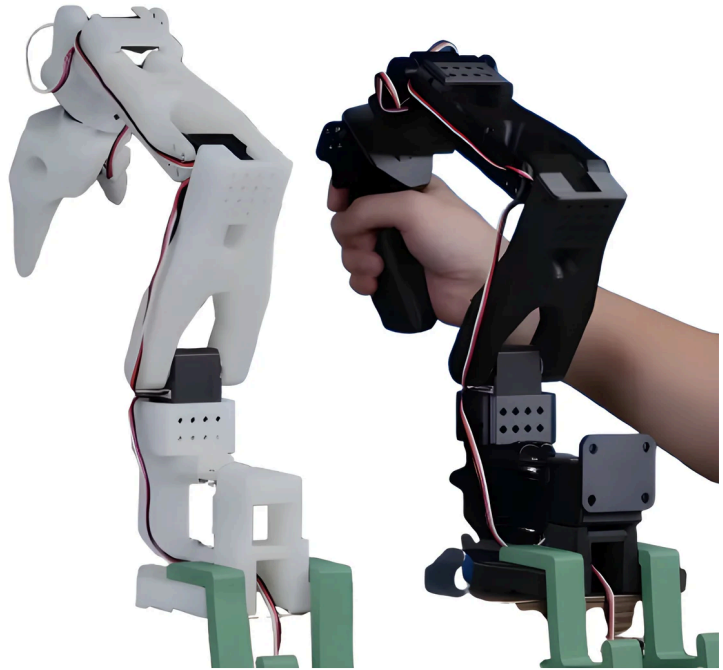
> *관측 이미지 + 관절 상태 + latent z → Encoder → Decoder → k개 미래 액션 예측*



- **Encoder 입력**: 카메라 이미지 + 관절 상태 s(t) + latent z → context 생성
- **Decoder**: Encoder context + timestep query(t+1 ~ t+k) → k개 관절 액션 병렬 예측
- **출력**: action chunk a'(t+1) ~ a'(t+k), 각각 전체 관절(Joint 1~6)의 목표값

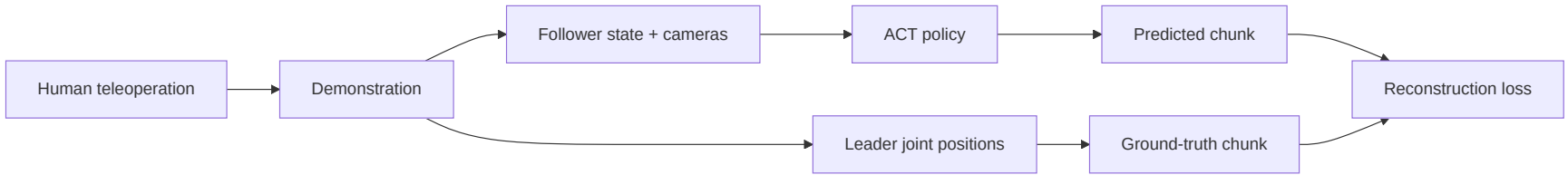
Leader-Follower Teleoperation

> *사람이 leader arm을 조작하면 follower arm이 동기화되어 시연 데이터를 수집*



ACT 학습 파이프라인

> *leader-follower teleoperation → action chunk 복원 학습*



단계	의미
데이터 수집	사람이 leader arm 조작
행동 라벨	leader joint position = ground truth
관측 입력	follower joint + multi-view camera
학습 목표	k-step joint position sequence 예측

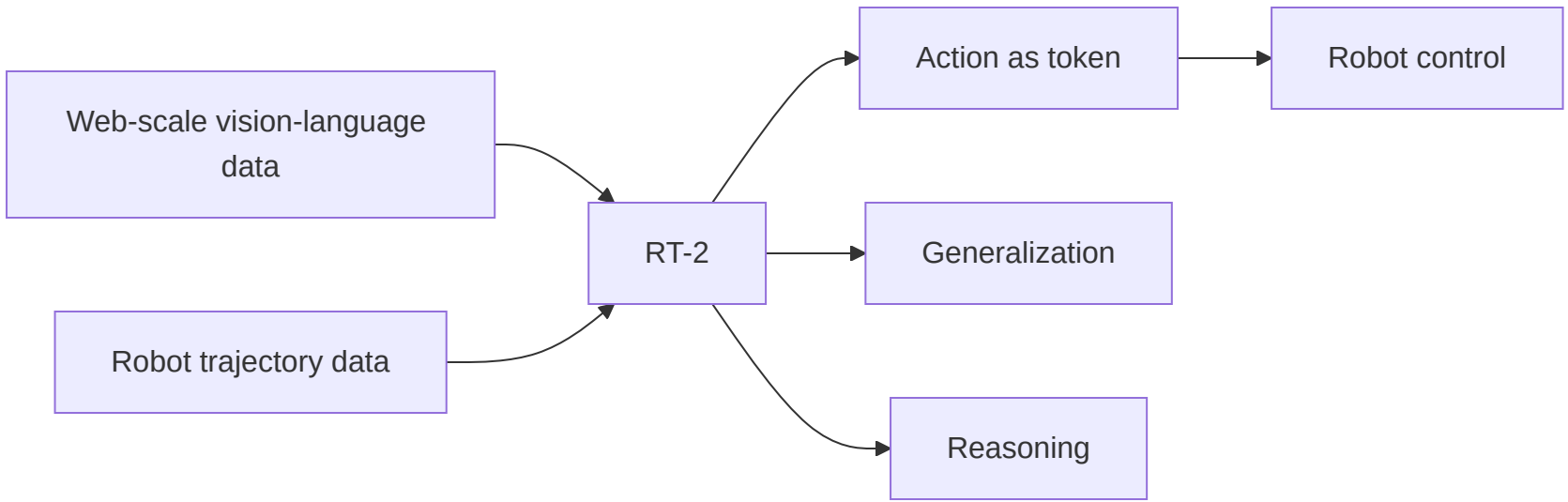
ACT의 위치

VLA가 아닌 가ㅈ한 action policy baseline — VLA로 가는 길의 출발점

관점	의미
모델링	action을 sequence로 다룸
데이터	적은 teleop data로 데모 가능
생태계	LeRobot 기본 추천 모델
한계	language/world knowledge 없음

경계점: RT-2 이후

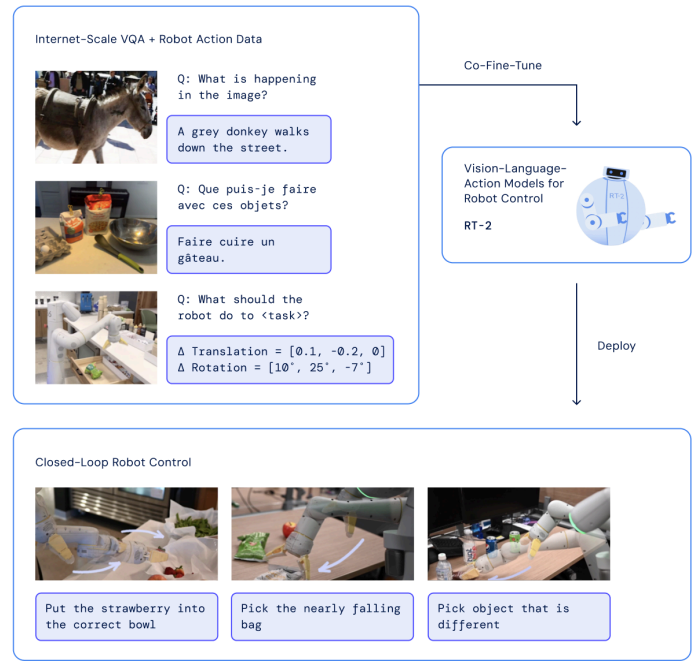
> *웹에서 배운 지식을 로봇 행동으로 연결할 수 있다는 가능성*



RT-2 요약

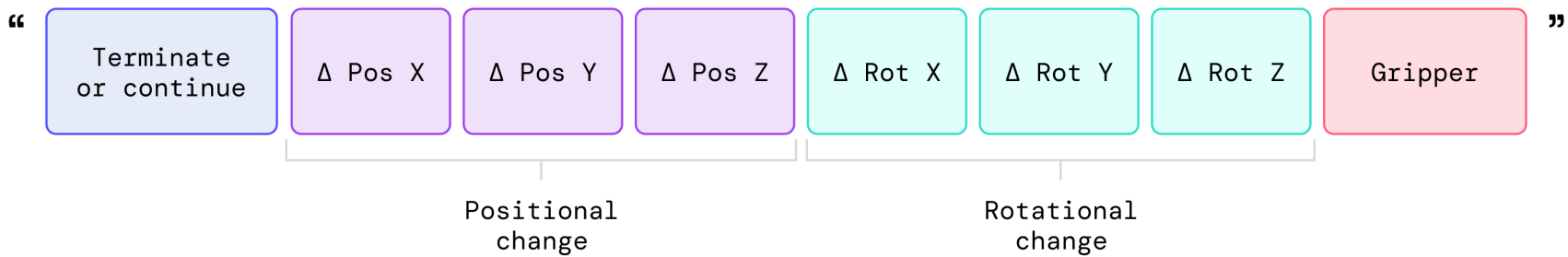
> *기존 VLM을 로봇 데이터와 co-fine-tune하여 action token을 출력*

구성	의미
Internet-scale VQA	웹 규모 reasoning 능력
Robot action data	이미지 + 명령 + 로봇 행동
Co-fine-tune	VQA + 로봇 데이터 함께 학습
Deploy	token → action으로 closed-loop 제어



RT-2 Approach Overview

로봇 행동을 언어 token처럼 다룸



Action을 token으로 바꾸기

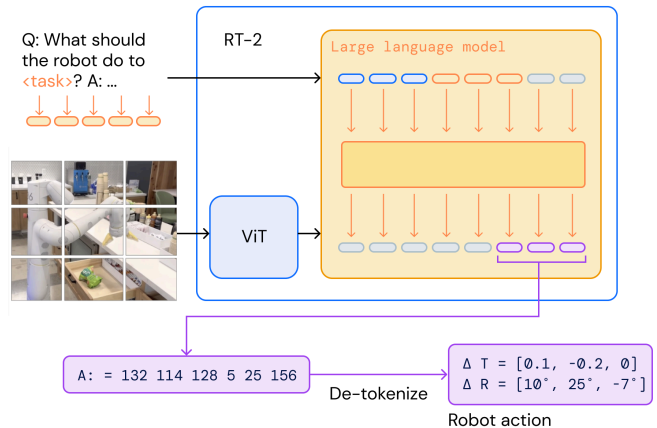
- terminate / continue
- position 변화: `Δ Pos X`, `Δ Pos Y`, `Δ Pos Z`
- rotation 변화: `Δ Rot X`, `Δ Rot Y`, `Δ Rot Z`
- gripper 상태

왜 중요한가

- 자연어 답변과 로봇 행동을 같은 token 형식으로 학습
- VLM backbone 그대로 활용
- action prediction = language modeling

RT-2 Inference: Token에서 제어로

> *image + instruction → action token 생성 → de-tokenize → 로봇 제어*



단계	설명
입력	camera image + 지시 prompt
출력	이산화된 action token sequence
de-tokenize	ΔTranslation, ΔRotation, gripper로 변환
closed-loop	매 timestep 관측 재입력

RT-2가 보여준 일반화

> *로봇 policy가 web-scale VLM의 semantic prior를 물려받을 수 있다*

일반화 축	예시
Novel objects	로봇 데이터에 없던 물체로 지시 수행
Symbol understanding	숫자, 아이콘 포함 명령 해석
Reasoning	가장 작은 물체 선택 등
Chain-of-thought	reasoning → Action token 출력

Diffusion Policy: 액션을 생성한다

> *연속 액션 분포를 반복적 denoising으로 샘플링*

```
순수 노이즈 → 중간 노이즈 → 최종 액션 시퀀스
```

항목	내용
입력	시각 관측 + 로봇 상태
출력	미래 액션 시퀀스
핵심	다중 모드 연속 액션을 denoising으로 생성

왜 Diffusion인가

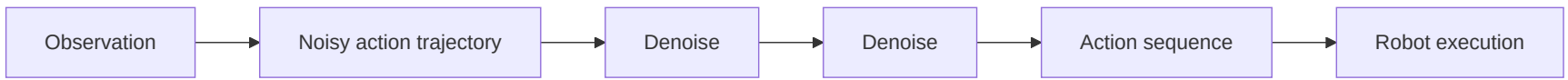
> *같은 장면에서 유효한 행동은 하나가 아님 → 다중 모드 처리 필요*

다중 모드 행동

- 왼쪽/오른쪽 어디로 밀어도 성공
- 위/옆 어디서 잡아도 성공

Diffusion의 처리

- 여러 모드를 학습하고 rollout마다 하나에 commit
- 고차원 연속 액션 공간에 적합



ACT vs Diffusion Policy

> *둘 다 action sequence를 다루지만, 액션을 만드는 방식이 다릅니다.*

항목	ACT	Diffusion Policy
생성 방식	CVAE decoder	Denoising diffusion
다중 모드	style variable `z`	action distribution 자체를 샘플링
추론 속도	단일 forward pass	여러 denoising step 필요
강점	빠르고 데이터 효율적	복잡한 연속 행동 분포에 강함
생태계	ALOHA, LeRobot	LeRobot, 후속 VLA action decoder

이후 pi0(flow matching), Octo(diffusion decoder)로 이어짐

왜 한 번에 액션을 찍지 못할까

> *맥락 없는 1:多 매핑 → 다중 모드를 평균내면 실패 궤적*

무작위 노이즈만 입력

- 가능한 행동이 너무 많음 (잡기, 밀기, 회피...)
- 하나의 출력으로 눌러야 함

ACT도 한 번에 추론하지만?

- CVAE의 z로 모드를 분리 → 추론 시 z=0 고정 → 결국 평균 모드만 출력.
- 모드가 적은 단순 task에는 충분하지만, 복잡한 다중 모드에서는 한계

결과: 평균 행동

- 다중 모드를 평균 → 성공 궤적이 아님
- 떨림, 허공 찌르기, 애매한 움직임

한 번에 순간이동 vs 조금씩 이동

> *불가능한 one-shot을 작은 방향 벡터의 적분 문제로 바꿈*

방식	직관	모델이 해야 하는 일
One-shot	순간이동	$x_0 \mapsto x_1$ 한 번에 추정
Flow / Diffusion	나침반 보며 조금씩 이동	x_t에서 다음 방향 dx/dt 예측

매 step 현재 위치를 보고 방향을 갱신 → 복잡한 매핑을 작은 국소 문제로 분해

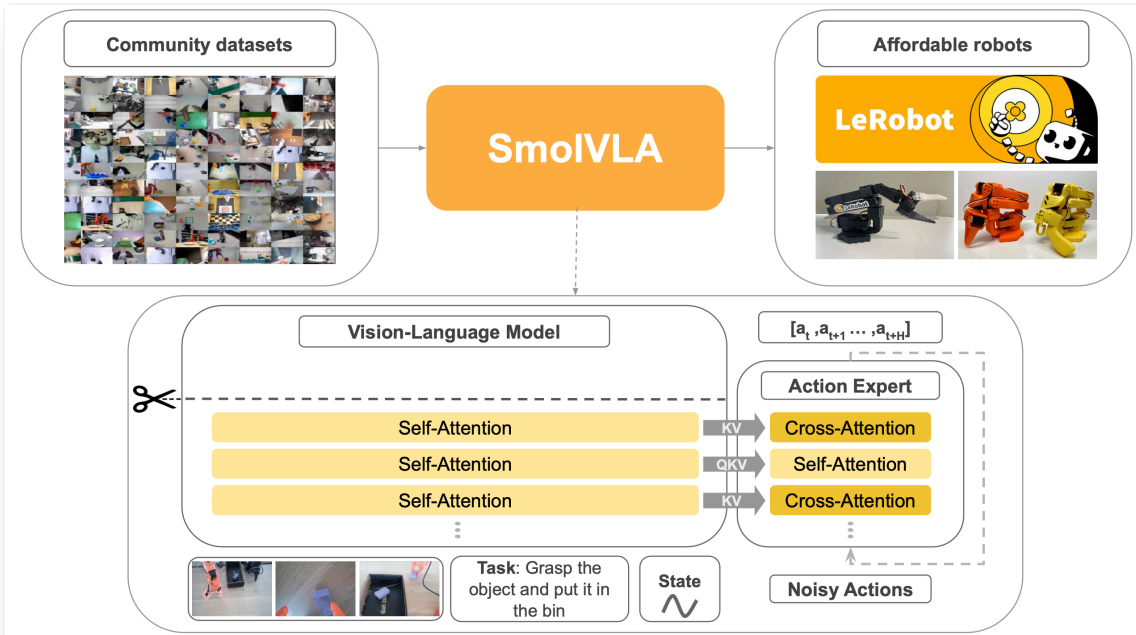
SmolVLA: 작게 만든 VLA

> *0.45B 경량 오픈소스 VLA — compact VLM + flow matching action expert*

항목	내용
규모	0.45B (π_0 3.3B의 1/7)
구성	compact VLM + flow matching action expert
입력	multi-view RGB + robot state + language
출력	action chunk

SmolVLA: 아키텍처

> *VLM과 Expert 모델로 이루어져 있음*



SmolVLA: VLM과 Expert 모델 구조 비교

> *SmolVLM2-500M 기준, 기본 설정 (`num_vlm_layers=16`, `expert_width_multiplier=0.75`)*

VLM (SmolVLM2-500M)

항목	값
레이어	16 / 32
hidden	960
heads	15 (kv: 5)

Vision Encoder + Connector 포함

Expert (Action LM)

항목	값
레이어	16
hidden	**720** (×0.75)
heads	15 (kv: 5)

Transformer decoder만 (임베딩 제거)

SmolVLA 모델 구성: VLM + Action Expert

VLM은 조건 표현 생성, Action Expert는 연속 제어 출력

VLM: 인지

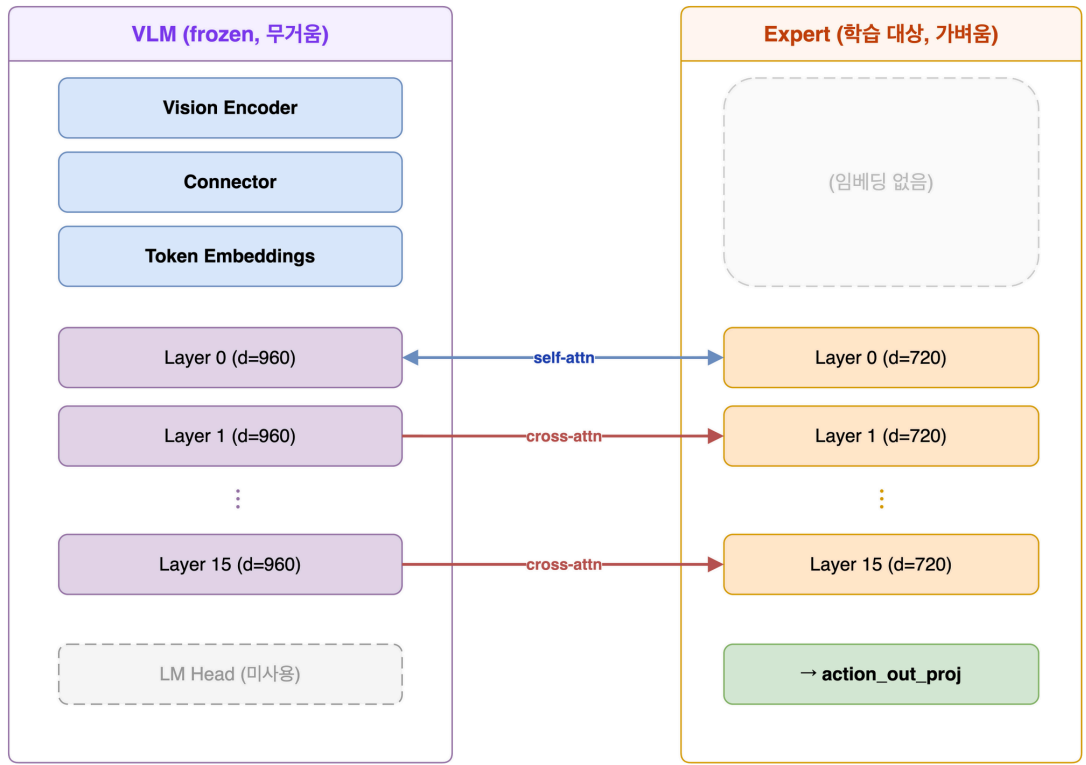
- SmolVLM-2 (frozen)
- 시각 + 언어 + 로봇 상태 통합
- action expert가 쓸 feature 생성

Action Expert: 행동

- VLM feature를 조건으로 받음
- future action chunk 출력
- “무엇을” → “어떻게 움직일까”로 변환

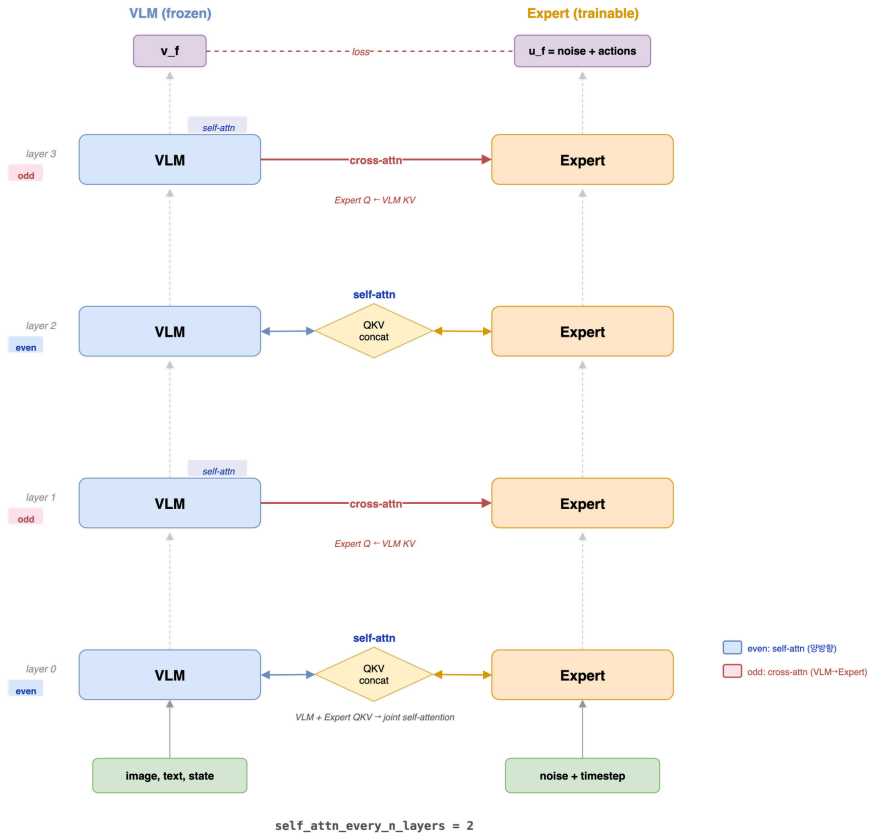
SmolVLA: 듀얼 스트림 아키텍처

> *VLM은 frozen 상태로 시각-언어 표현을 제공하고, Expert만 학습하여 action을 생성한다.*



SmolVLA: Attention 모드

> *레이어마다 VLM과 Expert가 정보를 교환하는 방식이 달라진다.*



`self_attn_every_n_layers = 2`

SmolVLA: Cross vs Self Attention 상세

Cross-Attention (대부분의 레이어)

Expert가 VLM 정보를 **읽기만** 하는 단방향 흐름

```
Expert Q ──┐
           ├→ Attention → Expert 업데이트
VLM K, V ──┘
```

- Expert의 Q는 자기 토큰에서 생성
- K, V는 VLM의 hidden state에서 가져옴
- "이미지에 뭐가 있고, 지시가 뭔지"를 참조

Self-Attention (일부 레이어)

두 스트림을 concat하여 **양방향 상호작용**

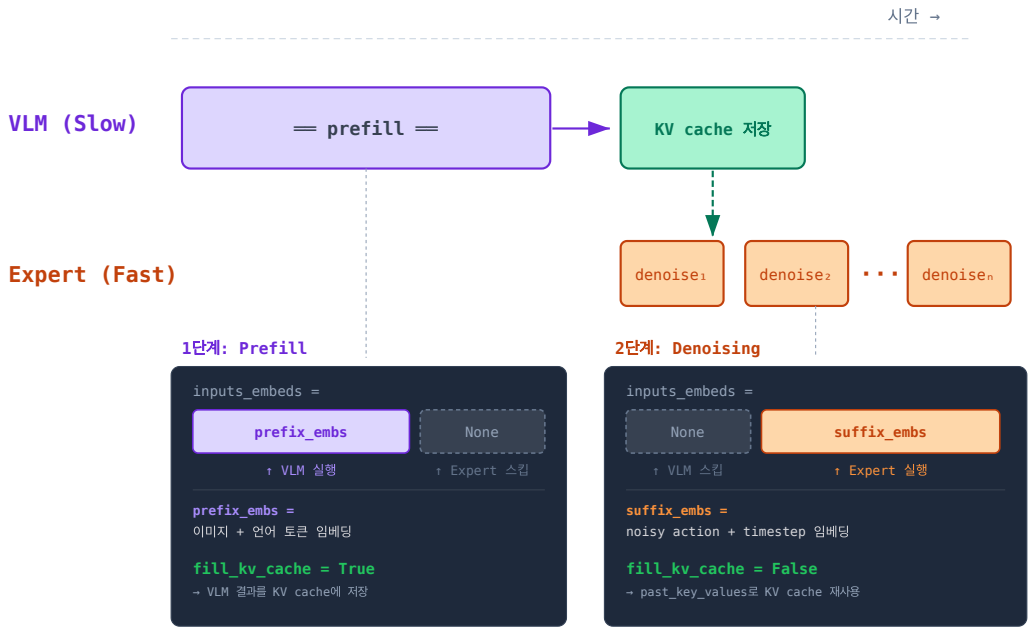
```
[VLM tokens | Expert tokens]
       ↕ 전체가 서로 attend
```

- Expert → VLM, VLM → Expert 모두 가능
- cross-attention만으로는 없는 역방향 피드백 제공

> 대부분 cross-attention + 주기적 self-attention으로 균형을 맞춘다

SmolVLA: 비동기 추론 루프

> *VLM은 한 번만 실행하고 KV cache로 저장, Expert는 여러 번 denoising step을 반복한다.*



- **Slow loop**: 새 이미지가 올 때만 VLM 처리
- **Fast loop**: Expert가 KV cache를 참조하며 action denoising 반복
- VLM 재계산 없이 Expert만 빠르게 동작 → **실시간 로봇 제어 가능**

SmolVLA: Context가 지도 역할을 한다

> *action expert는 noise만이 아니라 VLM의 맥락(Context)을 함께 봄*

```
context = self.vlm(images, "pick up the cup", robot_state)
action = self.action_expert(noise, t, context)
```

Context 없이

```
noise + t
  -> action expert
  -> 어느 행동 모드인지 모름
```

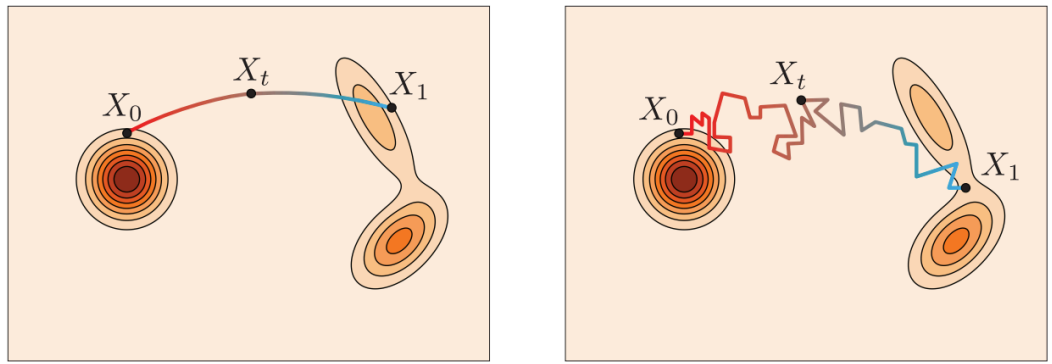
- 평균으로 무너지기 쉬움

Context와 함께

```
noise + t + image/instruction/state
  -> action expert
  -> 컵을 집는 방향장
```

- VLM이 장면+명령 해석 → 올바른 궤적으로 이동

Flow vs Diffusion: 경로의 차이



(a) Flow **(b)** Diffusion

Flow Matching

- $X_0 \rightarrow X_1$ 벡터장을 직접 학습
- 곧고 안정적인 경로 → 적은 step으로 생성

Diffusion

- stochastic denoising → 지그재그 경로
- step 많아질수록 제어 지연 증가

핵심: diffusion의 점진적 생성을 유지하되, 학습 목표를 **목표로 향하는 속도장**으로 바꿈

왜 10 steps가 1 step보다 나은가

다중 모드 분포를 한 번에 맞히면 평균으로 무너짐

1 step: `noise → action`

- 고차원 매핑을 한 번에 → 모드가 평균으로 섞임
- 예측 오차 = 최종 action error

10 steps: `noise → x0.1 → ... → action`

- 각 step은 locally correct하면 충분
- 적분이 경로 위로 계속 끌어당김 → 오차 보정

SmolVLA 입력과 출력

> *Multi-view RGB + Robot state + Language → VLM encoder → Action expert → Action chunk*

입력	의미
Multi-view RGB	top, side, wrist 등 여러 시점
Robot state	관절값, gripper 상태
Language instruction	수행할 task
출력: Action chunk	여러 timestep 제어값

SmolVLA 내부 구성 요소

> *VLM feature와 로봇 제어 공간을 맞추는 projection 계층들*

구성 요소	역할
State projector	robot state → VLM token 차원
Action projector	action → expert 차원
Feature projector	VLM feature → expert 차원
Visual token reduction	pixel shuffle, frame당 64 tokens
Layer skipping	VLM 32층 중 16층만 사용

SmolVLA 핵심 효율화 기법

VLM을 덜 쓰고 action expert를 정확히 쓰는 구조

기법	효과
Visual Token Reduction	1024 → 64 tokens/frame (16배 축소)
Layer Skipping	32층 중 16층만 사용 (50% 감소)
Interleaved Attention	SA/CA 교차로 상호작용 집중
Separate Action Expert	VLM = context, expert = action
Async Inference	VLM 1회 + expert 반복 → 실시간

Flow Matching Loss

> *Action expert는 VLM feature와 언어 명령을 조건으로, 로봇의 연속 action chunk를 만드는 벡터장을 학습합니다.*

$$\mathcal{L}^{\tau}(\theta) = \mathbb{E}_{p(A_t|o_t),\, q(A_t^{\tau}|A_t)} \left[\left\| v_\theta(A_t^{\tau}, o_t) - u(A_t^{\tau} \mid A_t) \right\|^2 \right]$$

noise가 섞인 action chunk A_t^{τ}에서 모델이 예측한 방향 v_θ와 실제 action으로 가야 하는 목표 방향 u의 차이를 줄이도록 학습합니다.

항	의미
A_t	데이터에 있는 실제 action chunk
A_t^{τ}	noise가 섞인 중가ㄴ action chunk
o_t	관찰, 언어 명령, 로봇 상태에서 온 조건
v_θ	action expert가 예측한 vector field
u	실제 action으로 가야 하는 목표 vector field

Flow Matching Loss 해석

action expert는 다음 토큰이 아니라 행동 궤적의 vector field를 배움

관점	해석
예측	action chunk가 움직여야 할 방향 예측
손실	예측 vs 목표 방향의 L_2 차이 최소화
결과	noise → 실제 로봇 제어 chunk
차이	discrete token이 아닌 continuous trajectory

과거엔 안 됐던 것

> *Physical AI가 주목받는 이유는 새로운 이름 때문이 아니라 **새로운 작업 성공 사례** 때문입니다.*

작업	왜 어려운가	VLA 시대의 변화
빨래 접기	변형 가능한 물체, 고정 기하 없음	연속 동작 policy로 성공 사례 증가
비닐 택배 처리	내용물마다 힘과 접촉점이 바뀜	장시간 물류 데모 등장
새로운 컵 집기	모양과 재질이 매번 다름	world knowledge로 물체 의미 추론

규칙을 쓰기 어려운 작업일수록 데이터 기반 generalist 접근의 장점이 커집니다.

주요 흐름은 비슷하다

> *세부 설계는 다르지만 큰 방향은 닮아 있음*

조직	대표	키워드
Physical Intelligence	π0, π0.5	VLA + flow matching
Google DeepMind	RT-2, Gemini Robotics	web knowledge + robot action
Figure AI	Helix	휴머노이드 VLA
NVIDIA	GR00T, Cosmos	foundation model + simulation
Stanford	ACT, ALOHA	action chunking + low-cost
HuggingFace	SmolVLA, LeRobot	compact open-source VLA

Fast-WAM

모델 구조와 PyTorch 구현

현재 관측으로 행동을 만들고, 학습에는 미래 영상을 활용한다

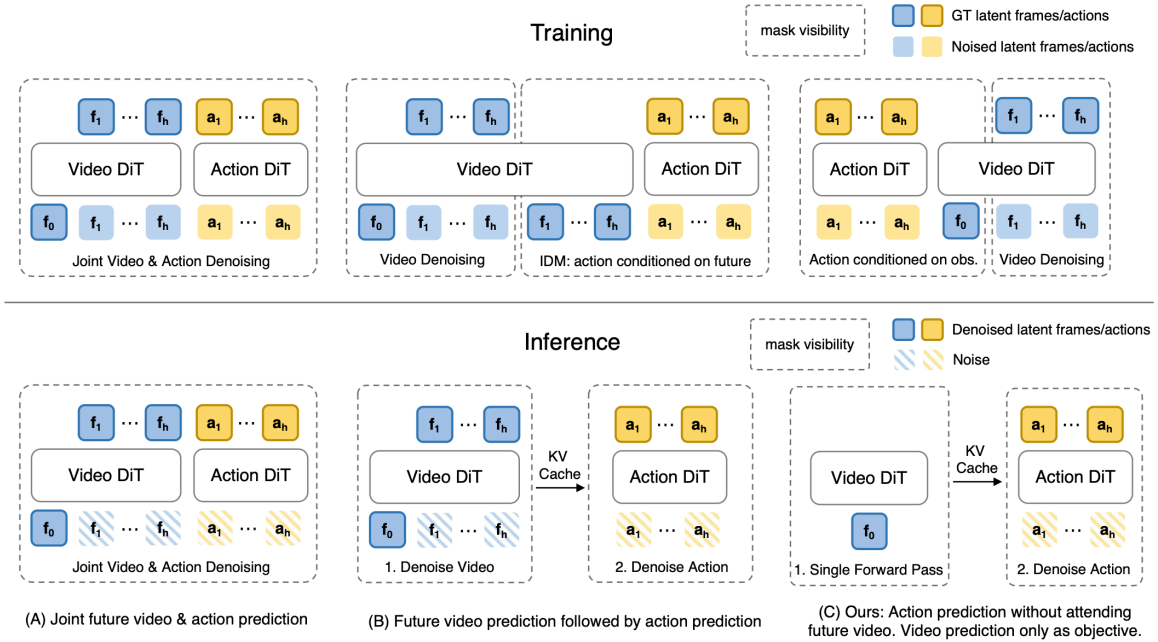
학습 목표와 설명 범위

1. 현재 프레임, 미래 영상, 행동 토큰의 역할을 구분한다.
2. MoT의 서로 다른 hidden dimension을 코드로 연결한다.
3. attention mask와 flow matching 학습을 구현한다.
4. KV cache를 사용하는 행동 추론과 공식 실행 절차를 이해한다.

전제: PyTorch의 tensor, Transformer attention, 역전파에 익숙한 독자.

코드는 공식 구현 해설과 교육용 축약 예제를 구분해 제시한다.

WAM의 세 가지 추론 방식



- **Joint:** 영상과 행동을 함께 생성한다.(A)
- **IDM:** 영상을 먼저 생성한 뒤 행동을 생성한다.(B)
- **Fast-WAM:** 현재 관측만으로 행동을 생성한다.©

모델 전체 구조

학습과 추론의 계산 경로

구분	학습	행동 추론
현재 프레임	깨끗한 latent	깨끗한 latent
미래 영상	noise를 섞어 복원 학습	토큰을 만들지 않음
행동	noise를 섞어 velocity 예측	Gaussian noise에서 반복 갱신
Video DiT	현재·미래 토큰 처리	현재 토큰 한 번 처리
출력	영상 손실 + 행동 손실	행동 chunk

영상 backbone은 추론에도 필요하다. 현재 관측을 읽어 층별 K/V를 제공한다. Action DiT에는 별도의 시각 입력이 없어, 장면 정보는 Video DiT의 층별 K/V로만 전달된다.

두 expert의 실제 차원

설정	Video DiT	Action DiT
hidden dimension	3072	1024
FFN dimension	14336	4096
Transformer 층 수	30	30
attention head 수	24	24
head dimension	128	128
Q/K/V의 전체 폭	3072	3072

행동 분기의 `hidden_dim / num_heads`로 head 크기를 계산하면 틀린다. Q/K/V projection이 공통 attention 공간을 만든다.

Wan2.2-TI2V-5B 기반 초기화

구성요소	Wan2.2에서 가져오는 것	학습
Video DiT	TI2V-5B DiT 가중치와 구조 그대로	학습
VAE	`Wan2.2_VAE`, latent 48채널	고정
텍스트 인코더	umT5-XXL, 출력 폭 4096	고정
Action DiT	Video DiT 가중치를 1024폭으로 축소	학습

- Action DiT는 같은 이름의 파라미터를 복사하고, shape가 다르면 선형 보가ㄴ 한다.
- 입력 폭이 줄어든 가중치에는 $\alpha = \sqrt{3072/1024}$를 곱해 출력 분산을 유지한다.
- Video DiT의 hidden dimension 3072, Action DiT의 hidden dimension 1024
- `action_encoder`, `head`는 대응 층이 없어 무작위로 초기화한다.

입출력 tensor 계약

변수	LIBERO 예시 shape	의미
`video`	`[B, 3, 9, 224, 448]`	두 카메라를 가로로 합친 9개 프레임
`action`	`[B, 32, 7]`	말단 자세 6차원 + gripper 1차원
`proprio`	`[B, T, 8]`	관측 상태, 모델은 첫 시점 사용
`context`	`[B, 128, 4096]`	미리 계산한 T5 임베딩
`context_mask`	`[B, 128]`	유효 언어 토큰 표시

`B` 는 batch 크기다. 행동 값은 데이터 통계로 정규화한다.

카메라·시간 전처리

VAE와 patch embedding

> *영상은 먼저 VAE로 압축하고, latent를 작은 patch로 나눈다.*

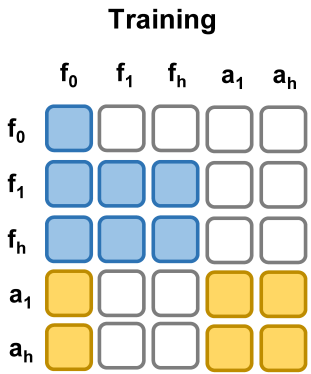
$$[B, 3, T, H, W] \longrightarrow [B, 48, T_z, H_z, W_z]$$

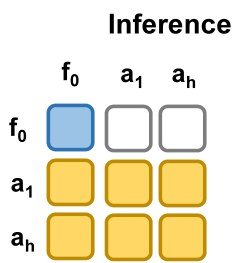
$$N_v = T_z \left(H_z/2\right) \left(W_z/2\right)$$

- Video DiT patch 크기는 `(1, 2, 2)`이다.
- patch embedding 출력은 `[B, N_v, 3072]`이다.
- 현재 프레임의 토큰 수는 `N_c = (H_z/2)(W_z/2)`이다.
- 실제 latent shape를 읽어 mask를 만든다. RGB 프레임 수를 대입하지 않는다.

언어와 로봇 상태의 조건 주입

학습·추론 attention mask





행은 Query, 열은 Key다. 행동은 현재 관측과 다른 행동을 읽는다. 현재 관측은 미래 영상이나 행동을 읽지 않는다.

mask를 블록 행렬로 읽기

Query \ Key	현재 C	미래 F	행동 A
현재 C	허용	차단	차단
미래 F	허용	허용	차단
행동 A	허용	차단	허용

- 현재 프레임 내부의 공간 토큰들은 서로 볼 수 있다.
- 미래 영상 토큰끼리, 행동 토큰끼리는 양방향 attention이다.
- C가 F를 읽으면, A가 C를 통해 미래 정보를 간접적으로 보게 된다.

따라서 **A–F 연결 차단과 C의 독립성**을 동시에 지켜야 한다.

구현 ① Boolean mask

> *교육용 구현. PyTorch SDPA에서는 `True`가 attention 허용을 뜻한다.*

```
def fastwam_mask(nc, nf, na, device="cpu"):
    nv = nc + nf
    m = torch.zeros(nv + na, nv + na,
                    dtype=torch.bool, device=device)
    m[:nc, :nc] = True       # C reads C
    m[nc:nv, :nv] = True     # F reads C, F
    m[nv:, :nc] = True       # A reads C
    m[nv:, nv:] = True       # A reads A
    return m
```

`nn.MultiheadAttention`의 Boolean mask 의미와 혼동하지 않는다.

구현 ② 공통 Q/K/V 공간

> *교육용 축약. expert의 hidden 폭과 attention 폭을 별도로 설정한다.*

```
n_heads, head_dim = 24, 128
attn_dim = n_heads * head_dim  # 3072
video_q = nn.Linear(3072, attn_dim)
action_q = nn.Linear(1024, attn_dim)
# K, V also project each expert to attn_dim.
video_out = nn.Linear(attn_dim, 3072)
action_out = nn.Linear(attn_dim, 1024)

# [B, N, 3072] -> [B, 24, N, 128]
q = q.view(B, N, n_heads, head_dim).transpose(1, 2)
```

공식 구현은 Q/K 정규화와 RoPE도 적용한다.

구현 ③ mixed attention

> *두 expert의 토큰을 공통 attention 공간에서 결합한다.*

```
# qv, kv, vv and qa, ka, va: [B, heads, N, d]
q = torch.cat([qv, qa], dim=2)
k = torch.cat([kv, ka], dim=2)
v = torch.cat([vv, va], dim=2)
y = F.scaled_dot_product_attention(
    q, k, v, attn_mask=mask[None, None], dropout_p=0.0)
yv, ya = y.split([nv, na], dim=2)
# Merge heads, then apply each expert's output projection.
```

MoT는 modality별 파라미터를 유지한다. 토큰별 top-k router는 없다.

DiT block의 나머지 연산

1. timestep 임베딩에서 shift, scale, gate를 만든다.
2. 정규화한 hidden state를 조절하고 Q/K/V를 계산한다.
3. mixed attention 결과를 expert별로 투영해 residual에 더한다.
4. 언어·상태 context에 cross-attention을 수행한다.
5. modulation을 적용한 FFN 결과를 residual에 더한다.

30개 층 모두 같은 연결 규칙을 사용한다. RoPE, 정규화, time modulation을 생략한 예제는 완전한 재현 모델이 아니다.

Action DiT의 입출력

flow matching의 목표

> *정규화된 행동 또는 미래 latent를 y, Gaussian noise를 ϵ으로 둔다.*

$$y_\sigma = (1-\sigma)y + \sigma\epsilon, \qquad u^* = \epsilon - y$$

$$\mathcal{L} = \lambda_a \mathcal{L}_a + \lambda_v \mathcal{L}_v$$

- 각 분기는 noisy 입력에서 목표 velocity와의 MSE를 줄인다.
- 영상과 행동의 noise 및 timestep은 각각 샘플링한다.
- 현재 프레임은 깨끗하게 유지하고 영상 복원 손실에서 제외한다.

구현 ④ noise와 target

> *교육용 기본 연산. 공식 scheduler는 timestep을 0–1000 척도로 전달한다.*

```
def make_noisy(y, sigma):
    eps = torch.randn_like(y)
    s = sigma.view(-1, *([1] * (y.ndim - 1)))
    return (1 - s) * y + s * eps, eps - y

video_t, video_target = make_noisy(video_latent, sigma_v)
video_t[:, :, :1] = video_latent[:, :, :1]
action_t, action_target = make_noisy(action, sigma_a)
# The network receives both noisy inputs and their times.
# Compute video loss only on [:, :, 1:].
```

`*_noise` 라는 함수명에도 불구하고 학습 target은 `noise - sample` 이다.

구현 ⑤ 손실과 padding

> *행동 차원에 대해 평균한 뒤, 유효 시점만 모아 sample별 손실을 구한다.*

```
error = (pred_action.float() - target.float()).square()
per_step = error.mean(dim=-1)              # [B, H]
valid = (~action_is_pad).to(per_step.dtype)
per_sample = (per_step * valid).sum(dim=1)
per_sample /= valid.sum(dim=1).clamp_min(1)
loss_a = (per_sample * scheduler_weight).mean()
loss = lambda_a * loss_a + lambda_v * loss_v
loss.backward()
```

영상 padding은 latent 시간축과 정렬해야 한다. RGB mask를 그대로 곱하지 않는다.

공동 학습이 현재 표현을 바꾸는 경로

> *행동이 미래 토큰을 보지 않아도 미래 영상 손실이 학습에 기여한다.*

- 현재·미래 토큰은 **같은 Video DiT 파라미터**를 사용한다.
- 미래 토큰은 현재 프레임의 K/V를 읽는다.
- 영상 손실의 gradient가 영상 expert와 현재 표현을 만드는 경로에 전달된다.
- 행동 expert는 공동 학습으로 바뀐 현재 표현을 읽게 된다.

공식 trainer는 Video·Action DiT와 proprio encoder를 학습하고, VAE·T5는 고정한다.

논문과 현재 scheduler의 차이

항목	논문 v1의 서술	확인한 공식 코드
학습 timestep	logit-normal로 서술	uniform을 shift 변환
학습 가중치	기본 FM 식	timestep별 가중 MSE
action shift	원 공개 설정과 구분 필요	기본 1.0
구형 공개 checkpoint 평가	원 설정 재현	`sigma_shift=5.0` 지정

$$u \sim U(0,1), \qquad \sigma = \frac{s\,u}{1 + (s-1)u}$$

재현 시 **코드 commit·checkpoint·scheduler 설정**을 함께 기록한다.

행동 추론과 KV cache

1. 최신 카메라 관측을 VAE로 인코딩한다.
2. 현재 프레임을 Video DiT에서 한 번 처리한다.
3. 각 층의 K/V를 저장한다.
4. 행동 noise를 반복 갱신하며 저장한 K/V를 재사용한다.
5. 정규화를 되돌리고 chunk 일부를 실행한 후 다시 관측한다.

현재 토큰은 행동과 미래에 의존하지 않으므로 반복 계산할 필요가 없다. **새 관측 또는 조건으로 재계획하면 cache도 갱신한다.**

구현 ⑥ cached attention

> *행동 Query만 계산하고, 현재 관측의 K/V와 현재 행동의 K/V를 합친다.*

```
# One action layer, head-first notation.
q_a, k_a, v_a = action_qkv(x_a)
k = torch.cat([video_cache[layer]["k"], k_a], dim=2)
v = torch.cat([video_cache[layer]["v"], v_a], dim=2)
y_a = F.scaled_dot_product_attention(
    q_a, k, v, dropout_p=0.0)
x_a = action_post_block(x_a, y_a, context, time)
```

이 경로에는 현재 관측과 행동만 남아 모든 key가 허용된다. 실제 코드에는 expert별 정규화·RoPE·gate·FFN이 포함된다.

구현 ⑦ 행동의 Euler 갱신

> *noise 쪽 $\sigma = 1$에서 data 쪽 $\sigma = 0$으로 이동한다.*

```
a = torch.randn(B, horizon, action_dim, device=device)
u = torch.linspace(1, 0, steps + 1, device=device)
sigmas = shift * u / (1 + (shift - 1) * u)
for i in range(steps):
    t = 1000 * sigmas[i]
    velocity = predict_with_cache(a, t, cache, context)
    delta = sigmas[i + 1] - sigmas[i]  # negative
    a = a + delta * velocity
```

`delta`는 음수다. target의 부호와 적분 방향을 함께 확인한다.

실행 가능한 교육용 실습

> *첨부 `fastwam_core_demo.py` 는 작은 tensor로 핵심 불변성을 확인한다.*

1. mask의 미래 정보 차단을 구현한다.
2. 미래 토큰을 바꿔도 행동 attention 출력이 같은지 확인한다.
3. 전체 attention과 cached action attention의 결과를 비교한다.
4. noise 혼합과 Euler 갱신의 부호를 확인한다.

```
python fastwam_core_demo.py
```

이 실습은 **구조 원리 검증용**이다. Wan 가중치와 로봇 정책을 학습하는 전체 구현은 공식 저장소를 사용한다.

공식 코드 탐색 순서

읽는 순서	파일·함수	확인할 내용
1	`configs/model/fastwam.yaml`	차원·scheduler·초기화
2	`fastwam.py: build_inputs`	shape·VAE·context
3	`fastwam.py: training_loss`	noise·target·loss
4	`mot.py: forward_joint_core`	expert 연결
5	`mot.py: prefill_video_cache_tensor`	층별 K/V 저장
6	`fastwam.py: infer_action`	cache·반복 갱신

모델 파일은 `src/fastwam/models/wan22/` 아래에 있다.

공식 환경과 backbone 준비

> *공식 안내 기준 Linux·NVIDIA CUDA 환경. 별도 실행 환경에서 설치한다.*

```
git clone https://github.com/yuantianyuan01/FastWAM.git
cd FastWAM
git checkout 7faa71108368fbb3b6885649f112af607427a2d4
conda create -n fastwam python=3.10 -y
conda activate fastwam
pip install torch==2.7.1+cu128 torchvision==0.22.1+cu128 \
  --extra-index-url https://download.pytorch.org/whl/cu128
pip install -e .
```

Wan2.2-TI2V-5B 가중치와 VAE·T5 파일을 확보해야 한다. 본 자료의 교육용 실습은 CUDA나 사전학습 가중치를 요구하지 않는다.

Action expert 초기화

> *공식 스크립트는 Wan 가중치를 action backbone의 크기에 맞게 보간한다.*

```
python scripts/preprocess_action_dit_backbone.py \
  --model-config configs/model/fastwam.yaml \
  --output checkpoints/ActionDiT_linear_interp_Wan22_alphascale_1024hdim.pt \
  --device cuda --dtype bfloat16
```

- shape와 backbone metadata를 로딩 시 확인한다.
- 행동 입출력처럼 modality 고유의 층은 별도로 초기화한다.
- 공개 전체 checkpoint 평가 설정은 backbone 선행 로드를 생략한다.

데이터 준비와 학습 실행

> *공식 LIBERO 데이터를 설정의 `dataset_dirs` 에 배치한다.*

```
python scripts/precompute_text_embeds.py \
  task=libero_uncond_2cam224_1e-4

bash scripts/train_zero1.sh 8 \
  task=libero_uncond_2cam224_1e-4 \
  model.compile_training_denoise=true
```

- T5 cache와 `context_mask` 가 가ㅌ은 task instruction에 대응해야 한다.
- `dataset_stats.json` 은 checkpoint와 함께 보관한다.
- 현재 task 설정은 epoch 기반이다. 논문의 step 수 재현은 별도 설정한다.

공개 checkpoint 평가

> *LIBERO 환경과 MuJoCo 3.3.2를 준비하고, checkpoint·통계 파일을 내려받는다.*

```
release=./checkpoints/fastwam_release
policy=libero_uncond_2cam224
python experiments/libero/run_libero_manager.py \
  task=libero_uncond_2cam224_1e-4 \
  ckpt="$release/$policy.pt" \
  EVALUATION.dataset_stats_path="$release/${policy}_dataset_stats.json" \
  EVALUATION.sigma_shift=5.0 \
  MULTIRUN.num_gpus=1
```

원 공개 checkpoint용 예시다. 최신 action shift 기본값과 구분한다. 평가 manager는 여러 task를 실행한다. 처음에는 단일 task부터 확인한다.

Python에서 행동 추론 호출

> *공식 모델과 processor를 준비한 뒤 사용하는 API 예시.*

```
result = model.infer_action(
    prompt="pick up the black bowl",
    input_image=image,        # [1, 3, 224, 448], [-1, 1]
    proprio=state,            # [1, 8], normalized
    action_horizon=32,
    num_inference_steps=10,
    sigma_shift=5.0,          # original released checkpoint
    text_cfg_scale=1.0,
    compile_action_infer=True,
)
action_normalized = result["action"]  # [32, 7], CPU float32
```

이미지 범위·카메라 순서·정규화는 공식 평가 processor와 일치시킨다.

로봇 제어 루프에 연결하기

1. 관측·상태·instruction을 전처리한다.
2. 32개 행동을 추론한다.
3. checkpoint와 짝을 이루는 통계로 행동을 복원한다.
4. 앞쪽 일부 행동을 실행한다.
5. 새 관측으로 다시 추론한다.

LIBERO 현재 설정은 `replan_steps=10`, `binarize_gripper=true`다.

행동 chunk 길이, 재계획 간격, 제어 주파수는 서로 다르다. 지연 시간의 역수를 로봇 관절 제어 주파수로 해석하지 않는다.

논문의 통제 실험

변형	영상 공동 학습	추론 미래 생성	LIBERO	RoboTwin 평균
Fast-WAM	있음	없음	97.6%	91.8%
Joint	있음	있음	98.5%	90.6%
IDM	있음	있음	98.0%	91.3%
영상 공동 학습 제거	없음	없음	93.5%	83.8%

영상 학습 제거에 따른 하락은 각각 **4.1%p, 8.0%p**다. 이 결과를 모든 환경에서 미래 예측이 불필요하다는 결론으로 확대할 수는 없다.

속도 수치를 읽을 때의 조건

출처	GPU·조건	수치
논문 v1	RTX 5090D V2 32GB, 논문 설정	190 ms
최신 README	RTX 4090, 최적화 전 / 후	190 / 110 ms
최신 README	H20, 최적화 전 / 후	470 / 210 ms

논문과 README의 숫자는 GPU·코드 경로가 달라 직접 합쳐 비교하지 않는다.

측정할 때 VAE·text encoding 포함 여부, warm-up, compile 비용, 행동 step 수와 입력 해상도를 함께 기록한다.

구현 검증에서 잡아야 할 오류

증상	확인할 지점
학습은 좋지만 추론이 무너짐	현재 토큰으로의 미래 정보 누설
attention shape 오류	hidden 폭과 Q/K/V 폭을 혼동했는지
행동이 발산함	velocity target과 Euler delta의 부호
정상적인 모양인데 행동이 틀림	카메라 순서·state·정규화 통계
cache 적용 후 출력이 달라짐	timestep·조건 의존성, RoPE, buffer 수명
공개 성능이 재현되지 않음	checkpoint별 shift와 simulator 버전

작은 tensor의 불변성 검증을 통과한 뒤 실제 데이터·정책 평가로 확장한다.

핵심 정리

- **구조:** 서로 다른 폭의 Video·Action DiT가 공통 attention 공간에서 연결된다.
- **학습:** 현재 프레임을 공유하며 영상·행동의 velocity를 함께 학습한다.
- **추론:** 현재 관측의 층별 K/V를 한 번 만들고 행동만 반복 갱신한다.
- **구현:** mask 방향, attention 폭, scheduler 부호, 정규화가 핵심이다.

전체 모델 재현은 공식 저장소를, 원리 학습은 첨부 tensor 실습을 사용한다.

출처와 재현 기준

- Fast-WAM 논문 v1, 2026-03-17
- 공식 프로젝트와 구조 그림
- 공식 GitHub 저장소
- 공개 checkpoint
- LIBERO 데이터셋

코드 기준: `7faa71108368fbb3b6885649f112af607427a2d4`

확인일: 2026-10-02. 각 슬라이드의 코드 링크는 이 commit으로 고정했다. 구조·mask 그림은 공식 프로젝트에서 내려받은 저자 원본이다.

고객 문의 샘플

> Hi, I was charged twice for invoice #4411. Please refund the duplicate today.

질문 ID	질문	선택지 순서
`team`	Which team should handle this?	billing · sales · technical
`refund`	Does the customer ask for a refund?	true · false
`urgency`	How urgent is this?	0 · 1 · 2

기대 판단: `billing` / `true` / `2` (today)

- `choice` : 키 정렬 · `noul` : 참/거짓 · `score` : 0부터 번호
- 질문 3개, 선택지 수 `[3, 2, 3]` → 총 8개

템플릿으로 바꾸기 전: 고객 문의와 질문 설정

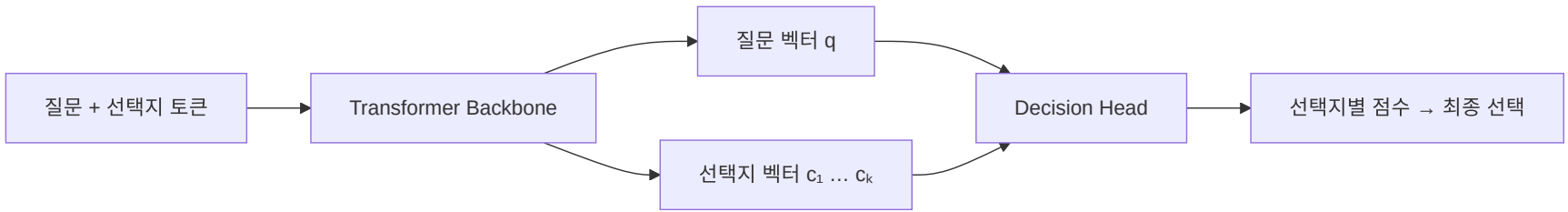
```
"Hi, I was charged twice for invoice #4411. Please refund the duplicate today.",
{
    "team": {
        "type": "choice",
        "instructions": "Which team should handle this?",
        "criteria": {
            "billing": "invoices, payments, refunds",
            "technical": "bugs, outages, errors",
            "sales": "pricing, new plans",
        },
    },
    "refund": {"type": "noul", "instructions": "Does the customer ask for a refund?"},
    "urgency": {
        "type": "score",
        "instructions": "How urgent is this?",
```

고객 문의 → `STATE` / **질문 설정** → `SCHEMA FIELDS`

Decision Head: 선택지별 점수를 만드는 채점기

> *질문의 맥락 벡터와 각 선택지의 표현 벡터를 받아 선택지마다 점수 하나를 계산한다*

수능 비문학 문제처럼, **지문·질문의 요약 표현**과 **보기별 핵심 표현**을 비교하는 과정



여기서는 **질문과 선택지 표현을 따로 추출해 채점하는 구조**를 예로 든다.

1. 선택지마다 벡터를 하나씩 만든다

> *보기 4개를 한 벡터로 합치지 않고, 보기별 대표 벡터 4개를 유지한다*

입력	형태 예시	의미
질문 벡터 q	`[1, 768]`	질문 영역의 대표 벡터
선택지 행렬 C	`[4, 768]`	선택지별 대표 벡터를 행으로 쌓음
점수 벡터	`[4]`	A·B·C·D 각각의 점수

```
C의 1행 = 선택지 A 벡터 [768]     q와 비교 → 점수 A
C의 2행 = 선택지 B 벡터 [768]     q와 비교 → 점수 B
C의 3행 = 선택지 C 벡터 [768]     q와 비교 → 점수 C
C의 4행 = 선택지 D 벡터 [768]     q와 비교 → 점수 D
```

벡터를 별도로 만든다는 뜻이지, 서로의 문맥을 전혀 보지 않는다는 뜻은 아니다.

2. Slicing: 토큰 구간을 나눈다

> *예시 입력:* `[질문: 오늘 날씨 어때?] [A: 맑음] [B: 비옴] [C: 흐림]`

Transformer 출력 H의 형태: `[전체 토큰 수, hidden 차원]` = `[12, 768]`

구간	토큰 인덱스	슬라이싱	결과 형태
질문	0~5	`H[0:6]`	`[6, 768]`
선택지 A	6~7	`H[6:8]`	`[2, 768]`
선택지 B	8~9	`H[8:10]`	`[2, 768]`
선택지 C	10~11	`H[10:12]`	`[2, 768]`

슬라이싱의 끝 인덱스는 포함하지 않는다. `H[6:8]` 은 토큰 6·7의 벡터다.

인덱스는 설명용 가정이다. 실제 구간은 토크나이저 결과·구분자·마스크에 맞춰 추적한다.

3. Pooling: 구간마다 대표 벡터 하나 추출

Mean Pooling

구간 안의 유효 토큰 벡터를 평균

$$c_A = \frac{H_6 + H_7}{2}$$

```
c_A = H[6:8].mean(dim=0)
```

패딩·불필요한 구분자는 마스크로 제외

추출 결과: 질문 q `[1, 768]` + 선택지 C `[3, 768]`

[CLS] 사용 여부와 풀링 방식은 백본 구조 및 학습 방식에 맞춰 선택한다.

Last-Token Pooling

구간의 마지막 유효 토큰 벡터 사용

$$c_A = H_7$$

```
c_A = H[7]
```

Causal 모델에서 앞선 토큰 문맥을 활용

4. 방식 A: 투영 후 내적으로 점수 계산

> *학습 가능한 선형 변환으로 비교 공간을 만든 뒤, 선택지마다 내적한다*

$$\tilde{q} = qW_q, \qquad \tilde{C} = CW_c$$

$$s = \tilde{q}\tilde{C}^\top \quad \Rightarrow \quad s_i = \tilde{q} \cdot \tilde{c}_i$$

연산	형태 예시
질문 투영	`[1, 768] → [1, d']`
선택지 투영	`[4, 768] → [4, d']`
점수 계산	`[1, d'] @ [d', 4] → [1, 4]`

내적은 **방향과 벡터 크기 모두**에 영향을 받는다. 정규화한 벡터의 내적은 코사인 유사도다.

유사도 채점만으로 전체 Cross-Attention 연산을 수행한 것은 아니다.

5. 방식 B: Concat + MLP로 채점

> *질문 벡터와 선택지 벡터를 연결하고, 비선형 신경망으로 점수 하나를 만든다*

$$x_i = [q; c_i]$$

$$s_i = W_2\,\mathrm{GELU}(W_1 x_i + b_1) + b_2$$

```
질문 q [768] + 선택지 cᵢ [768]
                    ↓ Concat
               xᵢ [1536]
                    ↓ Linear → GELU → Linear
               점수 sᵢ [1] (원소 1개)
```

- 같은 채점 MLP를 모든 선택지에 적용
- 질문 벡터를 선택지 수만큼 확장해 **여러 선택지를 한 배치로 처리 가능**
- 내적보다 유연한 비선형 관계를 학습하는 구조

5-1. s_i도 벡터 아닌가요?

> *구현에서는 원소 하나짜리 텐서 `[1]`, 수학적으로는 점수 하나인 스칼라로 다룬다*

```
xᵢ [1536] → Linear(1536, 256) → GELU → hᵢ [256]
                                          ↓ Linear(256, 1)
                                       sᵢ [1]
```

마지막 Linear가 **256개의 특징을 점수 하나로 압축**한다. (256은 예시)

$$W_2 \in \mathbb{R}^{1\times 256}, \quad s_i = \sum_{j=1}^{256} W_{2,1j} h_{i,j} + b_2$$

구분	예시	의미
중간 벡터 h_i	`[0.2, -0.7, …]` · `[256]`	질문과 선택지 쌍의 특징
개별 점수 s_i	`[2.1]` · `[1]`	i번째 선택지의 점수 하나
스칼라로 표현한 s_i	`2.1` · `[]`	같은 점수, 크기 1인 축을 제거

5-2. 선택지 4개를 한 번에 채점하면?

> *개별 점수 s_i 네 개를 모으면 전체 점수 벡터 s가 된다*

```
질문을 4행으로 확장 [4, 768] + 선택지 [4, 768]
                          ↓ Concat (특징 축)
                       X [4, 1536]
                          ↓ 같은 MLP를 각 행에 적용
                     scores [4, 1]
                          ↓ squeeze(-1)
                        s [4]
```

```python
scores = mlp(X)                # [4, 1]: [[2.1], [0.4], [-1.2], [0.8]]
s = scores.squeeze(-1)        # [4]:    [2.1, 0.4, -1.2, 0.8]
probs = torch.softmax(s, dim=-1)  # [4]: 선택지 사이의 확률
```

`squeeze(-1)`은 값은 유지하고 마지막 크기 1인 축만 제거한다.

문제 B개를 함께 처리하면 같은 흐름이 $[B, 4, 1] \rightarrow [B, 4]$가 된다.

6. Softmax: 점수를 선택지별 확률로 변환

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^{k} \exp(s_j)}$$

선택지	원시 점수(Logit)	Softmax 결과
A	2.1	**82.00%**
B	0.4	14.98%
C	-1.2	3.02%

확률 합은 100%이며, 가장 높은 점수의 선택지 A를 고른다.

$$\hat{y} = \arg\max_i p_i = \arg\max_i s_i$$

정답 인덱스만 필요하면 Softmax 없이 `argmax(logits)` 를 사용해도 된다.

Softmax 82%는 주어진 선택지 사이의 모델 확률이며, 실제 정답률 82%를 보장하지 않는다.

7. 전체 흐름을 코드로 연결

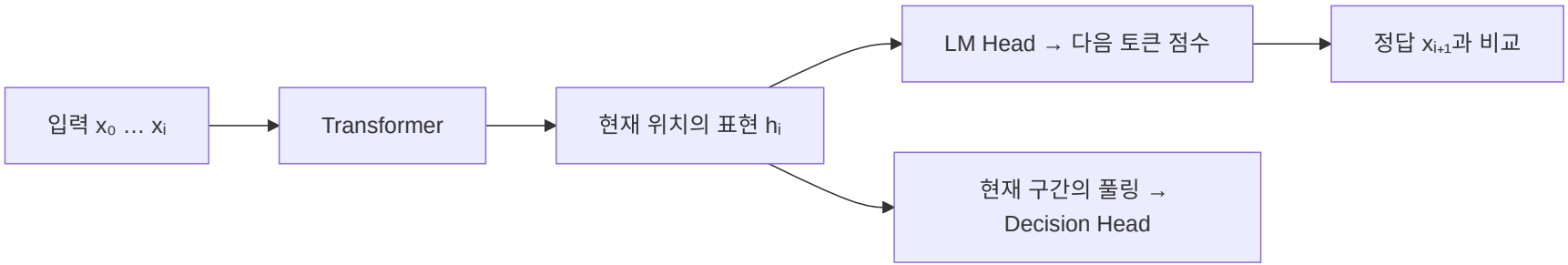
```
# H: [12, 768], 배치 차원 생략; 토큰 구간은 설명용
q = H[0:6].mean(dim=0, keepdim=True)  # [1, 768]
C = torch.stack([
    H[6:8].mean(dim=0),               # 선택지 A
    H[8:10].mean(dim=0),              # 선택지 B
    H[10:12].mean(dim=0),             # 선택지 C
])                                   # [3, 768]

# q_proj, c_proj: 학습된 Linear(768, d_prime)
q_hat = q_proj(q)                    # [1, d_prime]
C_hat = c_proj(C)                    # [3, d_prime]
logits = (q_hat @ C_hat.T).squeeze(0) # [3]
probs = torch.softmax(logits, dim=-1)
```

토큰 구간 → 풀링 → 선택지별 점수 → 최종 선택

답변을 토큰별로 생성할 필요는 없지만, 전체 계산에는 **백본·풀링·투영 또는 MLP**가 포함된다.

8. 다음 토큰 예측과 표현 추출은 위치 기준이 다르다

> *h_i는 현재 토큰 x_i까지 읽은 표현이고, LM Head는 이를 이용해 다음 토큰 x_{i+1}을 예측한다.*



목적	가져오는 가ㅅ	위치 처리
다음 토큰 학습	위치 i의 logits	정답 토큰 $i+1$과 정렬
질문·선택지 표현 추출	해당 구간의 hidden states	원래 토큰 인덱스 유지

```
# 다음 토큰 손실을 위한 정렬 예시
shift_logits = logits[..., :-1, :]
shift_labels = labels[..., 1:]
```

Shift는 예측과 정답을 맞추는 과정이다. Hidden State 자체를 이동시키는 연산이 아니다.

9. 선택지 벡터는 해당 토큰 위치에서 추출

> *예시: 질문은 토큰 **0~5**, 선택지 A의 `[맑, 음]` 은 토큰 **6~7***

Hidden State	읽을 수 있는 문맥	표현 추출 관점
h_5	질문 끝까지	선택지 A를 아직 읽지 않음
h_6	질문 + `맑`	선택지의 첫 토큰까지 읽음
h_7	질문 + `맑` + `음`	선택지 끝까지 읽은 위치

```
# 원래 구간 [6, 8)을 그대로 사용
c_A_mean = H[6:8].mean(dim=0)  # h6, h7의 평균
c_A_last = H[7]                # 선택지의 마지막 토큰
```

- 구간을 한 칸 당긴 `H[5:7]` 은 **질문 끝을 포함하고** `음` **을 빠뜨린다.**
- 마지막 위치를 한 칸 당긴 `H[6]` 도 `음` **을 아직 읽지 않은 표현**이다.

다음 글자를 맞힐 때는 다음 위치의 정답과 비교하고, 읽은 선택지를 표현할 때는 그 위치를 그대로 쓴다.

10. 선택지 벡터는 문맥화된 표현이다

> *h_7은 선택지 A만의 고정 벡터가 아니라, 질문 뒤에서 선택지 A를 읽은 상태다.*

앞선 질문	같은 선택지	문맥에서의 의미
백설공주가 먹고 쓰러진 것은?	사과	과일
친구에게 잘못했을 때 해야 하는 것은?	사과	잘못을 인정하고 용서를 구하는 행위

```
질문 토큰 → 선택지 A 토큰 → h_A
              앞선 문맥을 참조한 선택지 표현
```

- 질문과 선택지의 관계를 백본에서부터 반영할 수 있음
- Head는 이 표현을 정답 점수로 연결하도록 학습
- **문맥이 포함되었다고 해서 모델이 이미 정답 판단을 마쳤다는 뜻은 아님**

11. 생성형 LLM과 Decision Head: 세 가지 비교

비교	자유 텍스트 생성	LLM 라벨 Logit 채점	선택지 표현 기반 Head
출력	답변 문자열	A/B/C 등의 점수	선택지 슬롯별 점수
추론	Prefill + 후속 생성	1회 순전파 가능	1회 순전파 가능
점수 기준	다음 토큰·시퀀스 확률	라벨 토큰의 LM logits	풀링 표현 + 학습된 채점기
결과 연결	텍스트 파싱	라벨 ID ↔ 선택지 매핑	슬롯 ID ↔ 선택지 매핑
고려 사항	형식·출력 길이	토큰화·라벨·프롬프트	구간 추출·풀링·학습 방식

핵심 차이는 순전파 횟수만이 아니라, 어떤 출력 공간과 학습 목표로 점수를 만드는가이다.

일반적인 생성형 LLM은 매 생성 단계에서 전체 어휘의 토큰 점수를 계산하지만, 여기서 설명하는 Decision 모델은 주어진 선택지별 점수를 직접 계산한다.

속도·정확도는 백본 크기, 입력 길이, 선택지 수, 배치와 구현을 맞춰 측정한다.

12. 일반 LLM도 한 번에 선택지를 채점할 수 있다

> *프롬프트 마지막 위치의 LM logits에서 라벨 토큰 점수만 읽는다.*

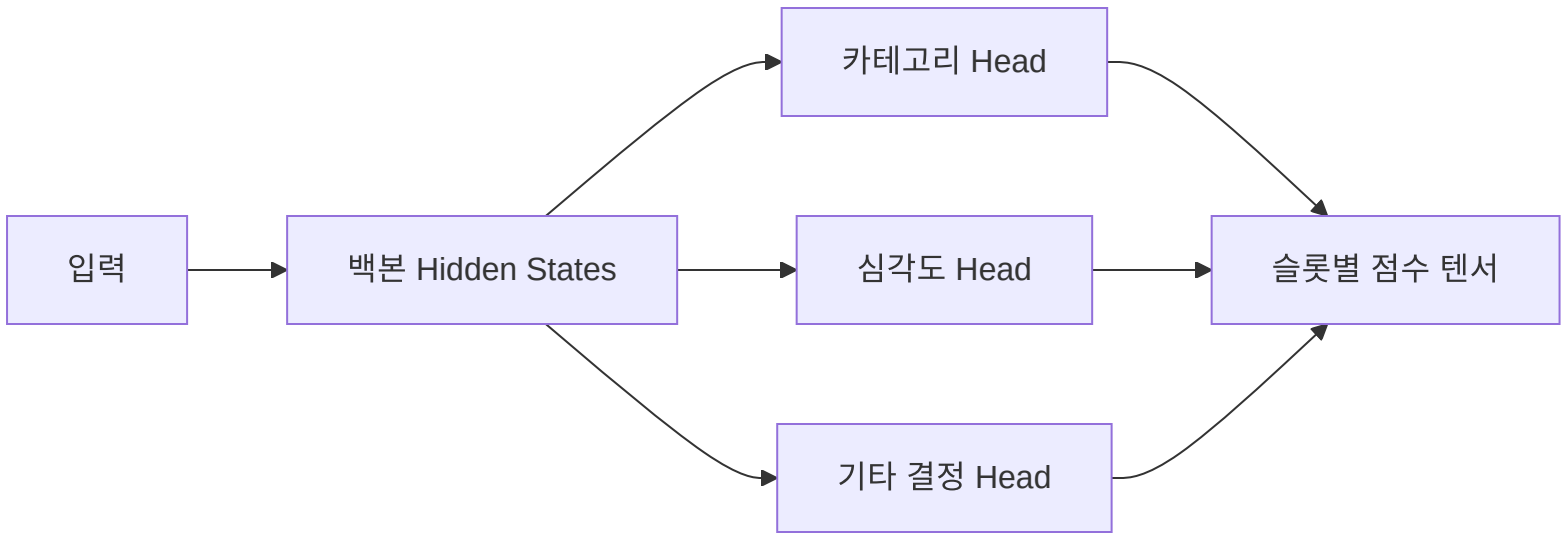
```
# 마지막 입력 위치의 전체 어휘 점수: [vocab_size]
z = outputs.logits[0, -1]
# A/B/C/D가 이 답변 위치에서 가ㄱ가ㄱ 한 토큰임을 확인한 경우
label_logits = z[label_token_ids]
probs = torch.softmax(label_logits, dim=-1)
answer_idx = label_logits.argmax()
```

- 한 토큰 답변은 Prefill 결과에서 선택 가능; 추가 생성 루프가 필수는 아님
- `max_new_tokens=1` 은 길이 제한이며, 허용 라벨 제한은 별도 설정
- 라벨 점수도 **전체 프롬프트를 읽은 은닉 상태**에서 계산됨
- 라벨·순서 편향을 평가해야 하지만, 단순 문자 빈도만 비교하는 것은 아님

출처: Transformers Generation · Llama 출력·logits

13. 구조화된 출력과 다중 결정

> *여러 결정 지점의 표현을 읽는 구조라면, 한 번의 백본 계산 후 여러 Head를 적용할 수 있다.*



- 슬롯별 점수는 자유 텍스트 파싱 없이 후속 코드에 연결 가능
- 구간·마스크·선택지 매핑 오류와 정답 오류까지 사라지는 것은 아님
- 일반 LLM도 독립 질문 배치·여러 위치 채점 등으로 다중 결과를 병렬 처리 가능
- **앞선 결정 결과를 알아야 다음 결정이 가능한 작업**은 추가 단계가 필요할 수 있음

생성형도 허용 토큰 제한·구조화된 디코딩으로 출력 형식 오류를 줄일 수 있다.

14. 계산 효율: LM Head를 줄이는 효과와 한계

항목	LLM 라벨 채점	선택지 표현 기반 Head
출력층	보통 hidden → 전체 어휘 logits	hidden 표현 → 선택지 점수
출력 계산	한 위치 기준 약 $O(dV)$	내적 채점 약 $O(kd')$ + 투영·풀링
최적화	마지막 위치만 계산; 구현에 따라 라벨 행만 계산 가능	작은 출력 공간; Head 크기는 설계에 따라 다름
KV 캐시	1회 채점만 하면 유지 불필요	후속 생성이 없으면 유지 불필요

d: hidden 차원, V: 어휘 수, k: 선택지 수, d': 투영 차원

출력층 절감이 전체 추론 비용의 동일 비율 절감을 뜻하지는 않는다.

긴 입력에서는 백본 계산이 지배적일 수 있다. Head 크기·지연·메모리·정확도를 함께 측정한다.

구현 참고: Llama의 logits_to_keep

15. 순서를 바꾸면 평가 문맥도 바뀐다

> *표준 Causal Attention은 현재 위치와 앞선 토큰만 참조한다.*

```
질문 [0~5] → 선택지 A [6~7] → 선택지 B [8~9]
                    h7                  h9
             질문 + A 참조       질문 + A + B 참조
```

순서	A의 대표 벡터가 보는 문맥	B의 대표 벡터가 보는 문맥
질문 → A → B	질문 + A	질문 + A + B
질문 → B → A	질문 + B + A	질문 + B

순서를 바꾸면 같은 선택지라도 참조 가능한 정보와 위치가 달라진다.

- 이 정보 비대칭은 점수·최종 선택이 달라지는 원인이 될 수 있음
- 앞쪽 선호(Primacy)·뒤쪽 선호(Recency)는 모델과 데이터에 따라 달라짐
- 더 많은 앞선 문맥을 본다고 뒤쪽 선택지가 항상 유리한 것은 아님

16. 블록 마스크: 질문은 공유하고 선택지는 격리

> *각 선택지가 질문과 자기 구간만 참조하도록 허용한다.*

읽는 구간 ↓ / 참조 대상 →	질문	선택지 A	선택지 B
질문	허용*	차단	차단
선택지 A	허용	허용*	차단
선택지 B	허용	차단	허용*

* 각 구간 내부에서는 기존 causal 규칙을 유지하는 예시

```
질문 ──→ 선택지 A → c_A
     └─→ 선택지 B → c_B        A ↔ B 정보 전달 차단
```

- 모든 레이어에서 적용해 다른 선택지로부터의 정보 유입을 차단
- 질문 구간도 선택지를 보지 않아야 질문을 통한 우회 전달을 막음
- **블록 마스크는 정보 비대칭을 줄이지만, 위치 차이까지 없애지는 않음**

17. 위치 인덱스도 같은 규칙으로 부여

> *질문 다음 위치에서 선택지별 position IDs를 다시 시작한다.*

구간	실제 입력 배열 인덱스	부여할 position IDs 예시
질문	`0, 1, 2, 3, 4, 5`	`0, 1, 2, 3, 4, 5`
선택지 A	`6, 7`	`6, 7`
선택지 B	`8, 9`	`6, 7`
선택지 C	`10, 11, 12`	`6, 7, 8`

같은 선택지의 토큰은 배치 순서를 바꿔도 같은 위치 번호를 받는다.

- 선택지 안의 토큰 순서는 유지; 모든 토큰을 동일 번호로 만들지 않음
- 블록 마스크와 함께 사용해 선택지 간 참조와 순서별 위치 차이를 줄임
- RoPE·상대 위치 편향·캐시 등 모델 구현과의 호환성 확인 필요

배열 인덱스는 슬라이싱 위치, position IDs는 위치 표현에 쓰는 번호다. 둘은 구분한다.

18. 순서 셔플 학습과 순열 검증

> *학습 시 선택지를 섞고, 평가 시 원래 선택지 ID로 되돌려 결과를 비교한다.*

```
원래 순서: [A, B, C]  → 점수 [s_A, s_B, s_C]
섞은 순서: [C, A, B]  → 점수 [s'_C, s'_A, s'_B]
                              ↓ 원래 ID로 재정렬
                         [s'_A, s'_B, s'_C]
                         원래 점수 [s_A, s_B, s_C]와 비교
```

방법	기대 효과	확인할 점
셔플 학습	특정 순서에 대한 과적합 완화	정답 인덱스·라벨도 함께 갱신
순열 평가	실제 순서 민감성 측정	점수 변화·정답률·선택 일치율

예: 정답이 A라면 `[A, B, C] → [C, A, B]`로 섞을 때 정답 인덱스도 `0 → 1`로 변경한다.

셔플 학습만으로 순서 독립성이 보장되지는 않는다. 여러 순열에서 결과가 일관적인지 검증한다.

19. Calibration: 확신도와 실제 정답률의 간격 줄이기

> *확률이 비슷한 예측을 모았을 때, 그 확률과 관측된 정답률이 가까워지도록 보정한다.*

예시 집단	평균 확신도	실제 정답률	해석
보정 전의 과확신 예시	99%	70%	틀릴 가능성을 과소평가
잘 보정된 집단의 예시	약 90%	약 90%	확신도와 정답률이 비슷함

두 행은 개념 설명을 위한 별도 집단 예시다.

- **정확도:** 정답을 얼마나 자주 고르는가?
- **보정:** 제시한 확률이 실제 성공 빈도와 얼마나 잘 맞는가?
- 높은 정확도와 좋은 보정은 서로 다른 성질

개별 답의 정답 여부를 보장하거나 확률을 완벽하게 1:1 일치시키는 작업은 아니다.

출처: Guo et al., On Calibration of Modern Neural Networks

20. 언제? 학습 후, 배포 전 별도 데이터로 보정

데이터	용도
학습셋	백본·Decision Head 학습
Held-out 보정셋	정답 라벨과 logits로 보정 계수 추정
별도 평가셋	보정 효과와 운영 임계값의 성능 확인

보정 데이터는 모델 가중치 학습에 쓰지 않고, 운영 입력을 대표하도록 구성한다.

모델·프롬프트·데이터 분포가 달라지면 보정 상태도 다시 점검한다.

출처: Temperature Scaling의 사후 보정 절차

21. Temperature Scaling: logits의 크기 조절

> *가중치를 다시 학습하는 대신, 양수 온도 T를 피팅한다.*

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \qquad T > 0$$

온도	확률 분포에 미치는 영향
$T = 1$	원래 Softmax
$T > 1$	분포를 완만하게 → 과확신 완화에 활용
$0 < T < 1$	분포를 더 뾰족하게 → 과소확신 조정에 활용

기본적인 피팅 목표: 보정셋 정답의 음의 로그우도(NLL)를 최소화

$$T^* = \arg\min_{T>0} \; -\sum_n \log p_{y_n}(T)$$

한 결정의 모든 선택지에 같은 양수 T를 적용하면 순위와 argmax는 유지된다.

출처: Guo et al., Temperature Scaling

22. Unsloth 구현과 운영 임계값

> `FastDecisionModel.calibrate(...)`는 *logits*를 얻고 보정 설정을 갱신한다.

- `decision.py`는 Clef 여부에 따라 보정 경로를 분기
- 질문 유형별 온도를 다루므로 항상 전역 온도 하나만 쓰는 것은 아님
- 보정된 설정을 모델과 함께 저장하고 서빙에 반영

```
보정된 선택 확률 ≥ τ  → 자동 처리 후보
보정된 선택 확률 < τ  → 상담원 검토 또는 추가 판단
```

운영에서 확인할 것	목적
확신도 구간별 정답률·ECE 등	확률과 실제 결과의 차이 확인
임계값별 오류율·자동 처리 비율	오류 비용과 자동화 범위의 균형
운영 데이터 분포 변화	기존 보정의 유효성 점검

예시 임계값 0.8을 그대로 믿지 않고, 업무별 오류 비용과 검증 결과로 정한다.

구현 확인: Unsloth decision.py — calibrate / evaluate (2026-10-25 확인)